UNIT II: Problem Solving : State Space Approach and Search Strategies

CO2-To Design state-space search models for real-world problems and implement appropriate heuristic, local, and online search algorithms to obtain effective solutions

Mrs. Priyadarshani Doke

INDEX

- State Space Search
- Informed (Heuristic) Search Strategies:
- Introduction to:
  - Greedy BFS, A* Search, Iterative-deepening, Heuristic Functions.
- Local Search and Optimization Problems:
  - Hill-climbing search, Simulated annealing, Local beam search. Online Search Agents and
- Unknown Environments: Online search problems.
- Case Study: Warehouse robots (Amazon Kiva) and self-driving cars, Logistics and Routing: Traveling Salesman Problem, Google DeepMind – AI for Energy Efficiency in Data Centers

SOLVING PROBLEMS BY SEARCHING

- Goal-based agents can succeed by considering future actions and desirability of their outcomes.
- Problem solving agent is a goal-based agent that decides what to do by finding sequences of actions that lead to desirable states

SOLVING PROBLEMS BY SEARCHING

- **States:** a problem is divided into a set of resolution steps from the initial state to the goal state
- A problem is defined by its elements and their relations.
- In each instant of the resolution of a problem, those elements have specific descriptors (How to select them?) and relations.
- A state is a representation of those elements in a given moment.
- Two special states are defined:
  - Initial state (starting point)
  - Final state (goal state)

SOLVING PROBLEMS BY SEARCHING

- A **successor function** is needed to move between different states.
- A successor function is a description of possible actions, a set of operators. It is a transformation function on a state representation, which convert it into another state.
- The successor function defines a relation of accessibility among states.
- Given a particular state x, SUCCESSOR-FN(x) returns a set of (action, successor) ordered pairs

State space Search

- **State Space Search** is used to solve problems by finding different possible states and their transitions. In simple terms it's like finding the best route to goal by trying different paths.
- The **state space** is the set of all states reachable from the initial state.
- It forms a graph (or map) in which the nodes are states and the arcs between nodes are actions.
- The core idea behind state space search is to think of the problem as a graph where:
- **Nodes** represent different states of the problem.
- **Edges** represent transitions or actions that change one state into another.

State space Search

Terminologies consist of:

- **State:** A specific configuration of the problem.
- **Initial State:** Starting point of the search.
- **Goal State:** Desired end configuration.
- **Transition:** An action that changes one state to another.
- **Path:** A sequence of states connected by transitions.
- **Search Strategy:** Method used to explore the state space.

Method for exploring the state space. Common strategies include:

- Breadth-First Search (BFS): Explores all nodes at one depth level before moving to the next, ideal for unweighted graphs.
- Depth-First Search (DFS): Explores a branch as deeply as possible before backtracking. It uses less memory but may not guarantee completeness or optimality.
- Uniform Cost Search (UCS): Expands the least costly node first which ensures the lowest-cost solution.
- Greedy Best-First Search: Expands nodes that appear closest to the goal based on a heuristic.
- A* Search Algorithm: Combines path cost and heuristic to guarantee completeness and optimality.

Steps in State Space Search

1. **Define the State Space:** Identify all possible states, the initial state, the goal state, and valid transitions.
2. **Choose a Search Strategy:** Select a search algorithm such as **BFS, DFS, or A***.
3. **Start the Search:** Begin exploring from the initial state.
4. **Expand Nodes:** Generate successor (child) states and continue the search.
5. **Avoid Repeated States:** Keep track of visited states to prevent revisiting the same state.
6. **Stop the Search:** End when the goal state is found or when no more states are left to explore.

Breadth-First Search (BFS) algorithm on 8-Puzzle Problem Overview:

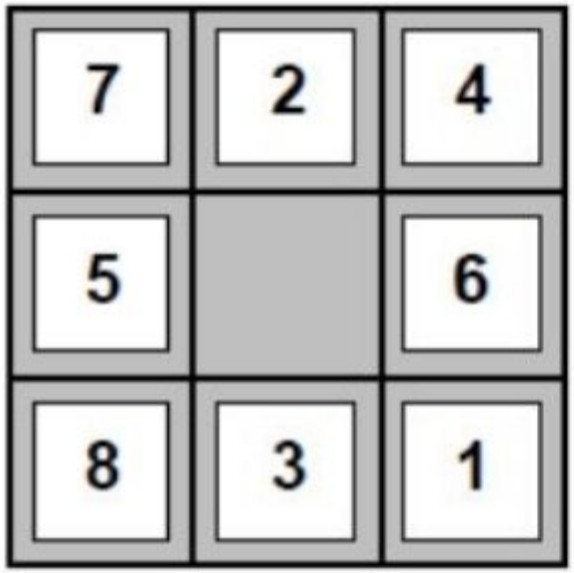
The **8-puzzle problem** is a sliding puzzle that consists of a 3x3 grid with 8 tiles numbered from 1 to 8 and one blank space.

The goal is to arrange the tiles in numerical order from 1 to 8 with the blank space in the lower-right corner.

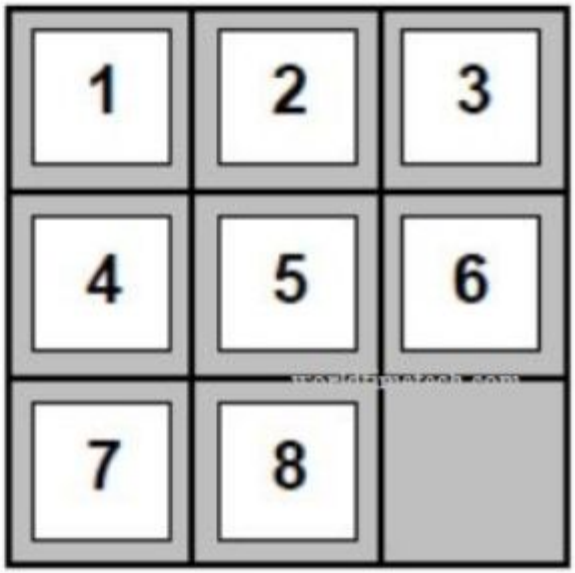
The puzzle begins with a shuffled arrangement of tiles and the objective is to reach the goal state using the least number of moves.

Here's a explanation of how the BFS algorithm works to explore state space and find the solution.

8-puzzle problem

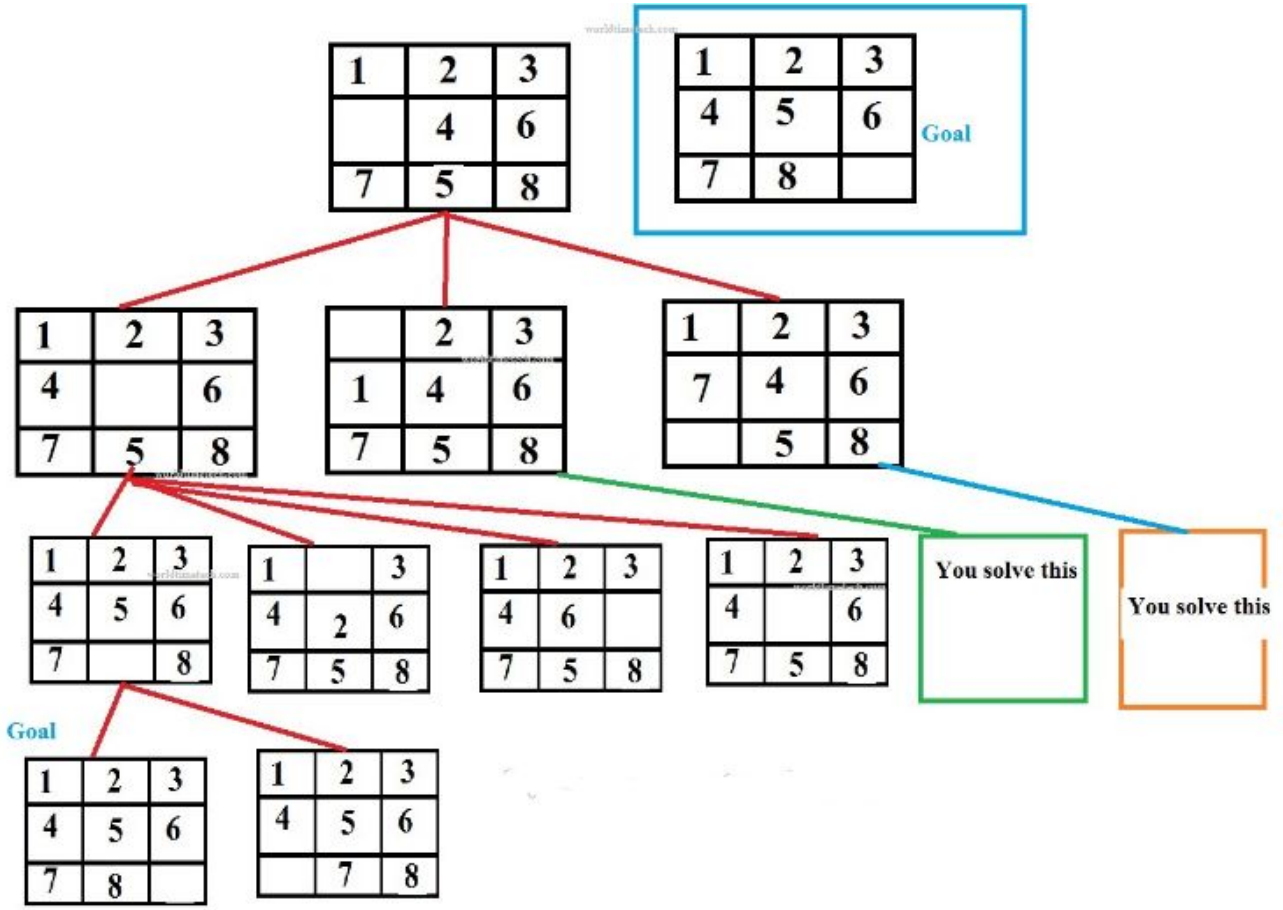


Start State



Goal State

8-puzzle problem



8-puzzle problem

How BFS Works

1. Start with the **initial state** and place it in the **queue**.
2. Remove the **first state** from the queue.
3. Generate all valid child states.
4. Ignore states that have already been visited.
5. Add new child states to the end of the queue.
6. Repeat the process until the **goal state** is found.

8-puzzle problem

Why BFS Finds the Shortest Path?

BFS explores **level by level**. It visits all states requiring **1 move**, then all states requiring **2 moves**, then **3 moves**, and so on. Therefore, the **first time the goal state is reached, it is guaranteed to be the solution with the minimum number of moves**.

Intelligent agent

- Intelligent agents are supposed to **maximize their performance measure**.
- This can be simplified if the agent can adopt a goal and aim at satisfying it.
- **Goal formulation**, based on the current situation and the agent's performance measure, is the first step in problem solving
- **Goal is a set of states**. The agent's task is to find out which sequence of actions will get it to a goal state
- **Problem formulation** is the process of deciding what sorts of actions and states to consider, given a goal

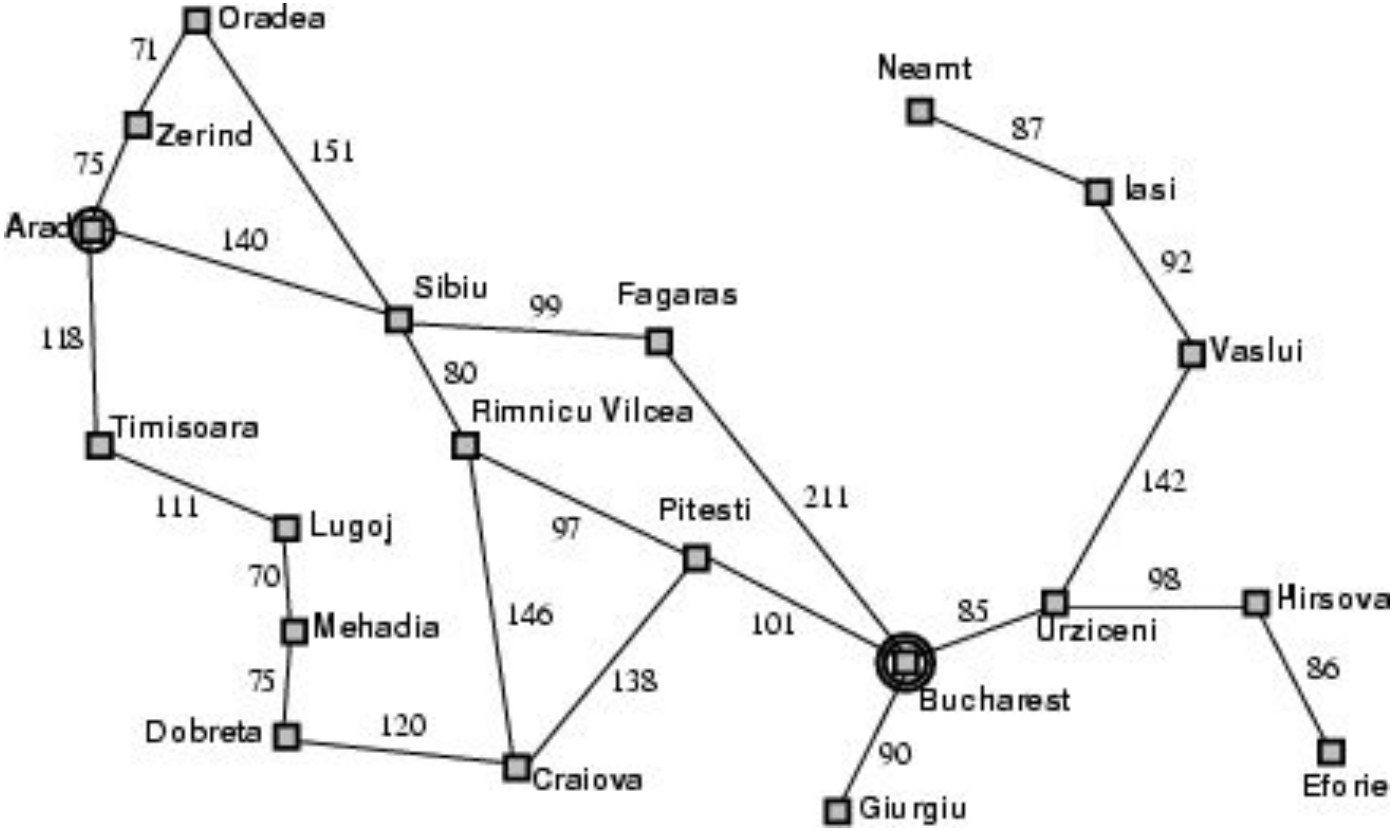
Intelligent agent

- **“formulate, search, execute”** design for the agent,
- After **formulating a goal and a problem to solve the agent calls a search procedure to solve it**,
- It then uses the solution to guide its actions, doing whatever the solution recommends as the next thing to do (typically the first action in the sequence),
- Then removing that step from the sequence,
- Once the solution has been executed, the agent will formulate a new goal.

EXAMPLE PROBLEM: ROMANIA

- On holiday in Romania; currently in Arad. Flight leaves tomorrow from Bucharest:
- Formulate goal:
  - be in Bucharest
- Formulate problem:
  - states: various cities
  - actions: drive between cities
- Find solution:
  - sequence of cities, e.g., Arad, Sibiu, Fagaras, Bucharest

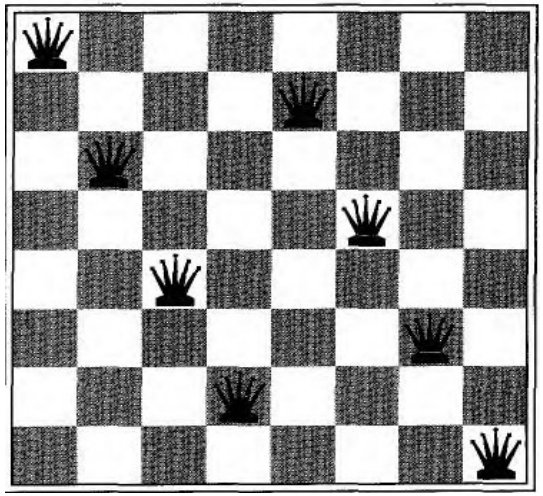
EXAMPLE PROBLEM: ROMANIA



EXAMPLE PROBLEM: ROMANIA

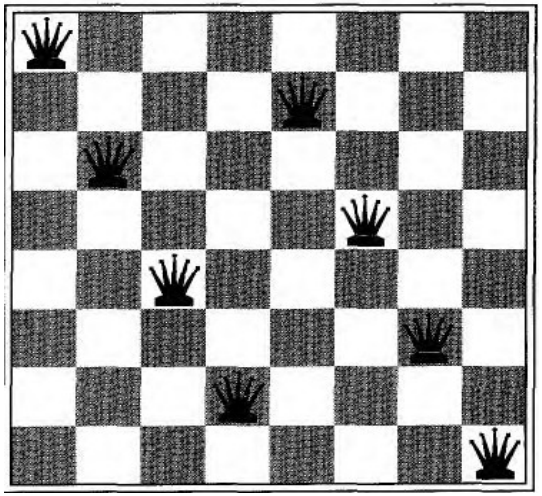
- **Initial state** that the agent starts in – e.g. In(Arad)
- A description of the possible actions available to the agent
  - **Successor function** – returns a set of <action, successor> pairs
  - e.g. {<Go(Sibiu),In(Sibiu)>, <Go(Timisoara),In(Timisoara)>, <Go(Zerind), In(Zerind)>}
  - Initial state and the successor function define the state space. A path in state space is a sequence of states connected by a sequence of actions
- **Goal test** determines whether a given state is a goal state – e.g. {In(Bucharest)}
- **Path cost** function that assigns a numeric cost to each path.

EXAMPLE: 8-QUEENS PROBLEM



- **States:**
- **Initial state:**
- **Successor function:**
- **Goal test:**

EXAMPLE: 8-QUEENS PROBLEM



- **States:** Any arrangement of 0 to 8 queens on the board is a state.
- **Initial state:** No queens on the board.
- **Successor function:** Add a queen to any empty square.
- **Goal test: 8** queens are on the board, none attacked.

SEARCH STRATEGIES

- A search strategy is defined by picking the order of node expansion
- Strategies are evaluated along the following dimensions:
  - **completeness:** does it always find a solution if one exists?
  - **time complexity:** How long does it take to find a solution?
  - **space complexity:** How much memory is needed to perform the search?
  - **optimality:** does it always find a least-cost solution?
- Time and space complexity are measured in terms of
  - **b:** maximum branching factor of the search tree
  - **d:** depth of the least-cost solution
  - **m:** maximum depth of the state space (may be ∞)

Search algorithm

```
Function SimpleProblemSolvingAgent(percept):
    state ← InterpretPercept(percept)
    if state is GOAL_STATE:
        return NoOp
    else:
        problem ← FormulateProblem(state)
        solution ← Search(problem)
        return ExecuteAction(solution)
```

Well-defined problems and solutions

A problem is defined by 5 components:

1. ***Initial state***
2. ***Actions***
3. ***Transition model or (Successor functions)***
4. ***Goal Test.***
5. ***Path Cost.***

A problem is defined by 5 components:

1. The ***initial state***

- The agent starts in

2. The set of possible **actions**

3. ***Transition model:*** *description of what each action does.*
(***successor functions***): refer to any state reachable from given state by a single action

- Initial state, actions and Transition model define the ***state space***
  - The set of all states reachable from the initial state by any sequence of actions.
- A ***path*** in the state space:
  - Any sequence of states connected by a sequence of actions.

4. The ***goal test***

- Applied to the current state to test
  - If the agent is in its goal

-Sometimes there is an explicit set of possible goal states.

-Sometimes the goal is described by the properties

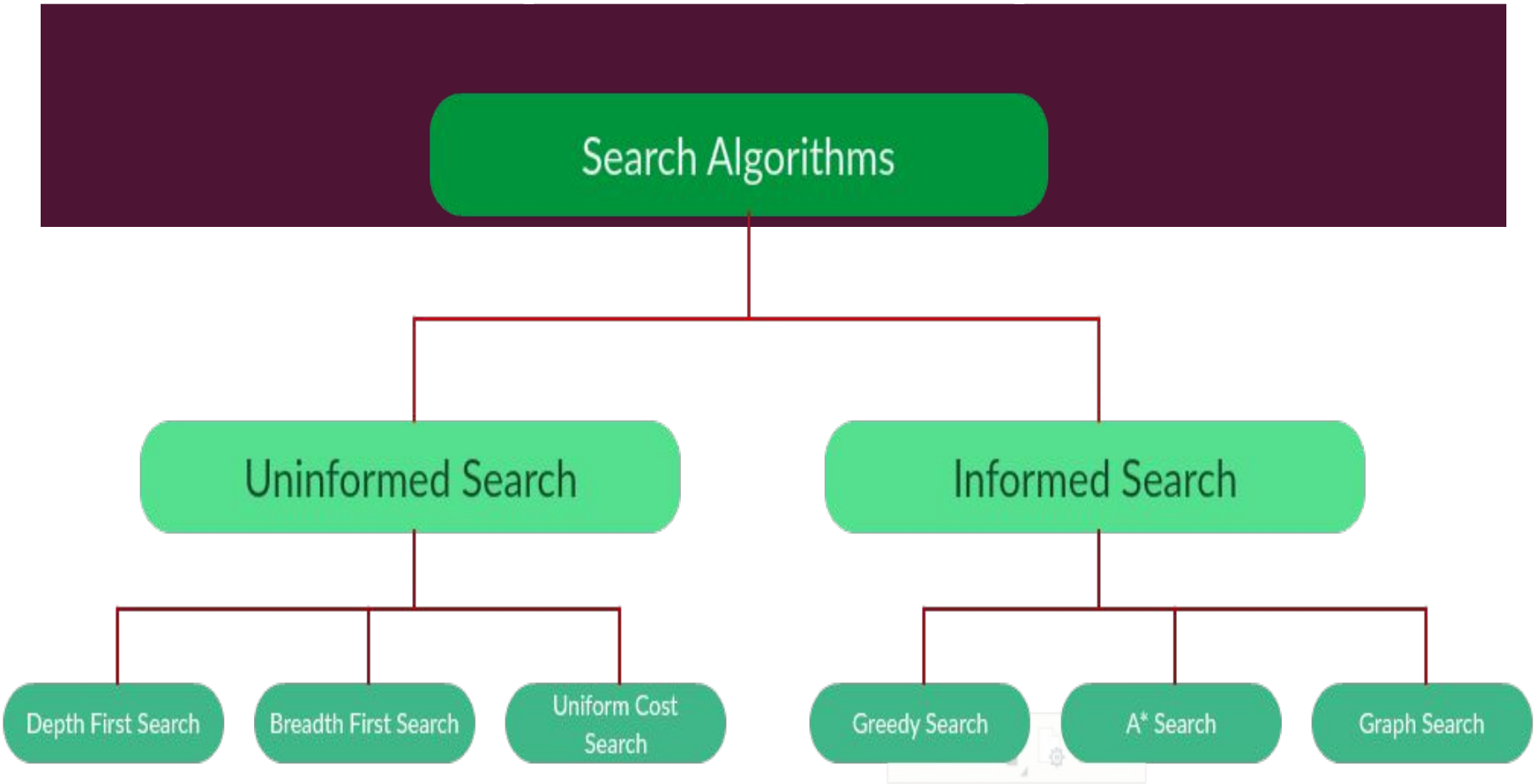
  - Instead of stating explicitly the set of states

- Example: Chess
  - The agent wins if it can capture the KING of the opponent on next move (checkmate).
  - no matter what the opponent does

5. A ***path cost*** function,

- assigns a numeric cost to each path
- = performance measure
- denoted by ***g***
- to distinguish the best path from others

- Usually the path cost is
  - the sum of the **step costs** of the individual actions (in the action list)


Search Algorithms
Uninformed Search
Informed Search
Depth First Search
Breadth First Search
Uniform Cost Search
Greedy Search
A* Search
Graph Search

UNINFORMED SEARCH STRATEGIES

- **Uninformed Search Algorithms** (also called **Blind Search Algorithms**) search for a solution **without using any heuristic or domain-specific knowledge**. They explore the search space using only the information available in the problem definition, such as the initial state, goal state, and successor function.

- Uninformed search strategies use only the information available in the problem definition
  - Breadth-first search
  - Uniform-cost search
  - Depth-first search
  - Depth-limited search
  - Iterative deepening search

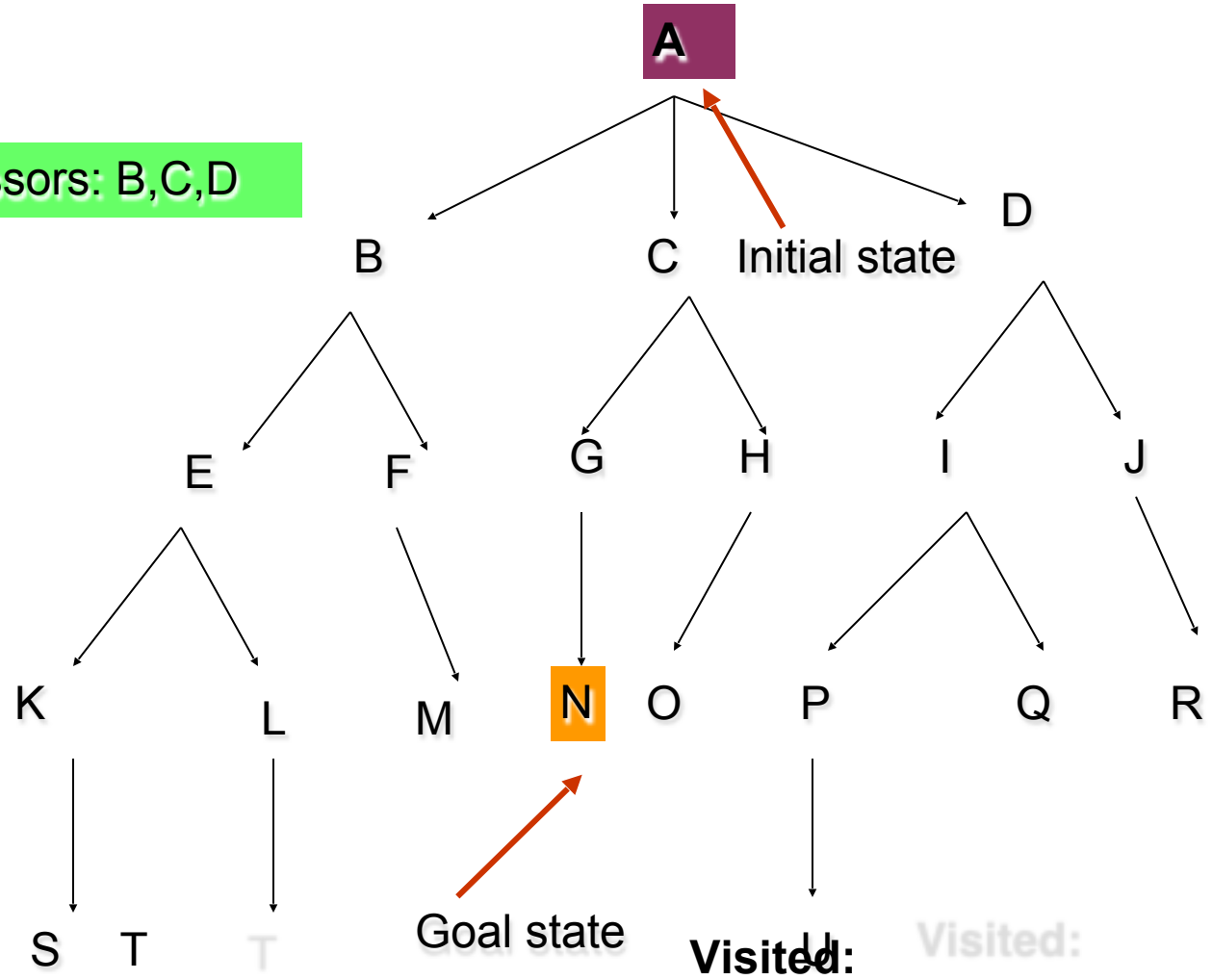
UNINFORMED SEARCH STRATEGIES

Characteristics

1. No heuristic function is used.
2. Explores the search space systematically.
3. Suitable when no additional information about the problem is available.
4. May explore many unnecessary nodes before reaching the goal.

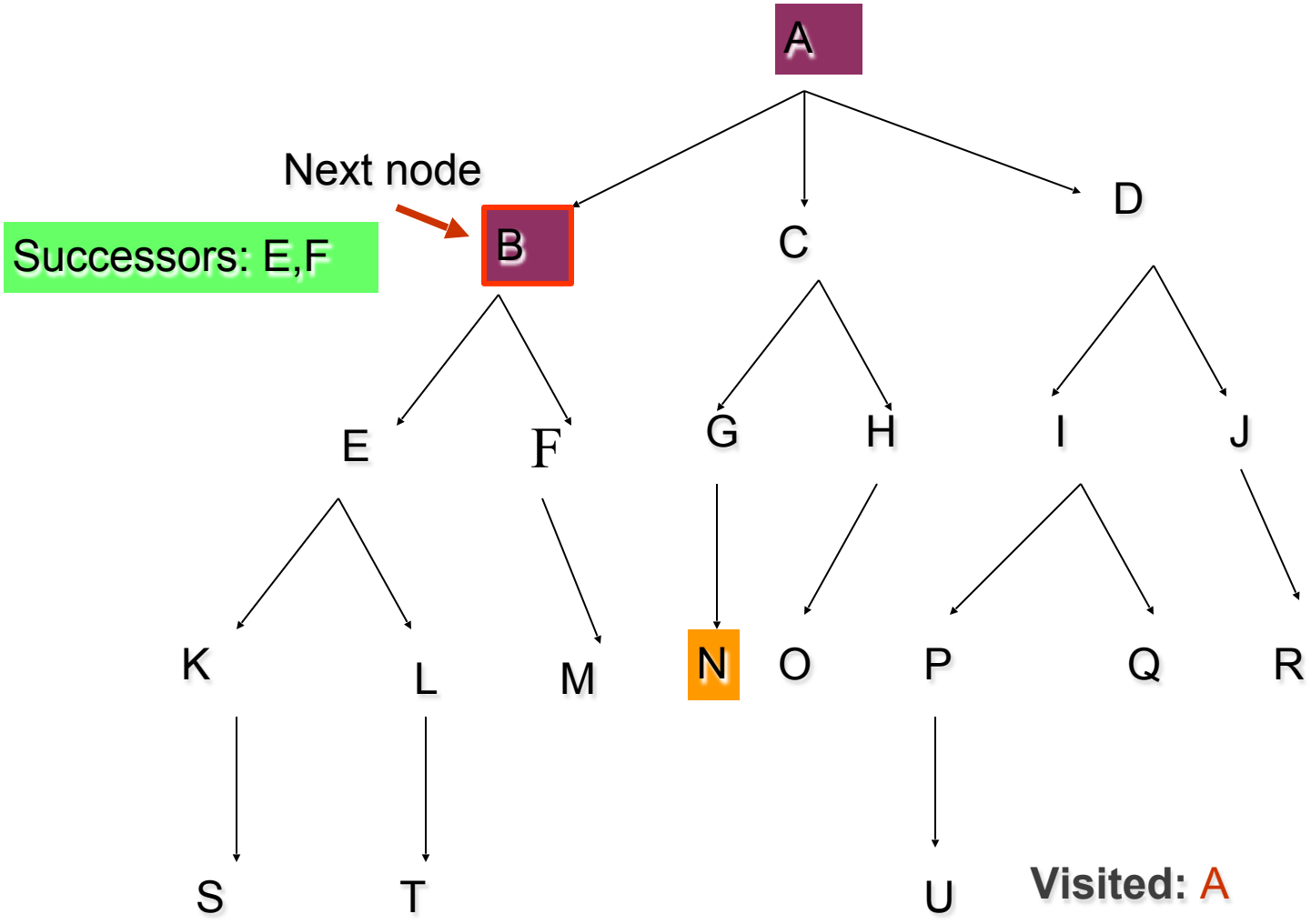
BREADTH-FIRST SEARCH

- The root node is expanded first, then all the successors of the root node, and their successors and so on
- In general, all the nodes are expanded at a given depth in the search tree before any nodes at the next level are expanded
- Expand shallowest unexpanded node
- Implementation:
  - fringe is a FIFO queue,
  - the nodes that are visited first will be expanded first
  - All newly generated successors will be put at the end of the queue
  - Shallow nodes are expanded before deeper nodes

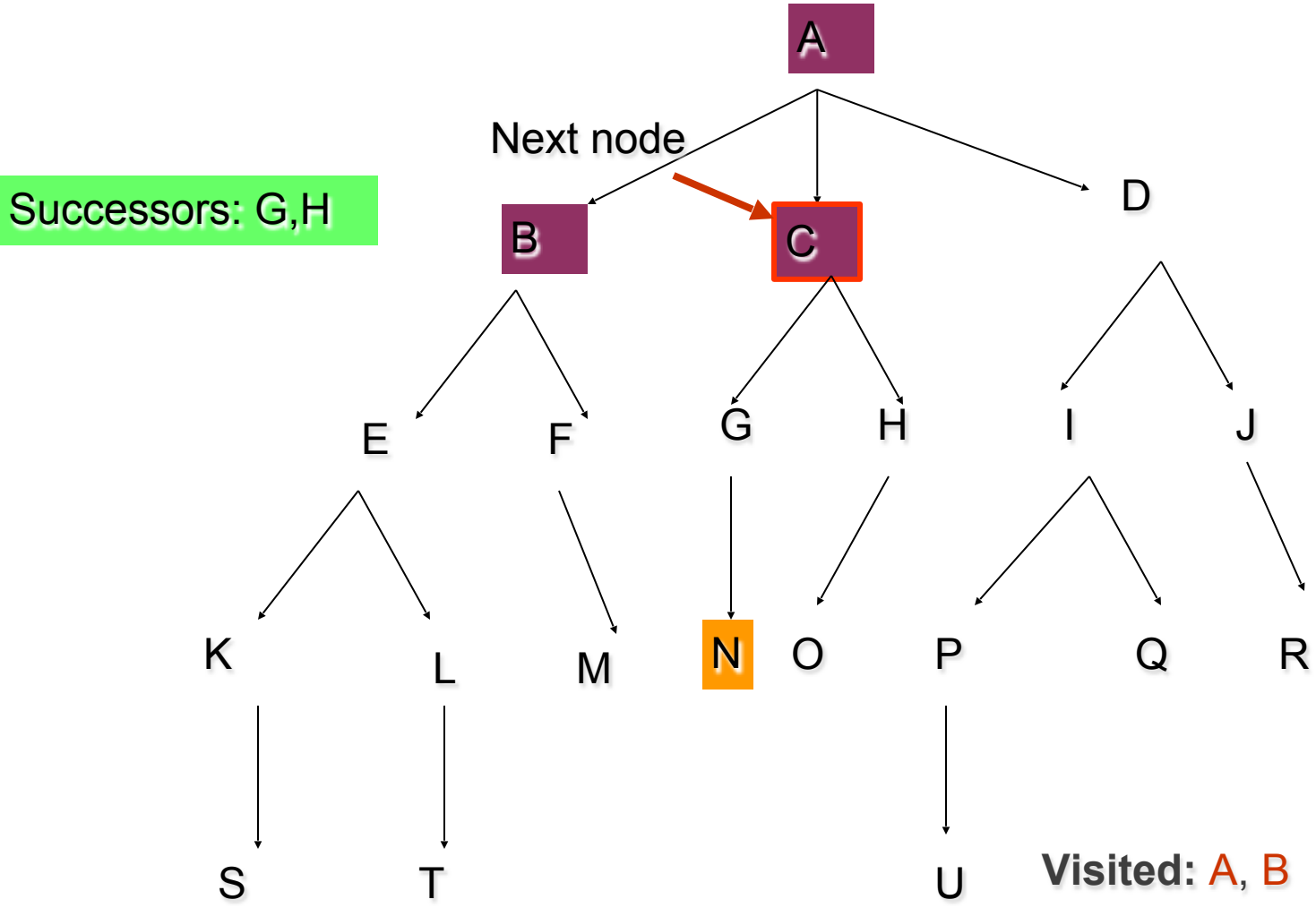


Fringe: A (FIFO)

The fringe is the data structure we use to store all of the nodes that have been generated



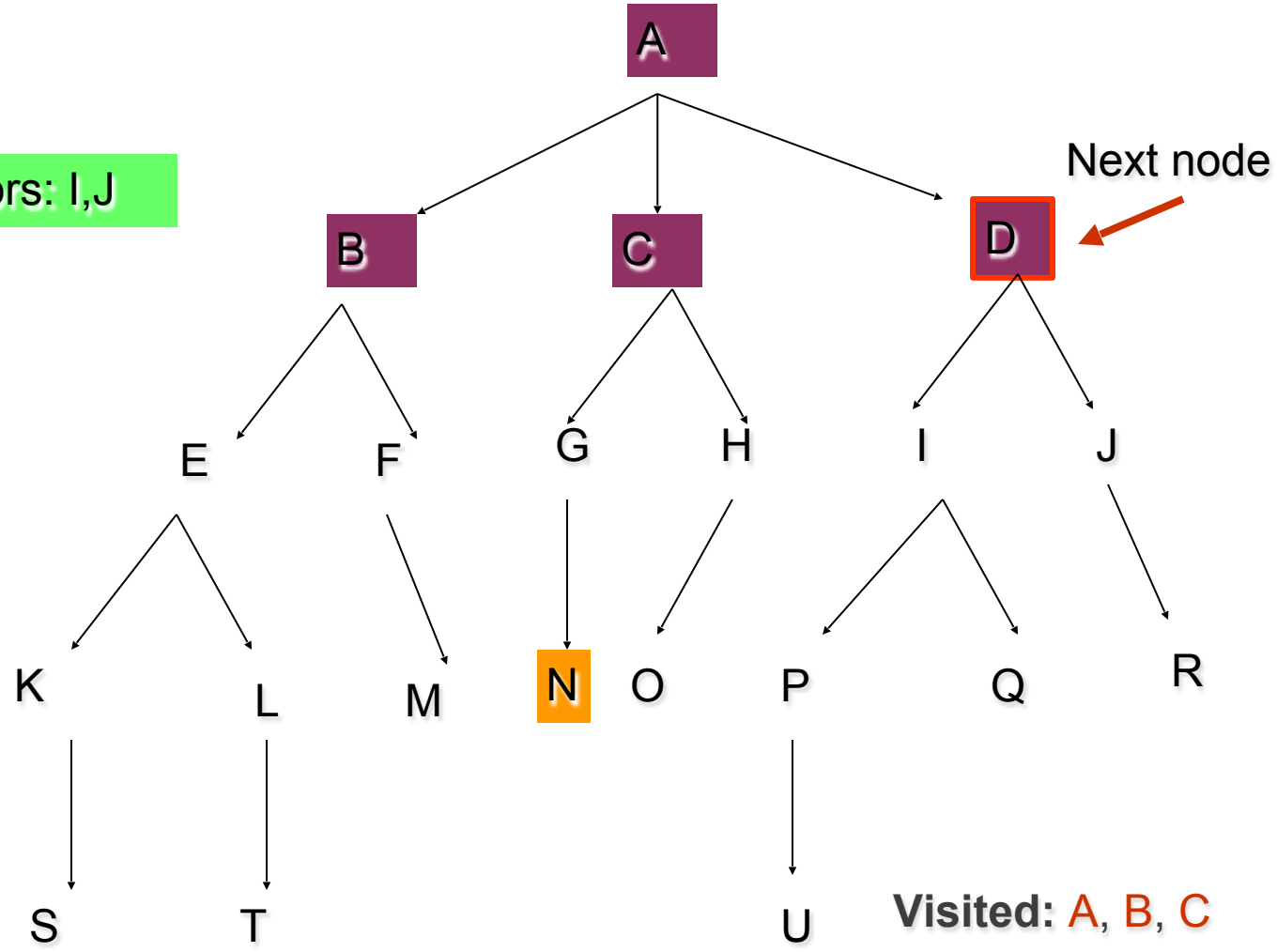
Fringe: B,C,D (FIFO)



Successors: G,H

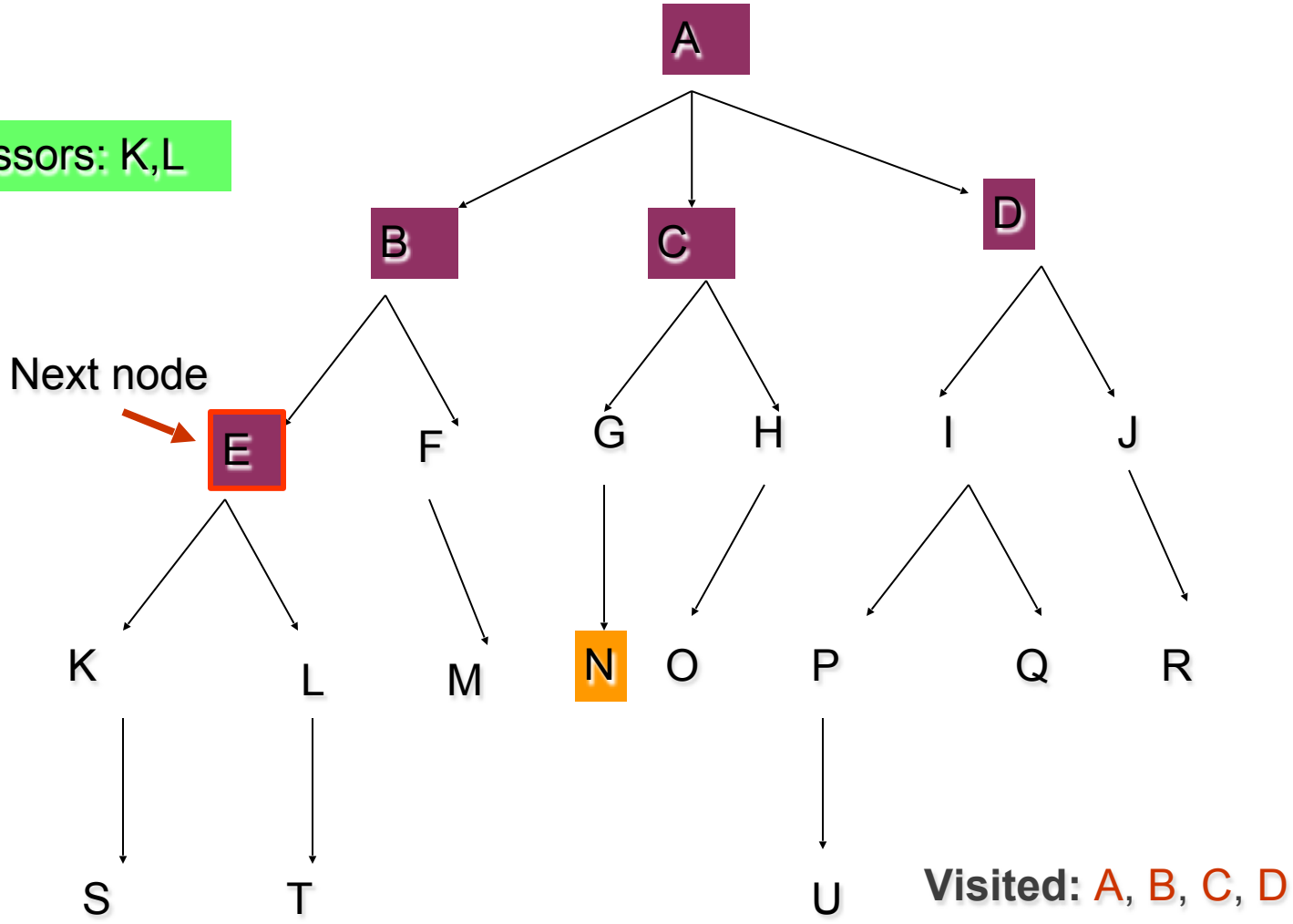
Fringe: C,D,E,F (FIFO)

Successors: I,J



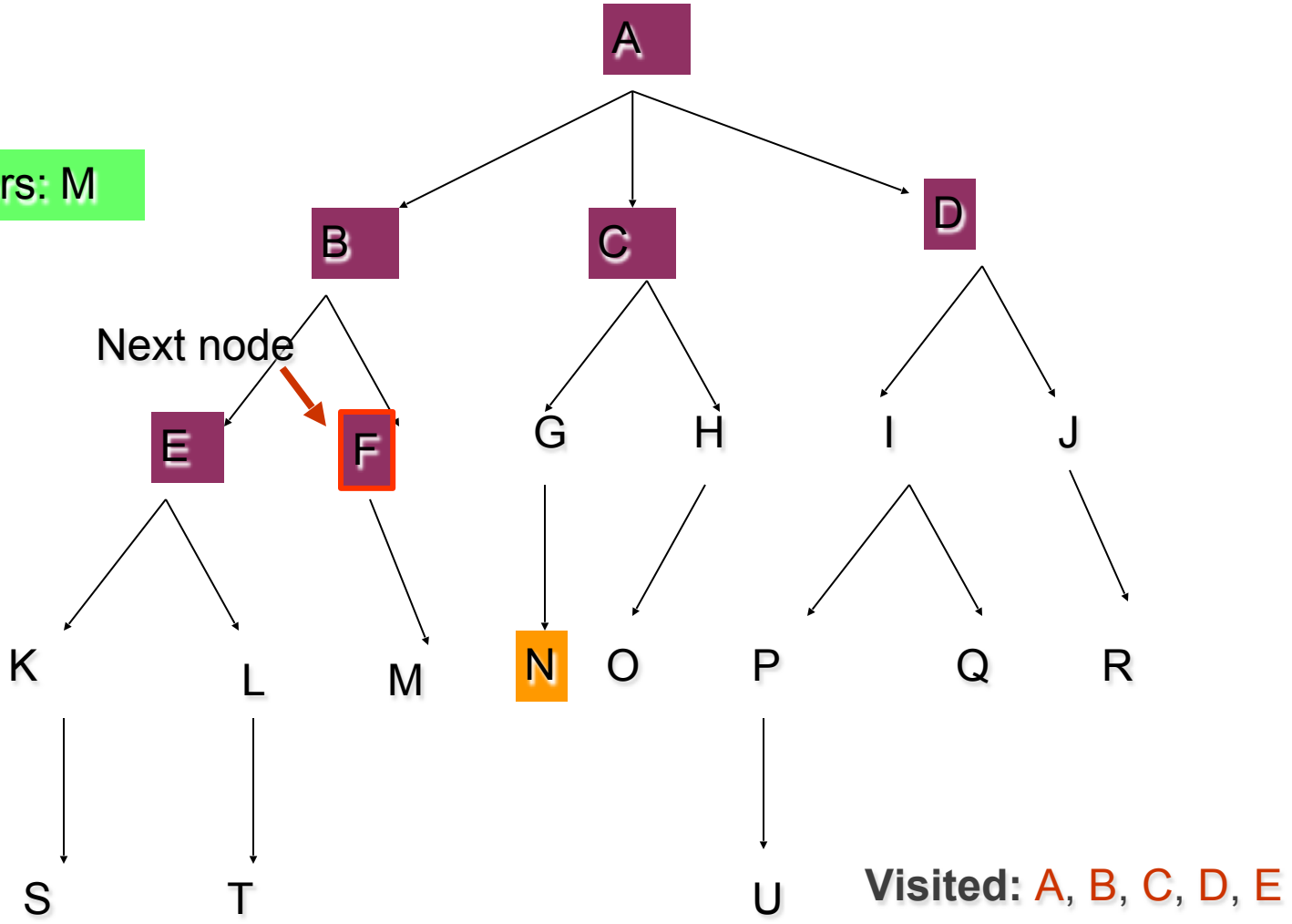
Fringe: D,E,F,G,H (FIFO)

Successors: K,L



Fringe: E,F,G,H,I,J (FIFO)

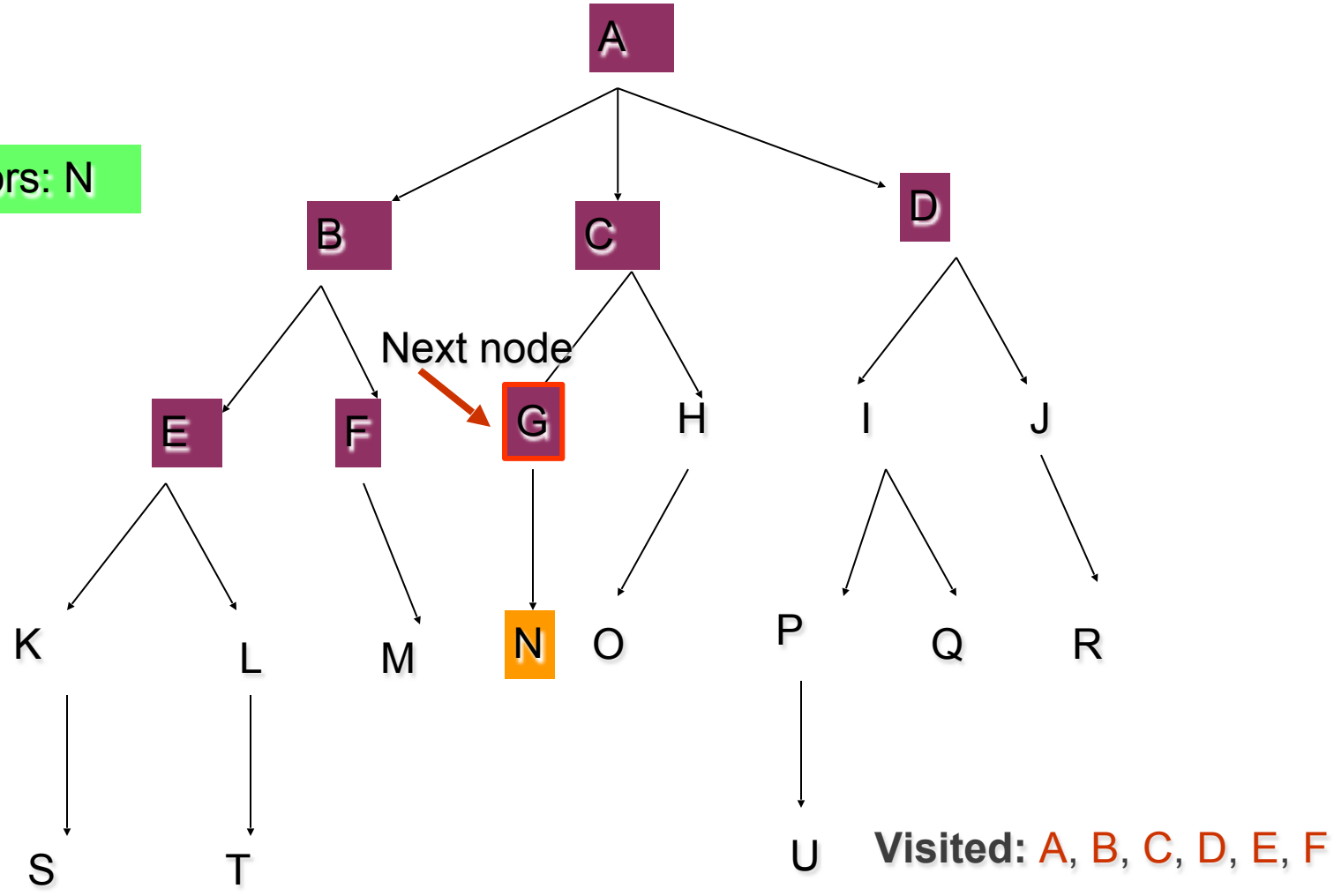
Successors: M



Visited: A, B, C, D, E

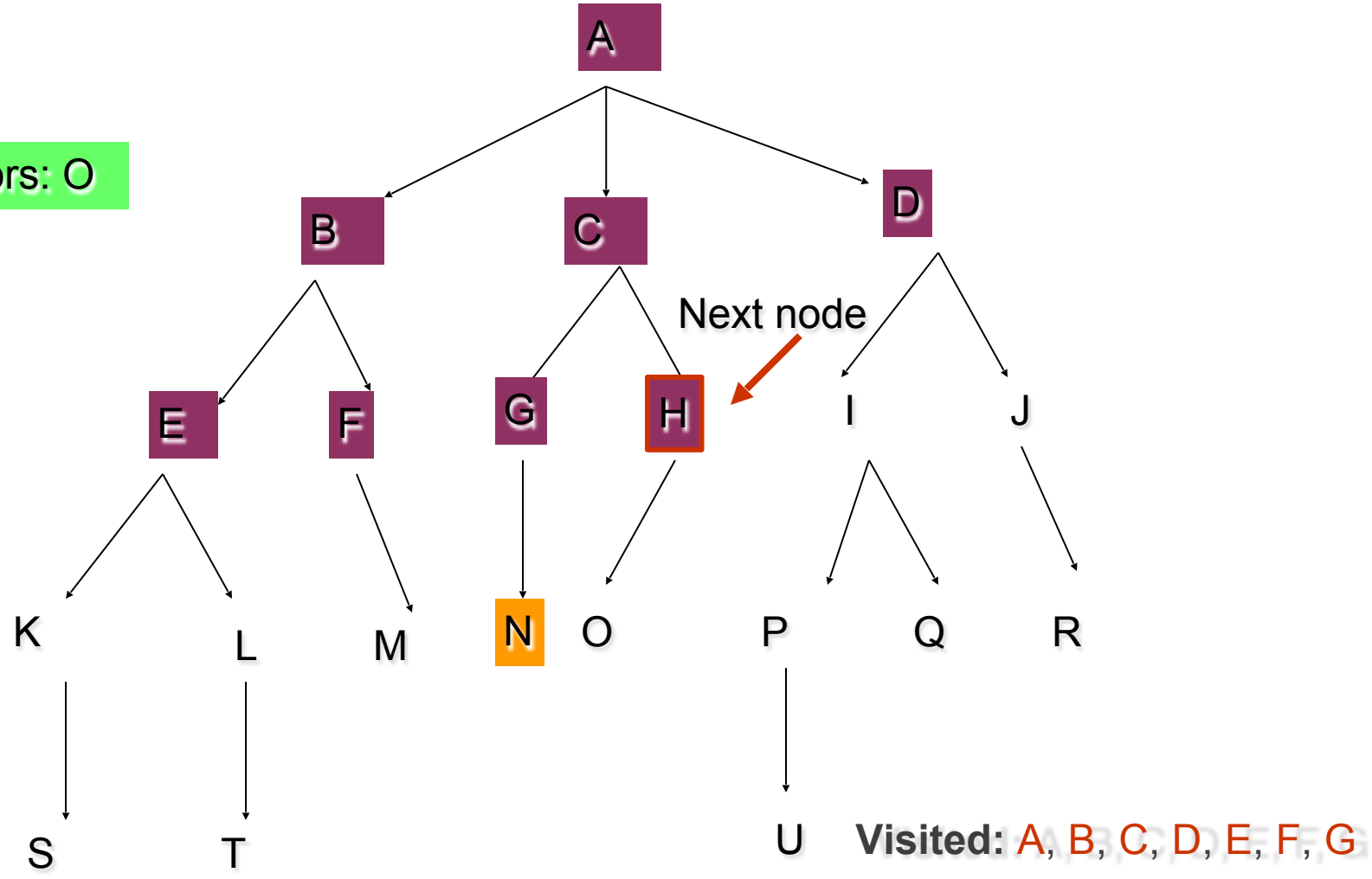
Fringe: F,G,H,I,J,K,L (FIFO)

Successors: N



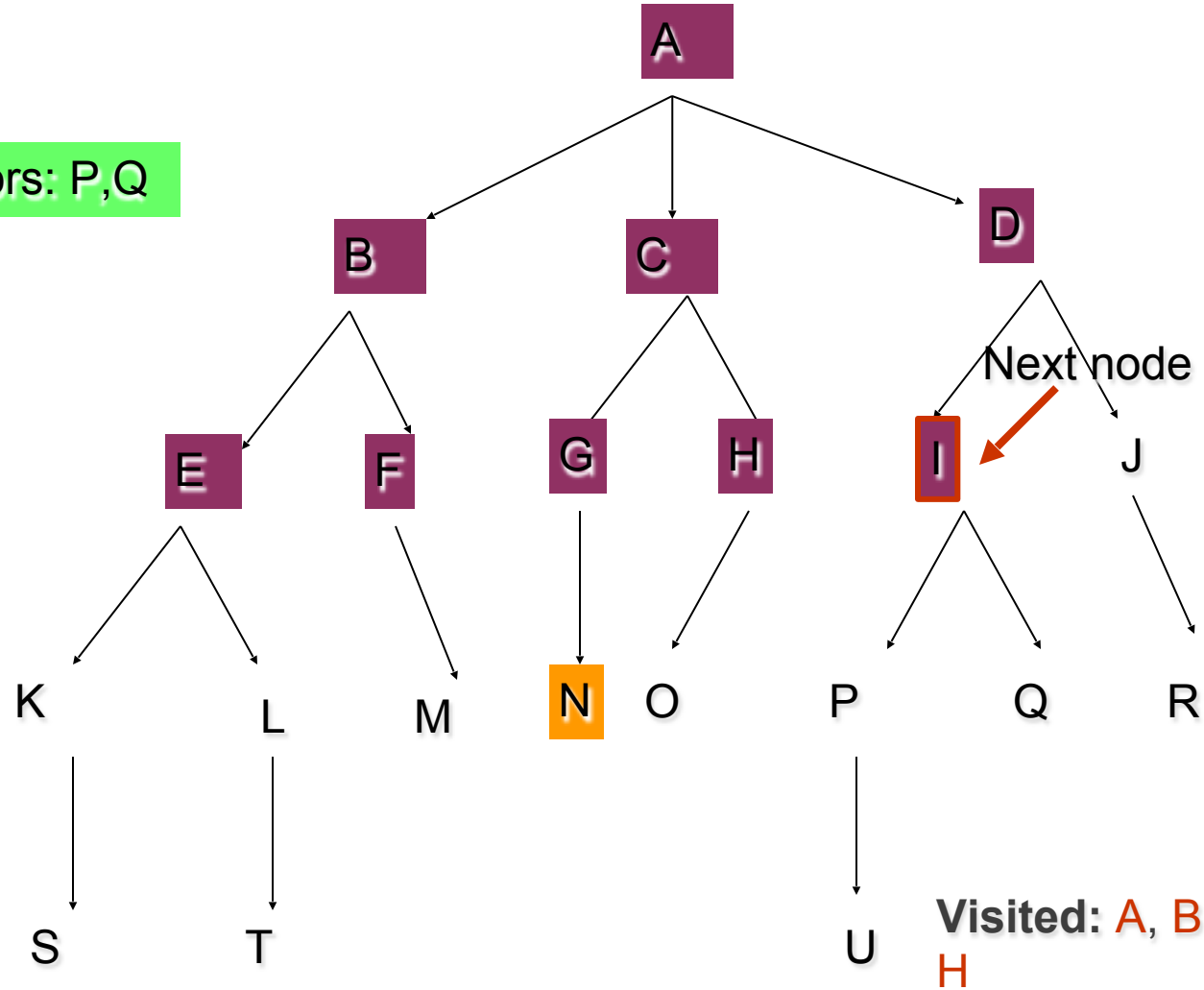
Fringe: G,H,I,J,K,L,M (FIFO)

Successors: O



Fringe: H,I,J,K,L,M,N (FIFO)

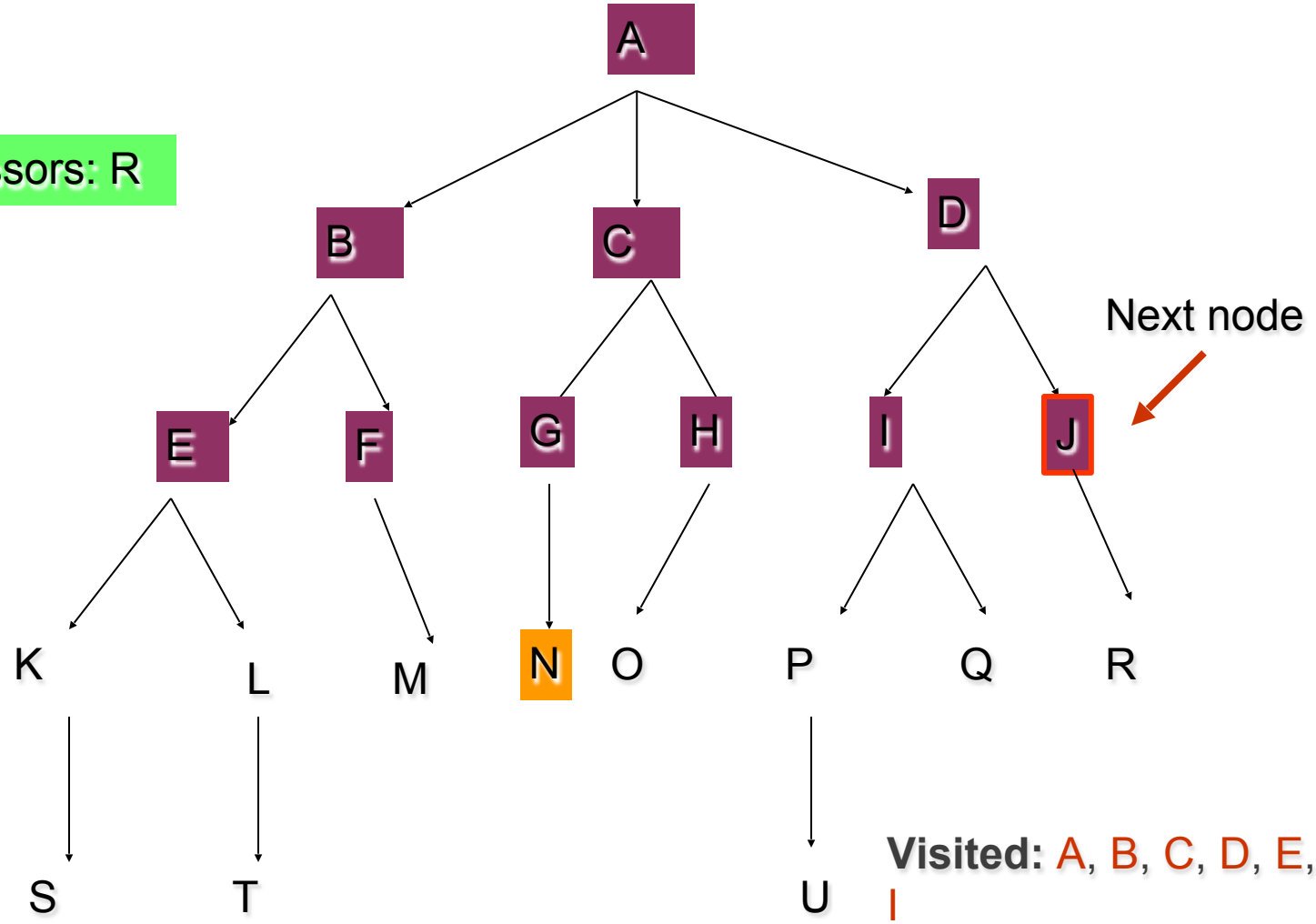
Successors: P,Q



Visited: A, B, C, D, E, F, G, H

Fringe: I,J,K,L,M,N,O (FIFO)

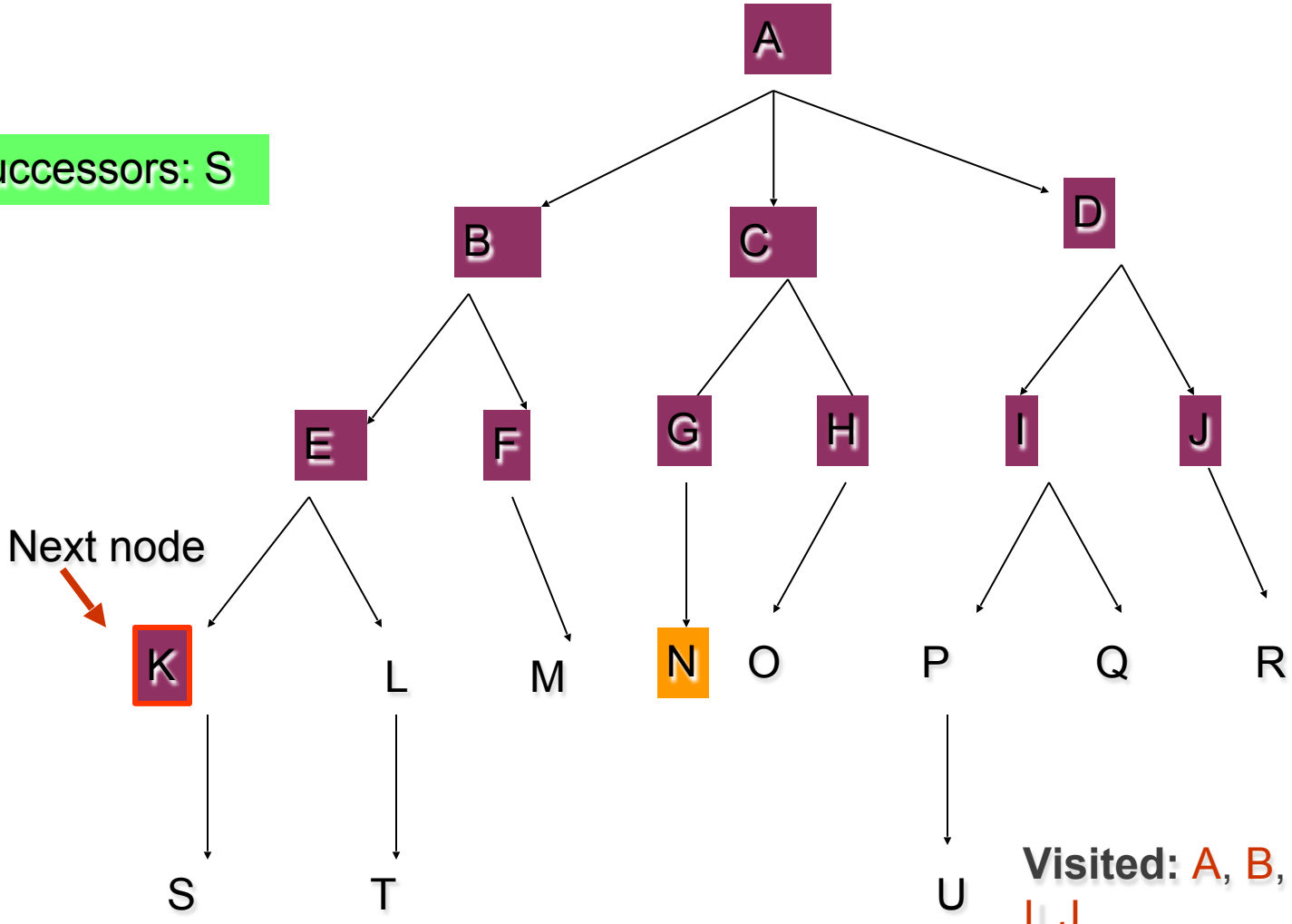
Successors: R



Visited: A, B, C, D, E, F, G, H, I

Fringe: J,K,L,M,N,O,P,Q (FIFO)

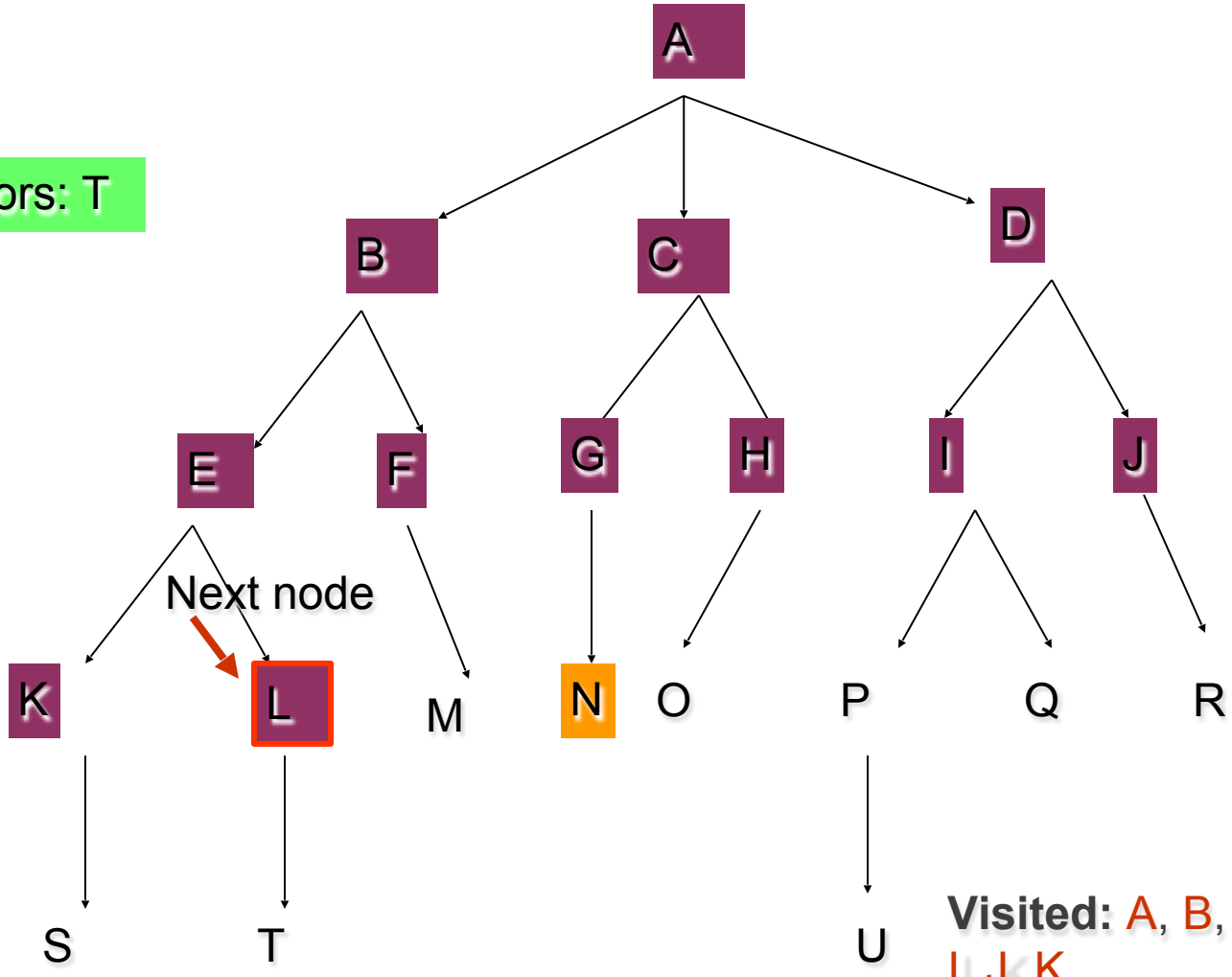
Successors: S



Visited: A, B, C, D, E, F, G, H, I, J

Fringe: K,L,M,N,O,P,Q,R (FIFO)

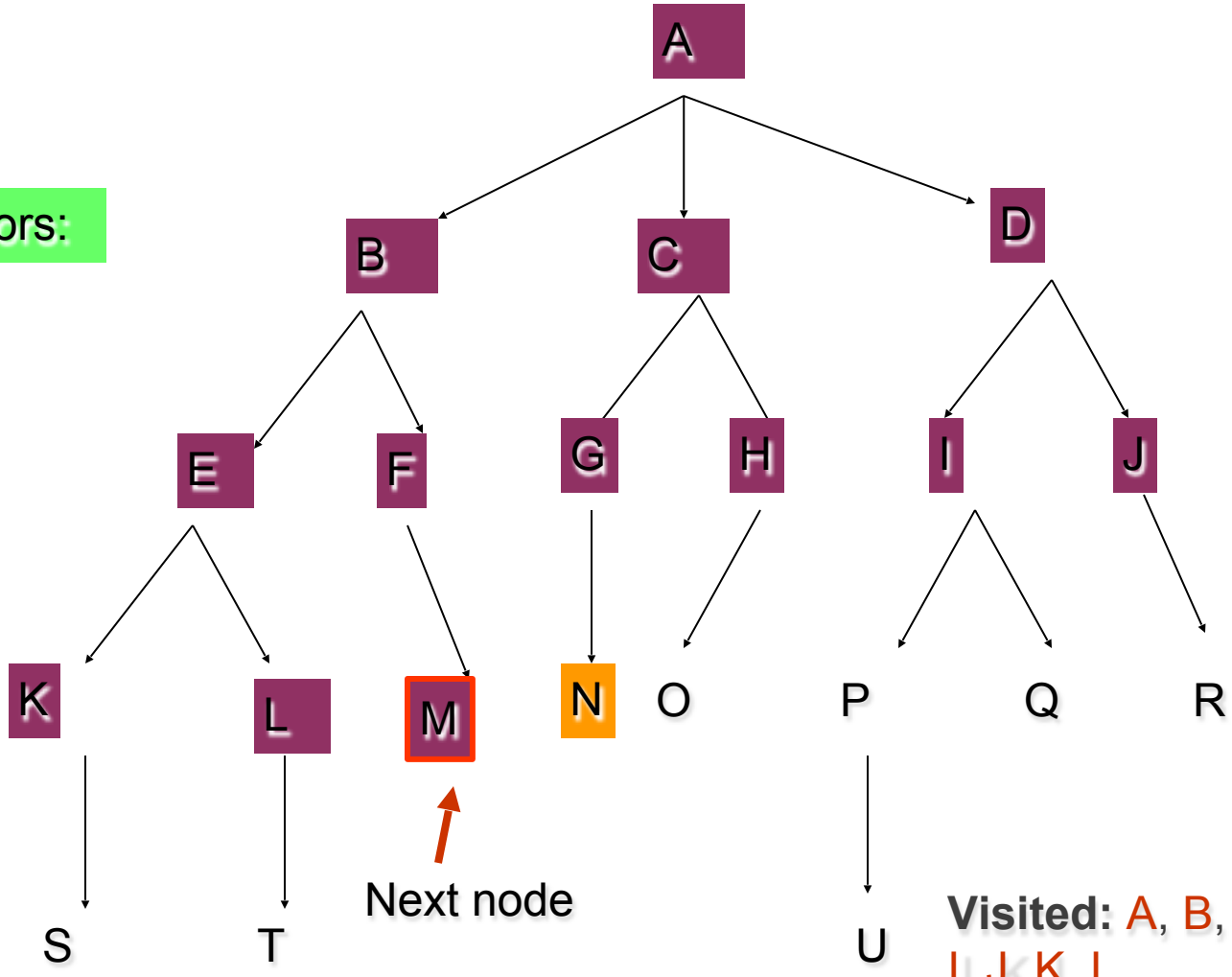
Successors: T



Visited: A, B, C, D, E, F, G, H, I, J, K

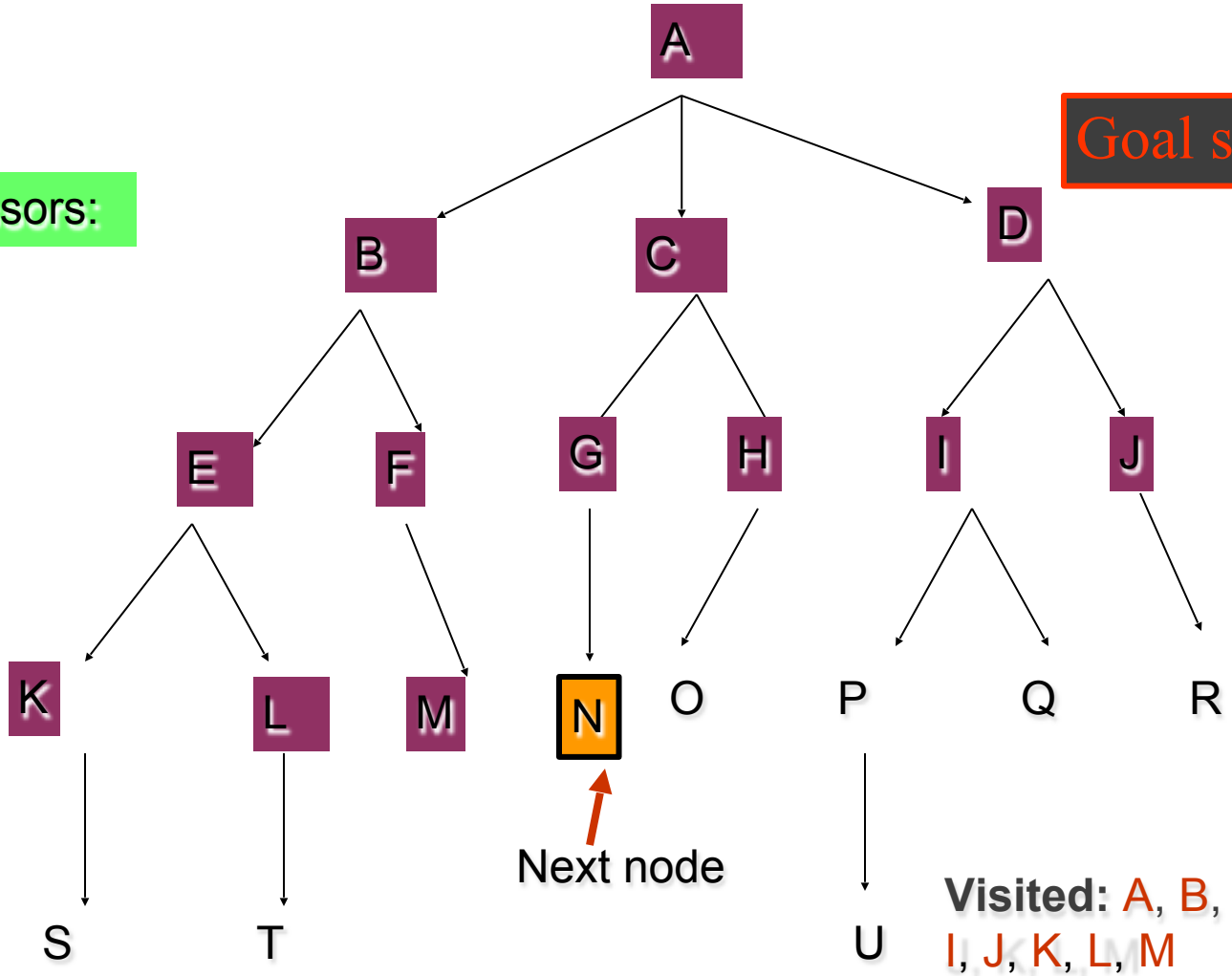
Fringe: L,M,N,O,P,Q,R,S (FIFO)

Successors:



Visited: A, B, C, D, E, F, G, H, I, J, K, L

Fringe: M,N,O,P,Q,R,S,T (FIFO)



Successors:

Goal state achieved

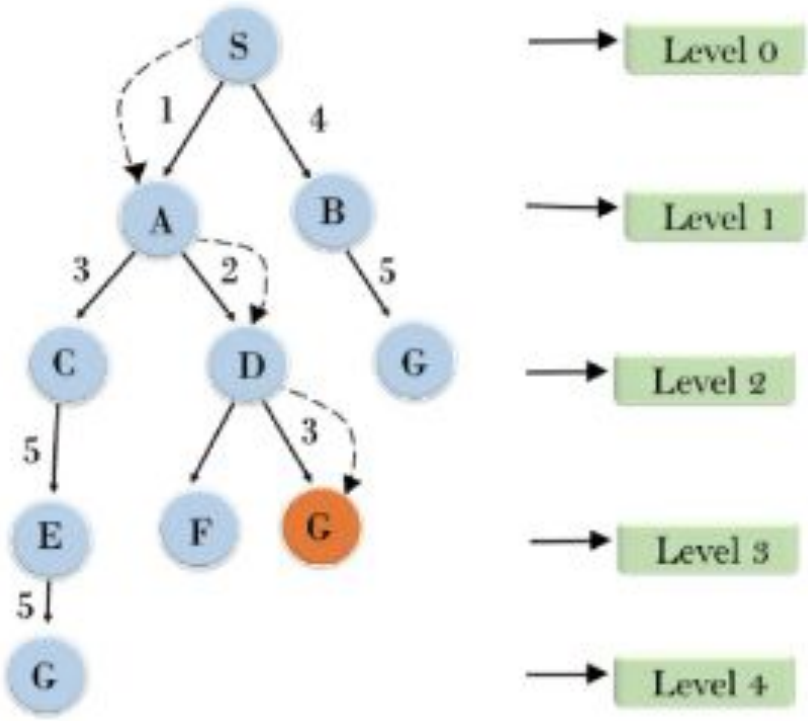
Visited: A, B, C, D, E, F, G, H, I, J, K, L, M

Fringe: N,O,P,Q,R,S,T (FIFO)

UNIFORM-COST SEARCH

- Uniform cost search is a search algorithm used to traverse, and find the shortest path in weighted trees and graphs.
- Uniform Cost Search or UCS begins at a root node and will continually expand nodes, taking the node with the smallest total cost from the root until it reaches the goal state.
- Uniform cost search doesn't care about how many steps a path has, only the total cost of the path.
- UCS behaves exactly like BFS when all edge costs are equal.

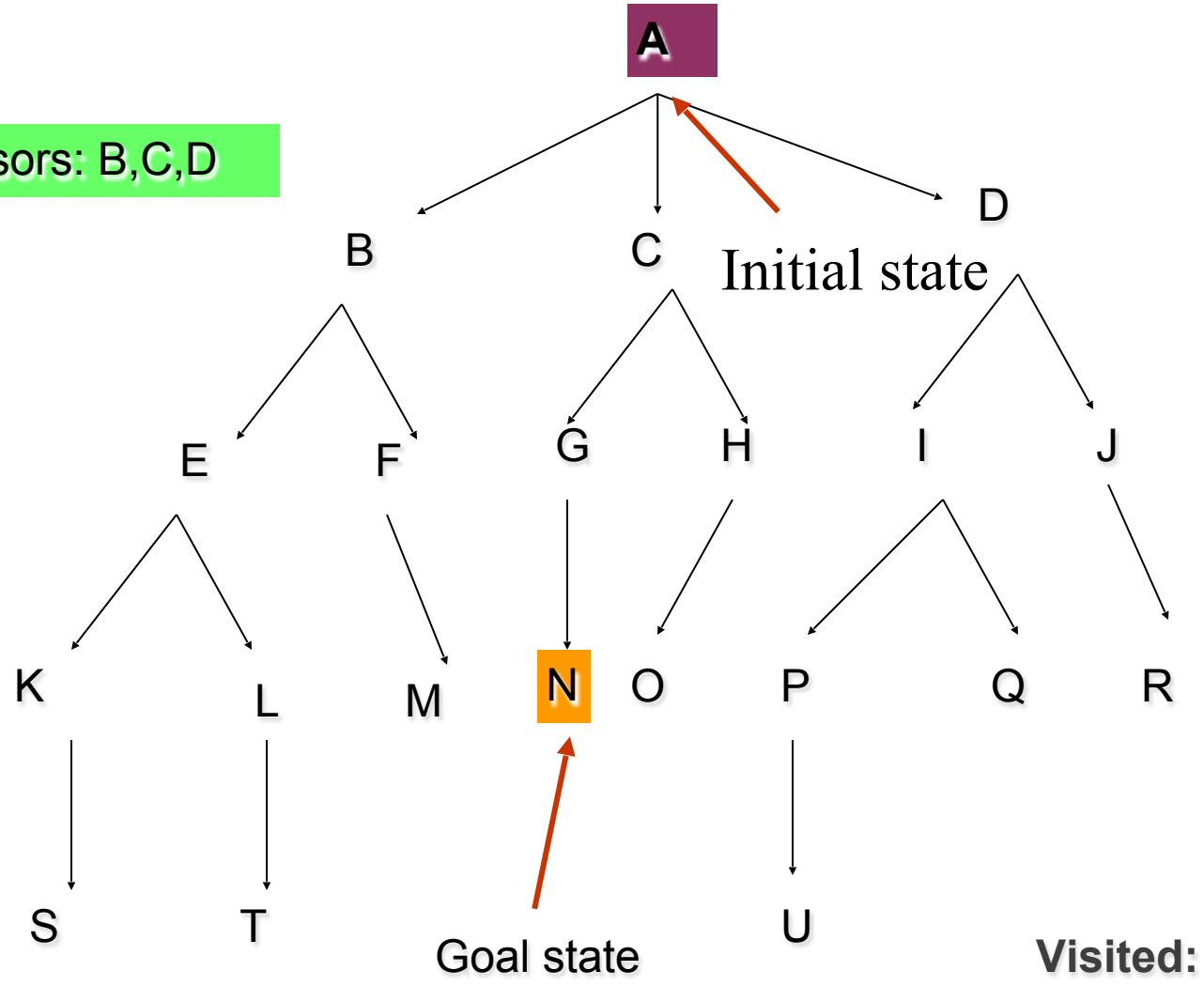
Uniform Cost Search



DEPTH FIRST SEARCH - METHOD

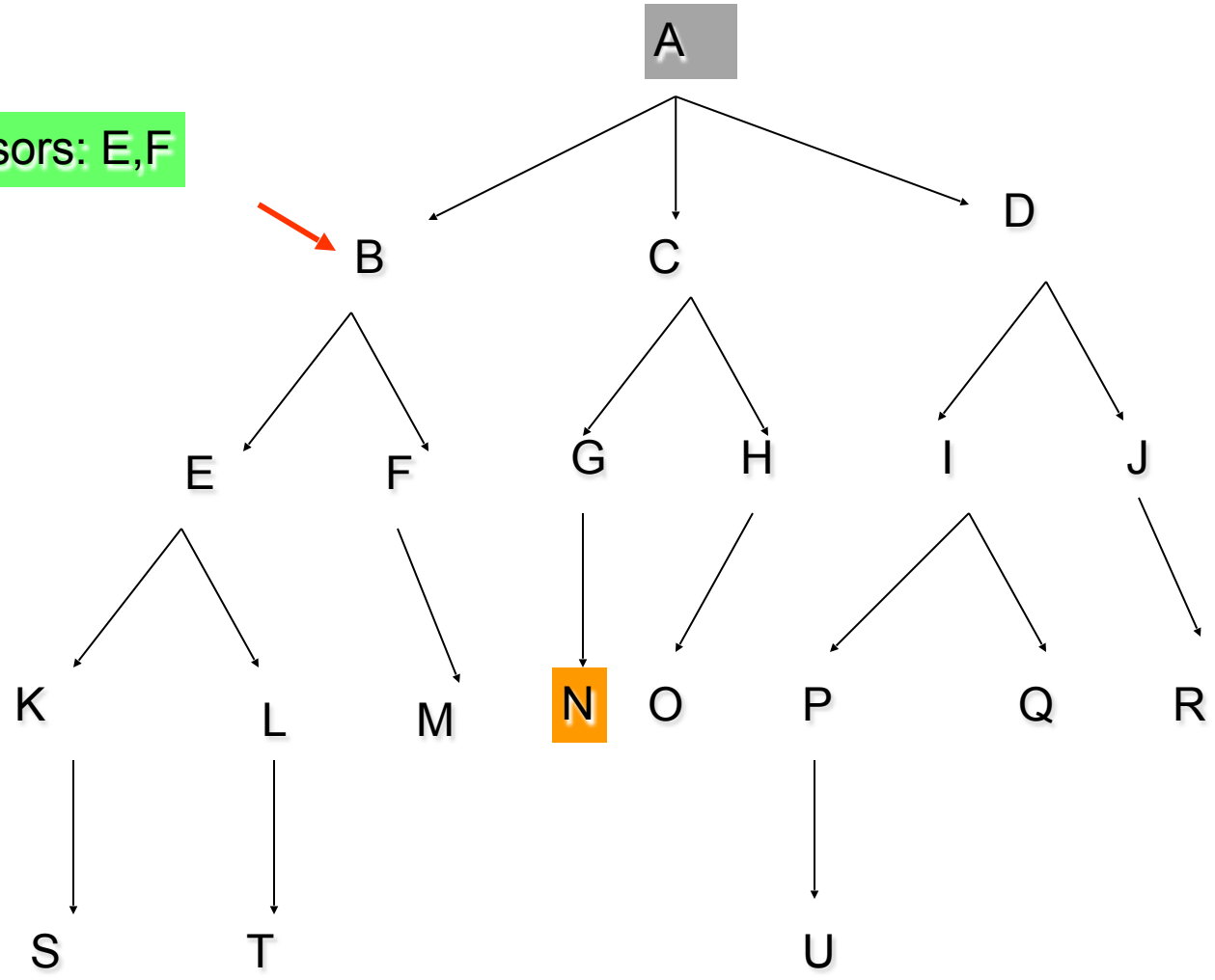
- Expand Root Node First
- Explore one branch of the tree before exploring another branch
- If a leaf node do not represent a goal state, search backtracks up to the next highest node that has an unexplored path

Successors: B,C,D



Fringe: A (LIFO)

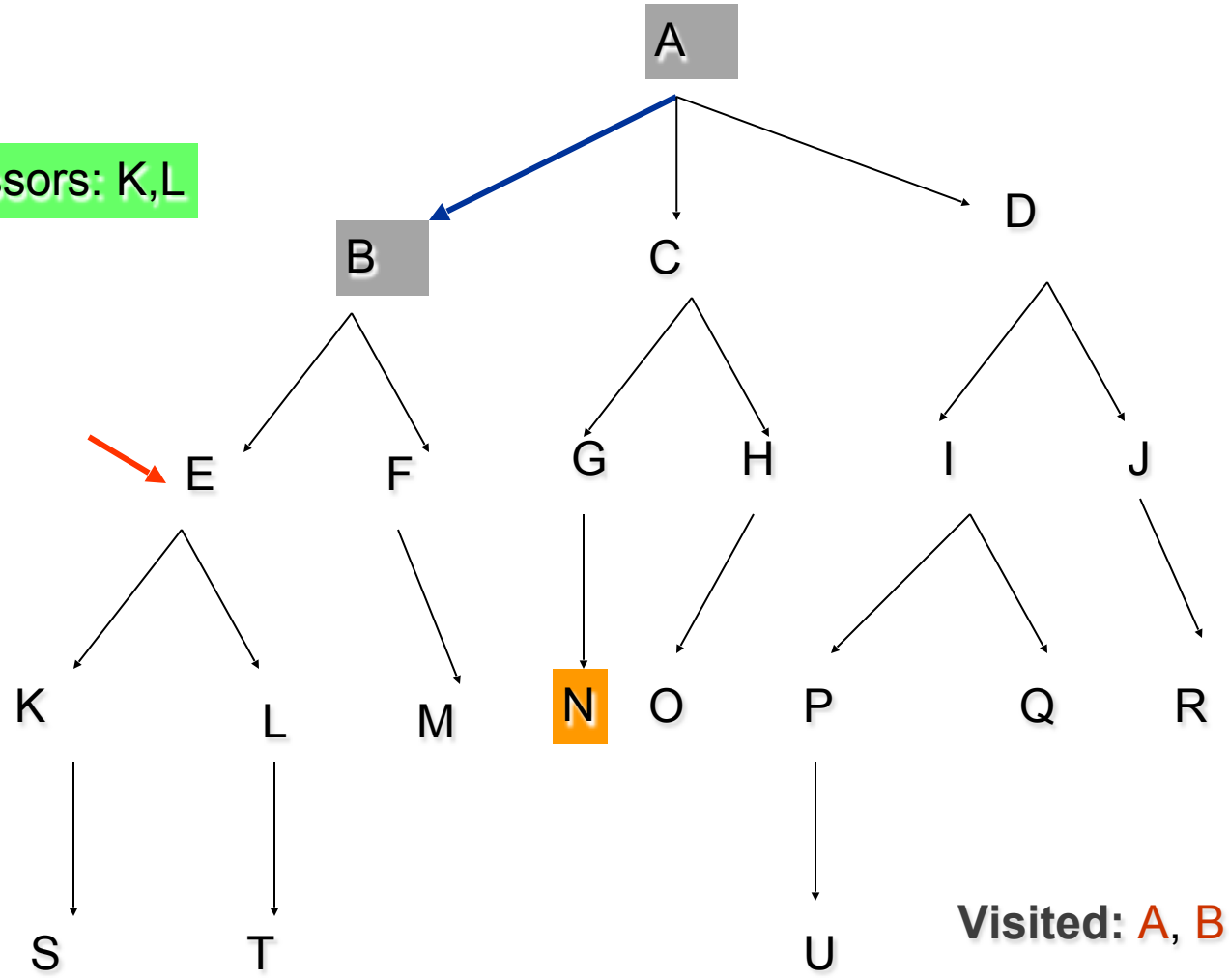
Successors: E,F



Visited: A

Fringe: B,C,D (LIFO)

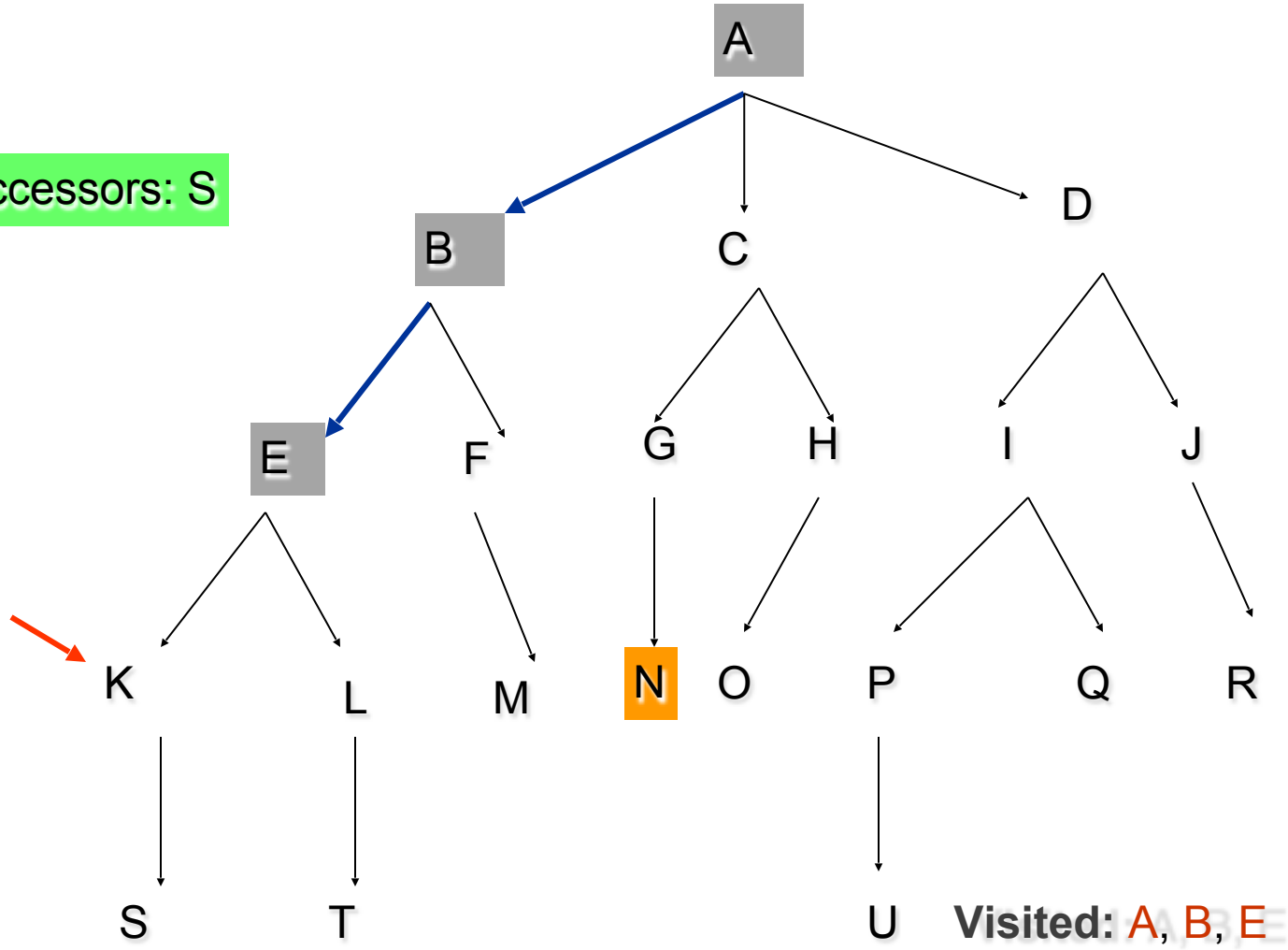
Successors: K,L



Visited: A, B

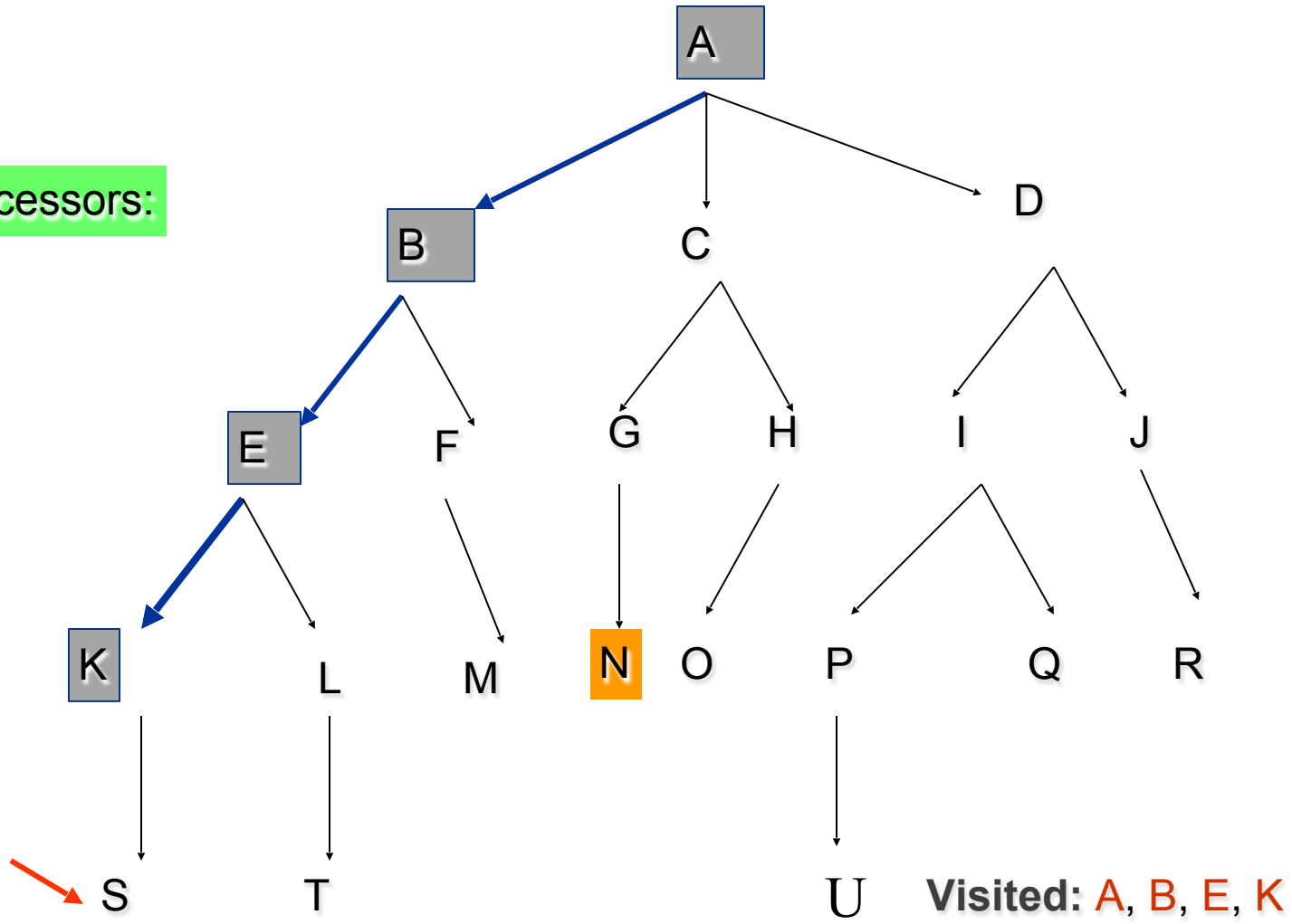
Fringe: E,F,C,D (LIFO)

Successors: S



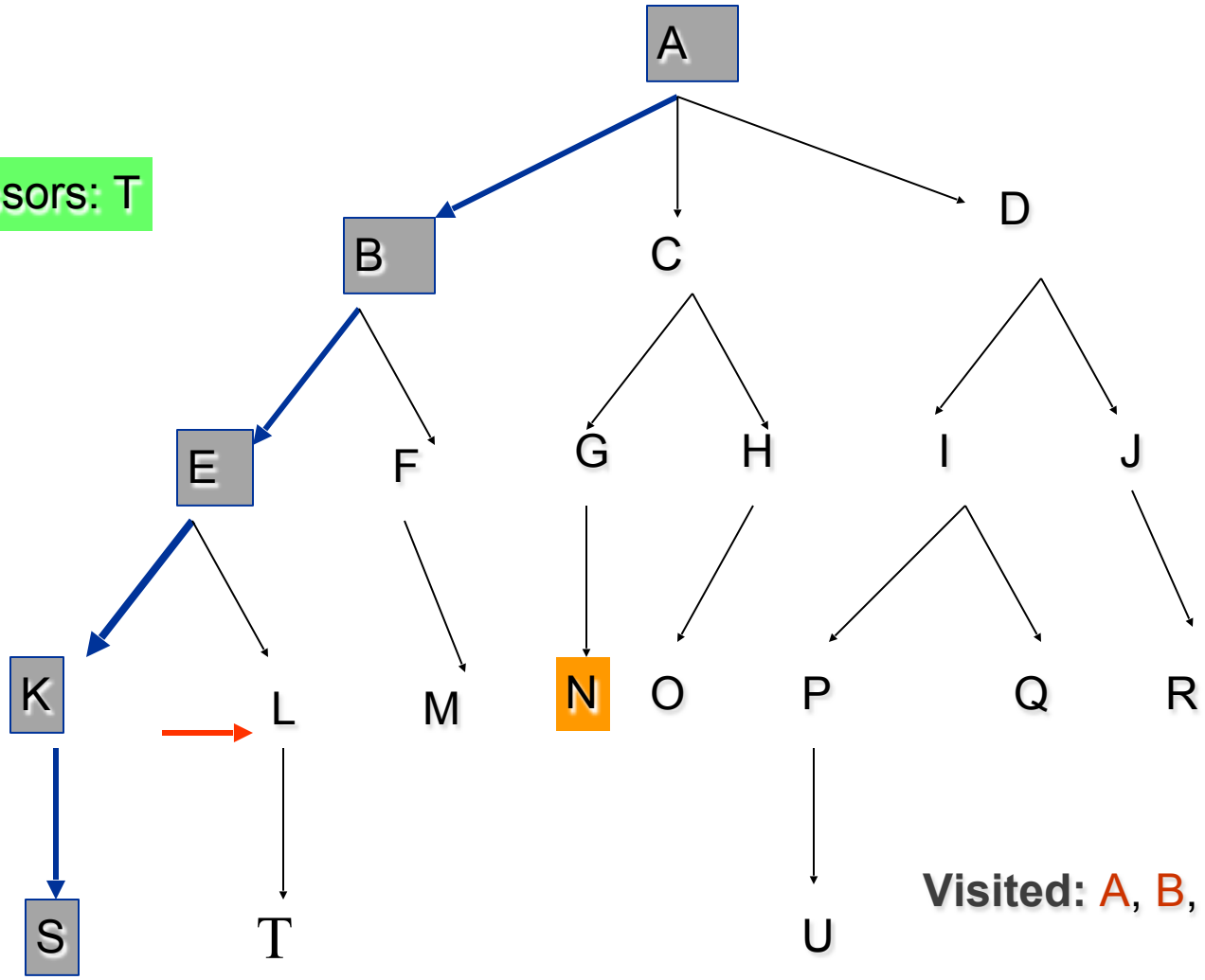
Fringe: K,L,F,C,D (LIFO)

Successors:



Fringe: S,L,F,C,D (LIFO)

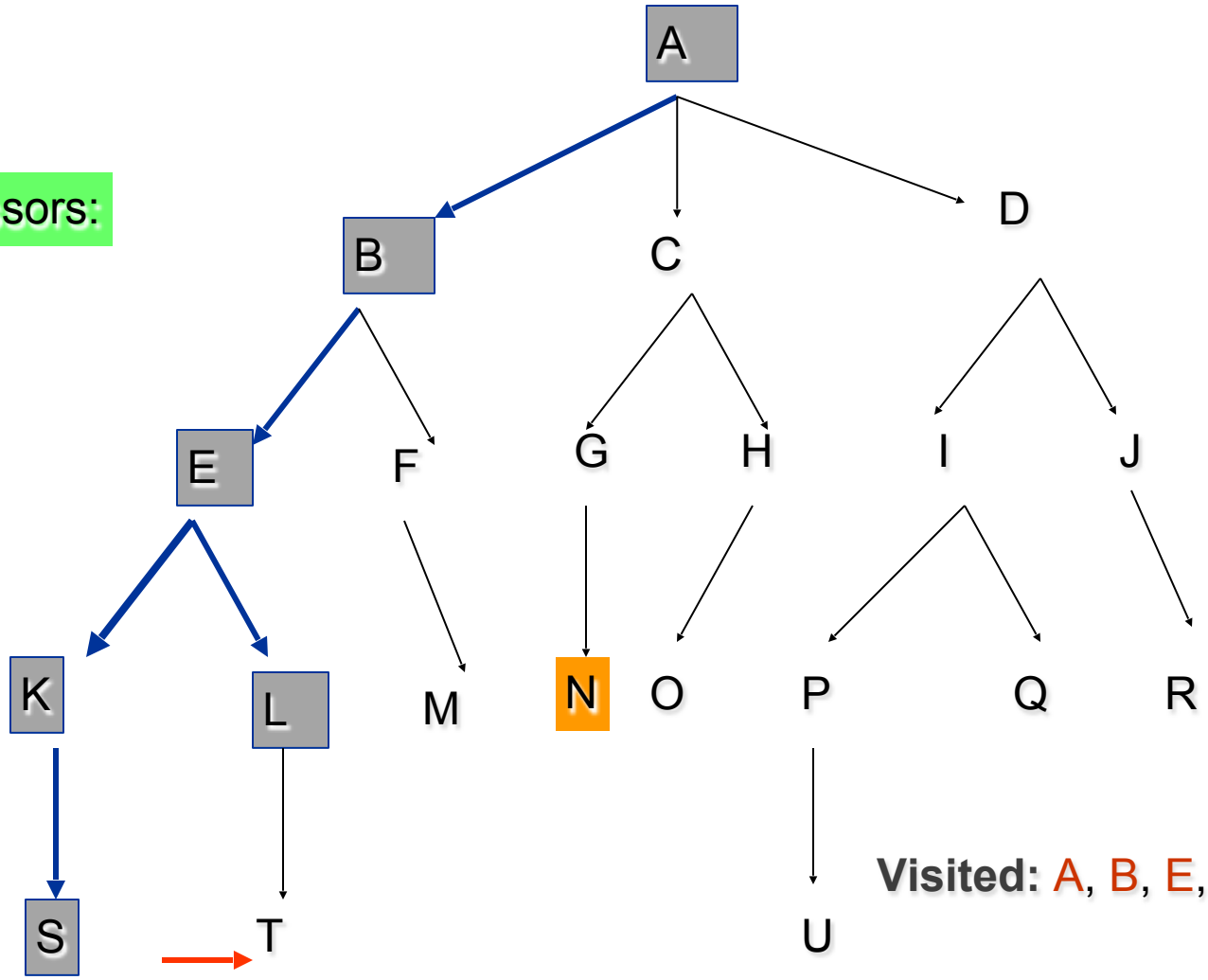
Successors: T



Visited: A, B, E, K, S

Fringe: L,F,C,D (LIFO)

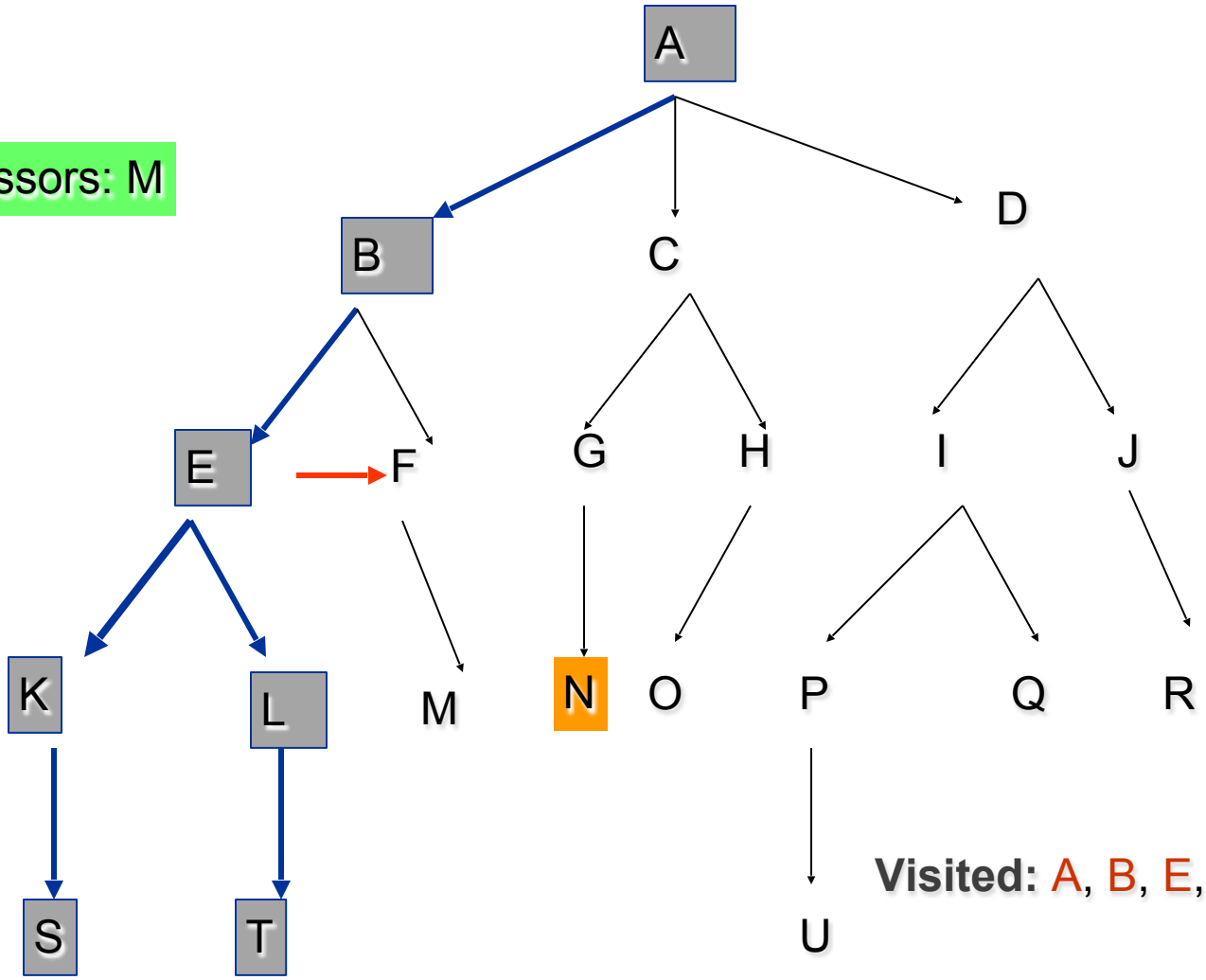
Successors:



Visited: A, B, E, K, S, L

Fringe: T,F,C,D (LIFO)

Successors: M

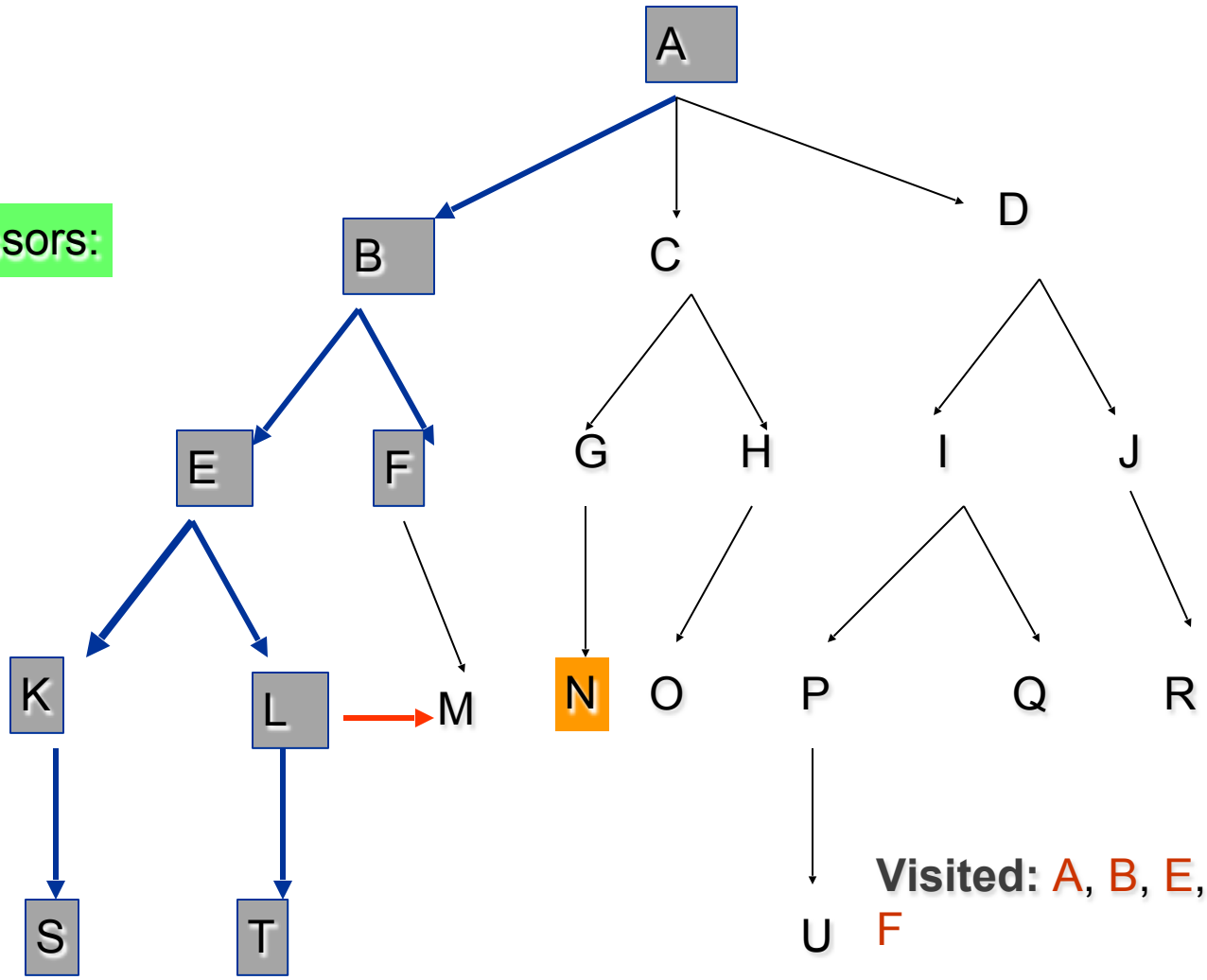


Visited: A, B, E, K, S, L, T

Fringe: F,C,D (LIFO)

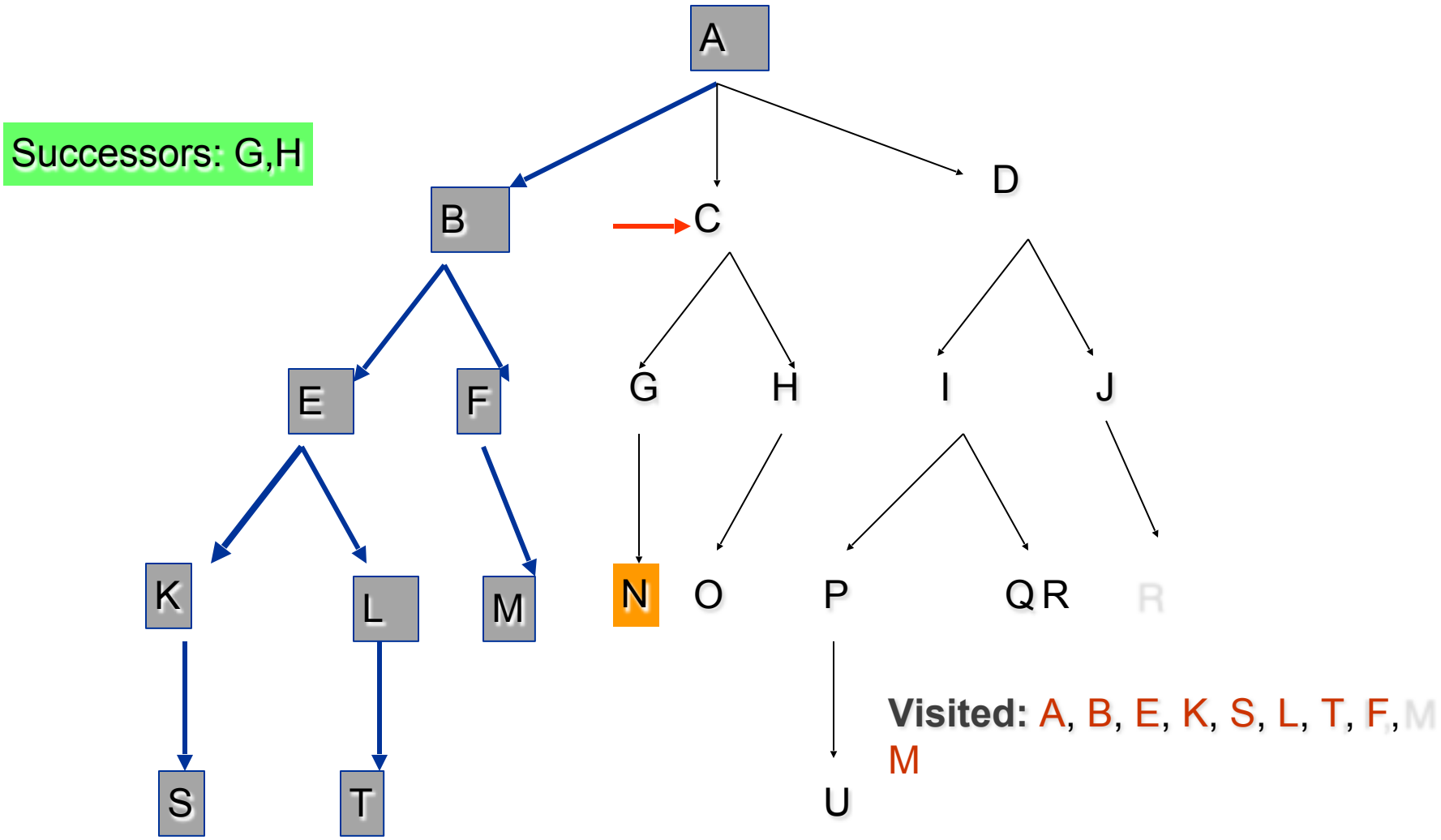
Backtracking

Successors:

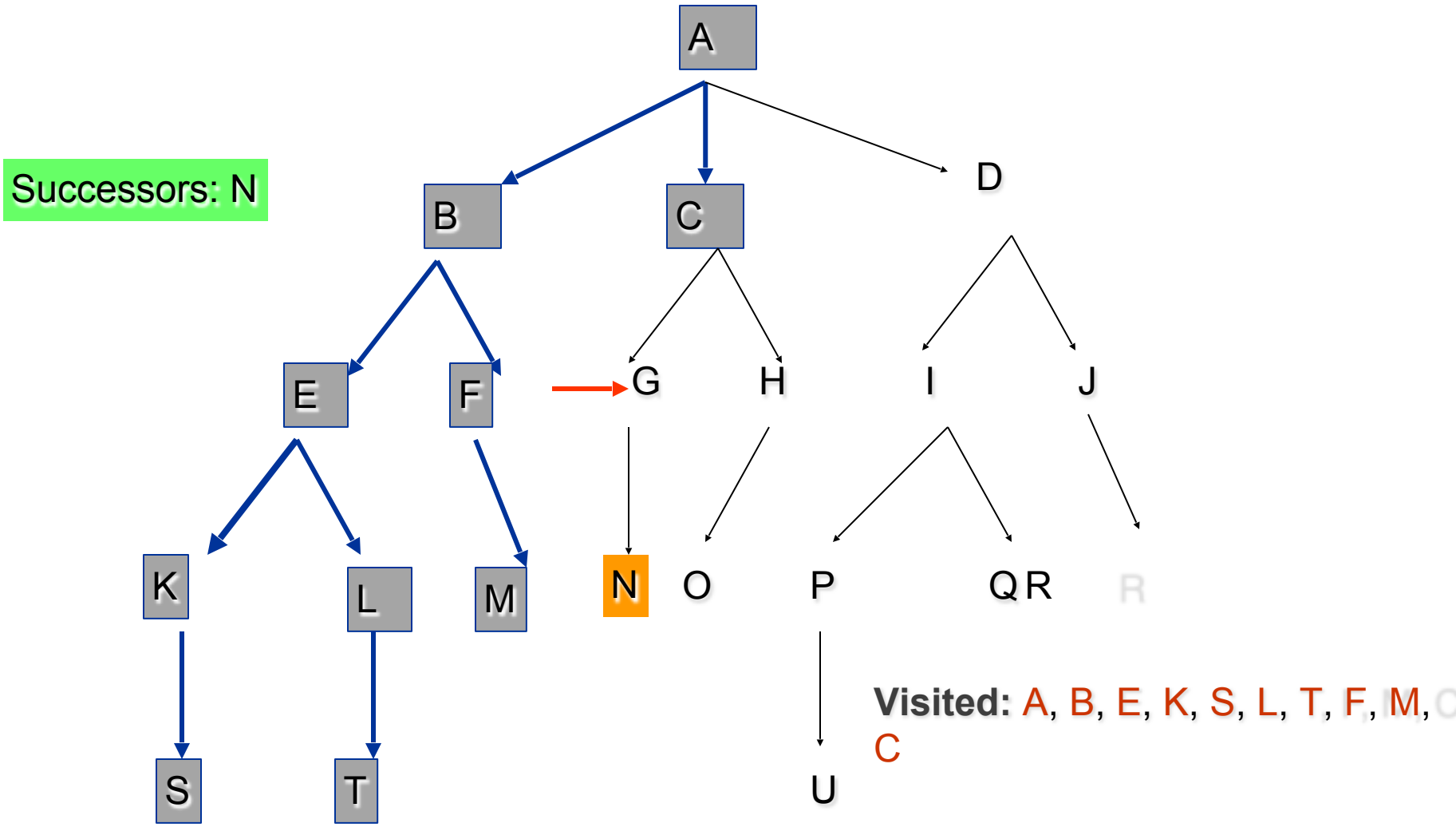


Visited: A, B, E, K, S, L, T, F

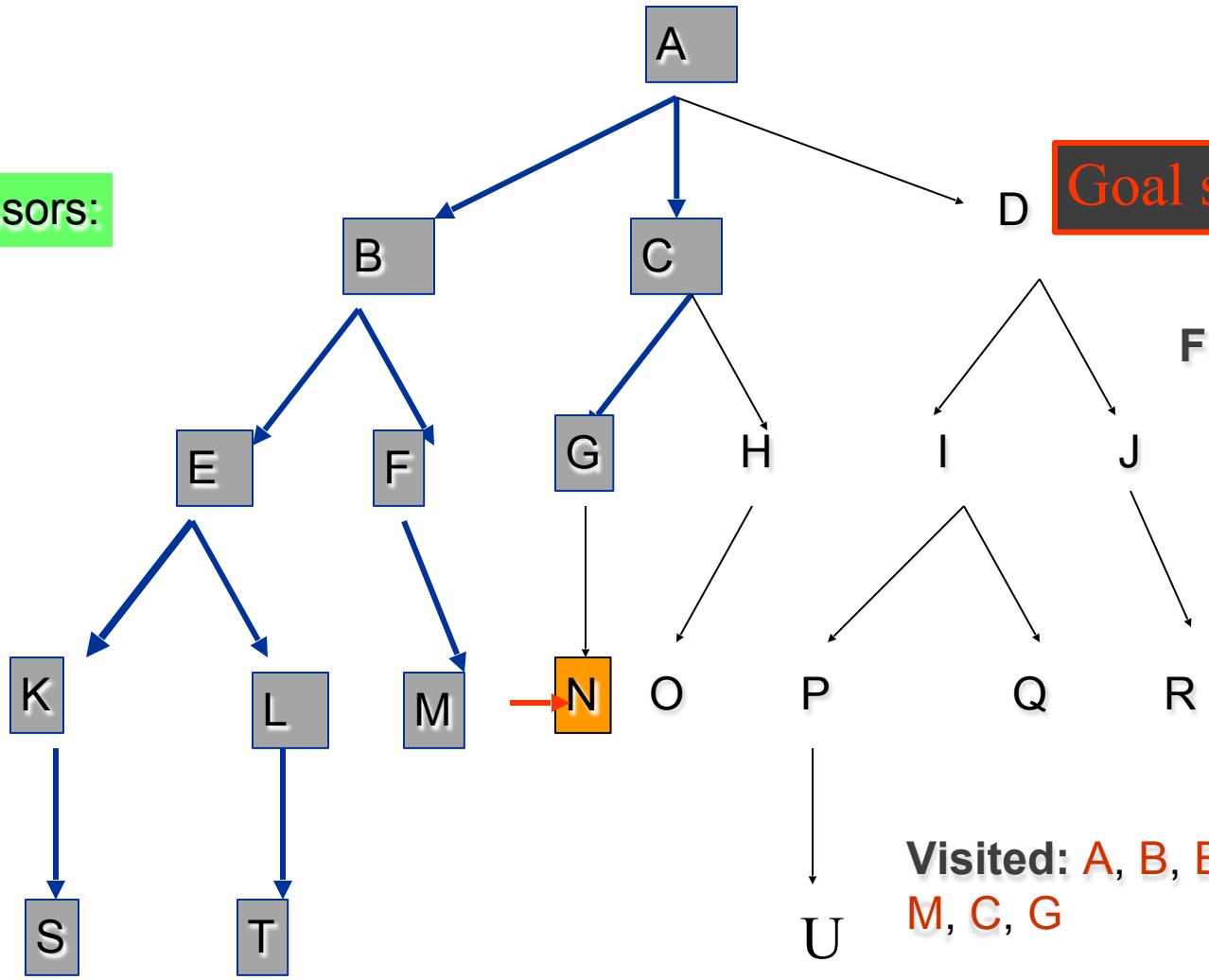
Fringe: M,C,D (LIFO)



Backtracking



Fringe: G,H,D (LIFO)



Successors:

Goal state achieved

Finished search

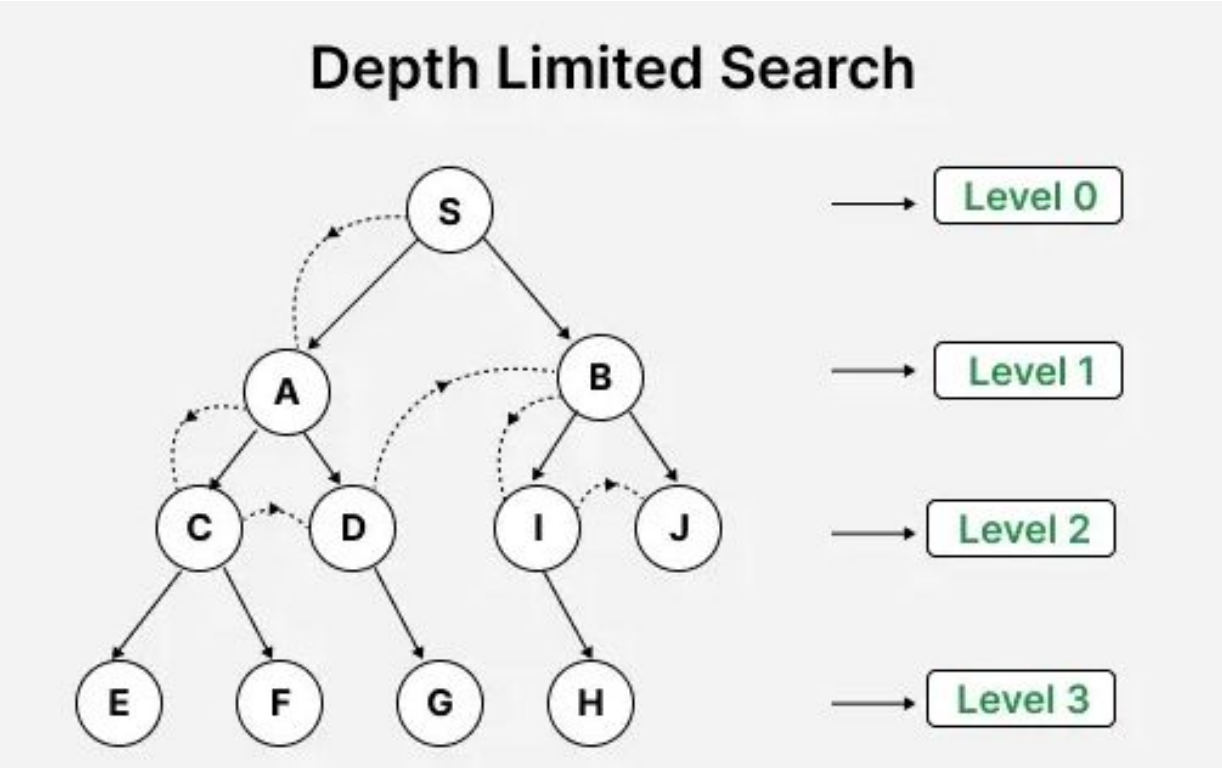
Visited: A, B, E, K, S, L, T, F, M, M, C, G

Fringe: N,H,D (LIFO)

DEPTH LIMITED SEARCH

- **Depth-Limited Search (DLS)** is a modified version of **Depth-First Search (DFS)** in which a **maximum depth limit** is specified. The search explores nodes only up to the given depth and does not go beyond it, preventing infinite paths.
- Like Depth first search, but the search is limited to a predefined depth.
- The depth of each state is recorded as it is generated.
- When picking the next state to expand, only those with depth less or equal than the current depth are expanded.

DEPTH LIMITED SEARCH



Suppose the **depth limit = 2**.

ITERATIVE DEEPENING SEARCH

Iterative Deepening Search (IDS) is a search algorithm that combines the advantages of **Depth-First Search (DFS)** and **Breadth-First Search (BFS)**. It repeatedly performs **Depth-Limited Search (DLS)** with increasing depth limits until the goal is found.

How It Works

1. Start with a depth limit of **0**.
2. Perform **Depth-Limited Search (DLS)**.
3. If the goal is not found, increase the depth limit by **1**.
4. Repeat the search until the goal is found.

ITERATIVE DEEPENING SEARCH L=0

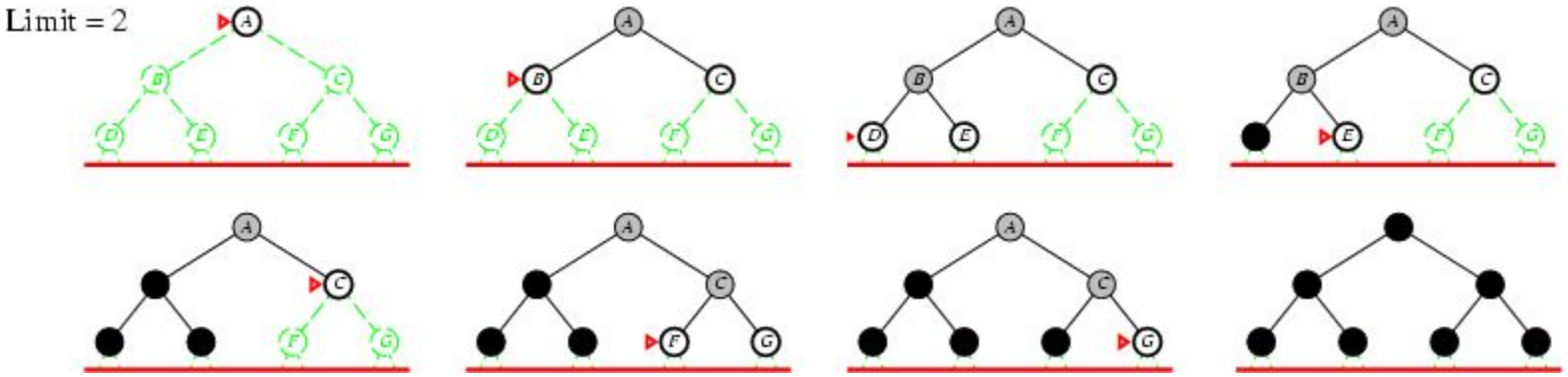
Limit = 0



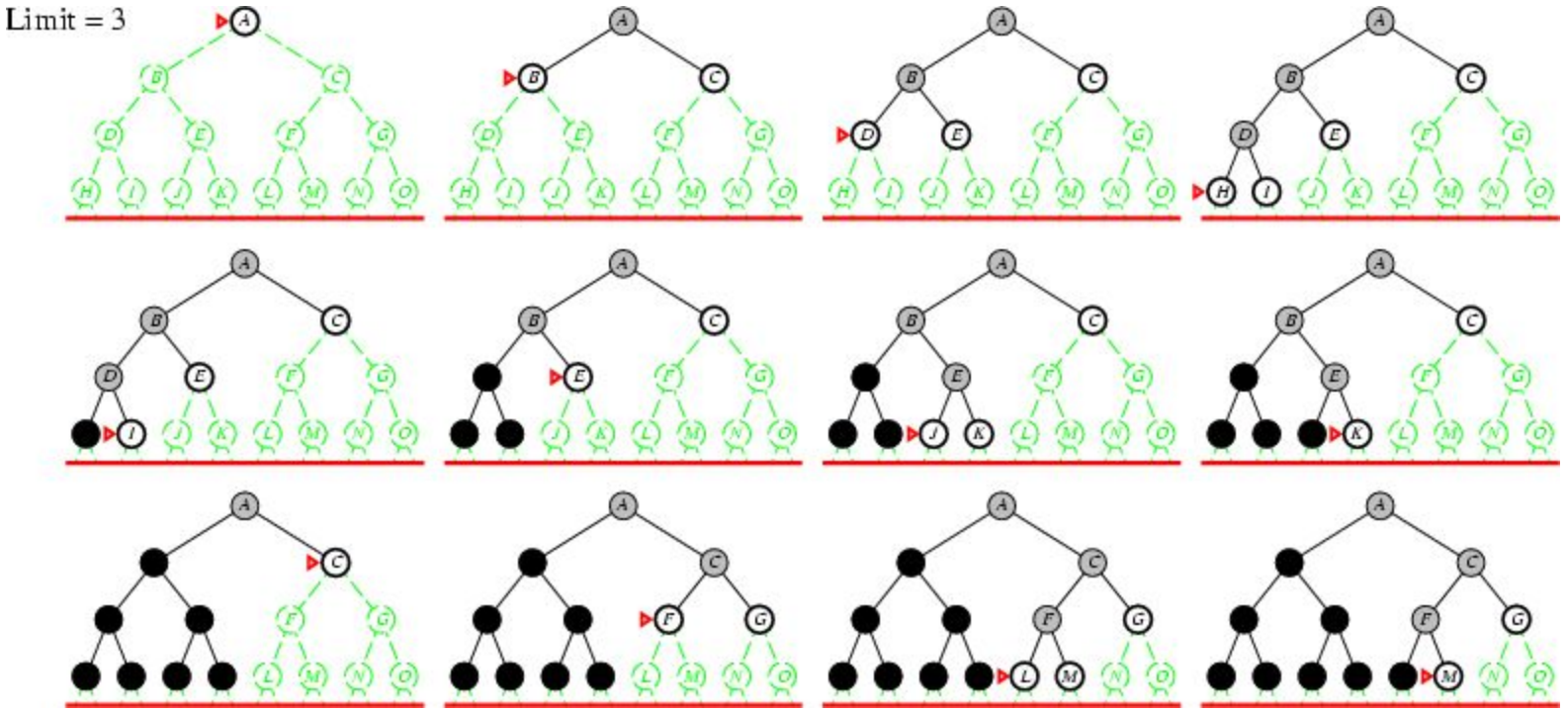
ITERATIVE DEEPENING SEARCH L=1



ITERATIVE DEEPENING SEARCH L=2



ITERATIVE DEEPENING SEARCH L=3



UNINFORMED SEARCH STRATEGIES

Algorithm	Data Structure	Complete	Optimal
BFS	Queue (FIFO)	✅ Yes	✅ Yes (equal costs)
DFS	Stack (LIFO)	❌ No	❌ No
UCS	Priority Queue	✅ Yes	✅ Yes
DLS	Stack (LIFO)	❌ Depends on limit	❌ No
IDS	Stack (LIFO)	✅ Yes	✅ Yes (equal costs)

UNINFORMED SEARCH STRATEGIES

Advantages

1. Does not require heuristic information.
2. Easy to implement.
3. Applicable when no domain knowledge is available.

Disadvantages

1. Explores many unnecessary nodes.
2. Can be slow for large search spaces.
3. Higher time and memory requirements than informed search.

UNINFORMED SEARCH STRATEGIES

Applications

1. Maze solving
2. Tree and graph traversal
3. Puzzle solving
4. Robot navigation
5. AI planning

INFORMED SEARCH ALGORITHMS

Informed Search Algorithms (also called **Heuristic Search Algorithms**) use **problem-specific knowledge (heuristics)** to guide the search toward the goal efficiently. Unlike uninformed search, informed search uses a **heuristic function h(n)h(n)h(n)** to estimate the cost from the current node to the goal.

INFORMED SEARCH ALGORITHMS

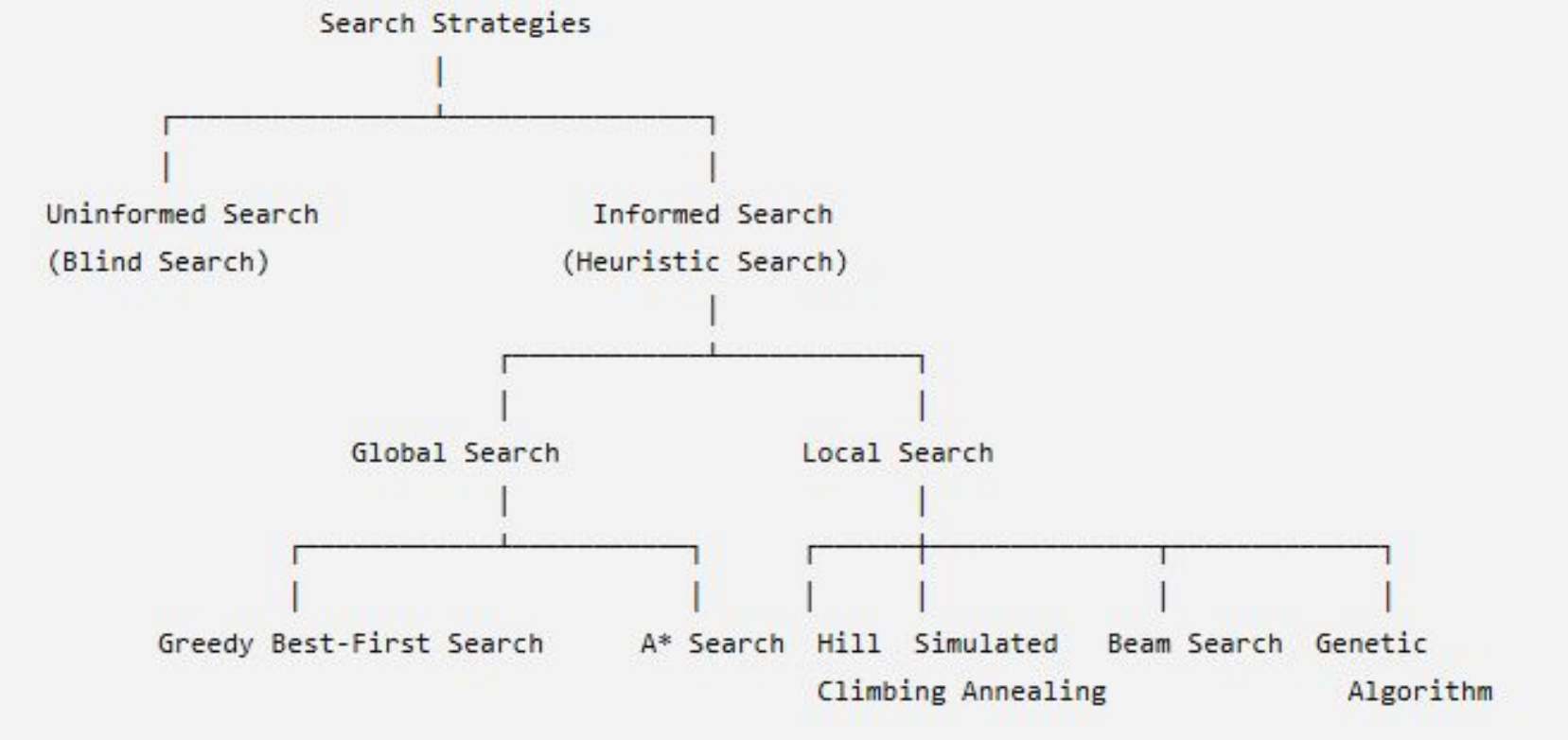
Heuristic Function

A **heuristic function** estimates how close a node is to the goal.

h(n)=Estimated cost from node n to the goal The search algorithm

chooses the node with the **lowest estimated cost**.

INFORMED SEARCH ALGORITHMS



GREEDY BEST-FIRST SEARCH

Greedy Best-First Search (GBFS) is an **informed search algorithm** that selects the node that appears to be **closest to the goal** based on a **heuristic function**.

It uses only the heuristic value:

f(n)=h(n)

where:

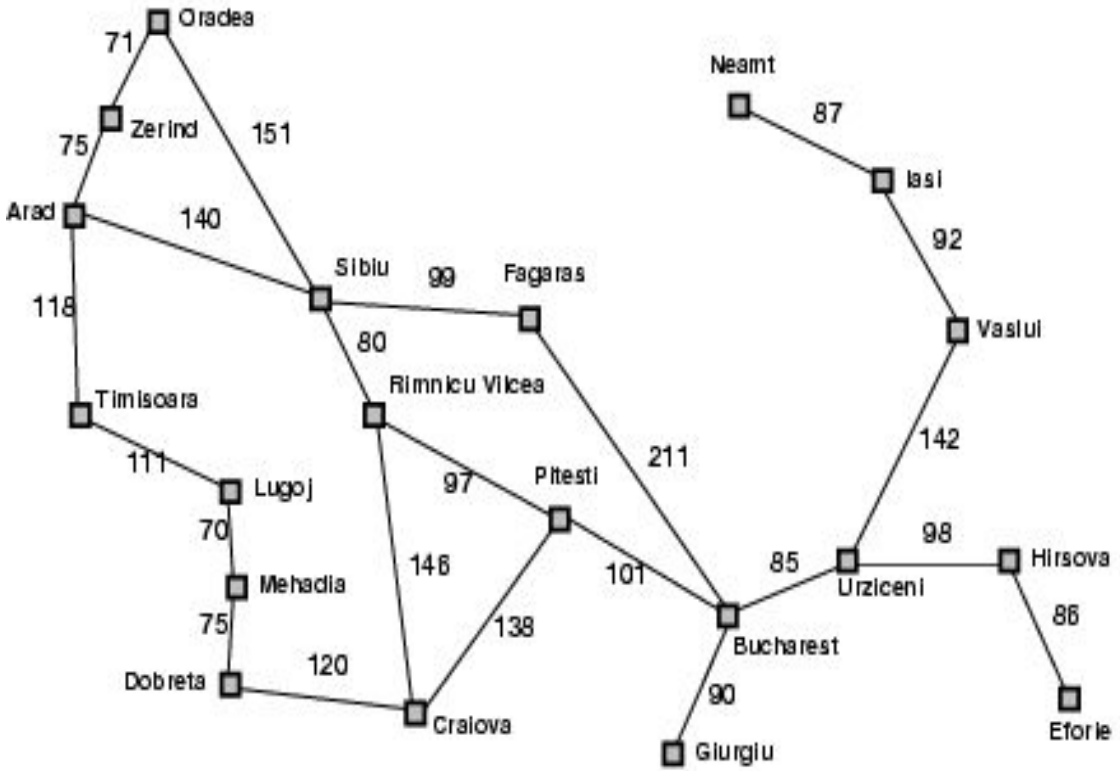
h(n) = Estimated cost (distance) from the current node to the goal.

GREEDY BEST-FIRST SEARCH

How It Works

1. Start from the initial node.
2. Calculate the heuristic value **h(n)** for all neighboring nodes.
3. Select the node with the **smallest heuristic value**.
4. Repeat until the goal node is reached.

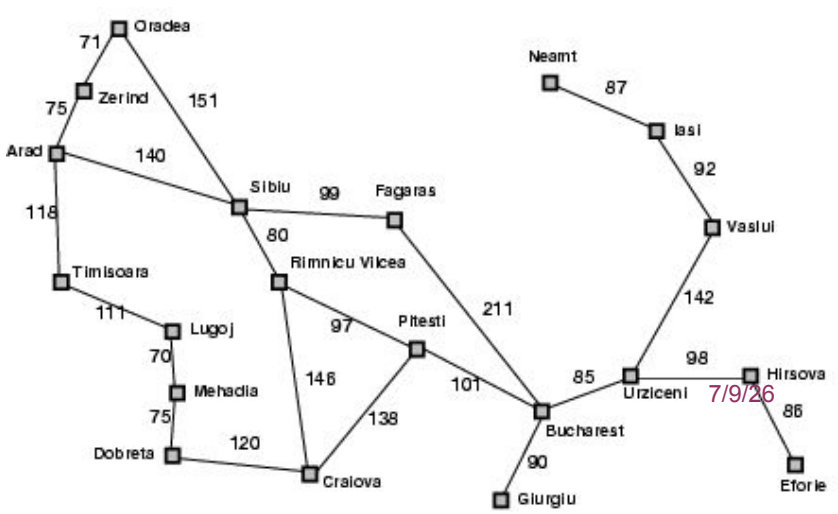
GREEDY BEST-FIRST SEARCH



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

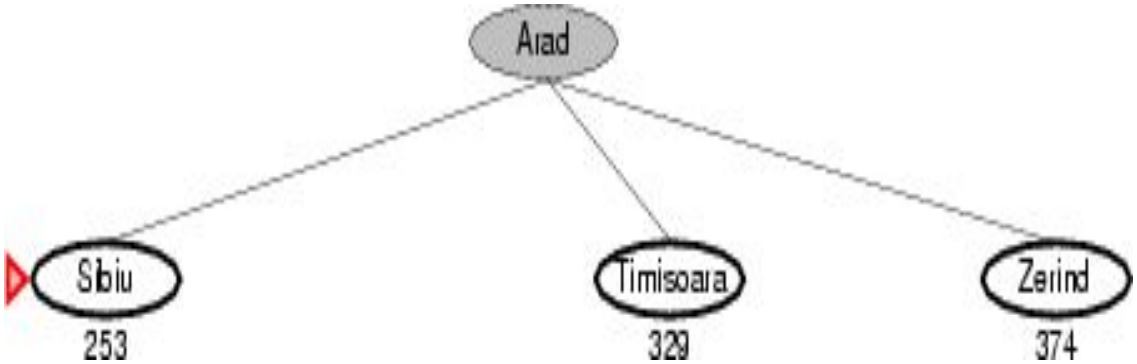
GREEDY BEST-FIRST SEARCH

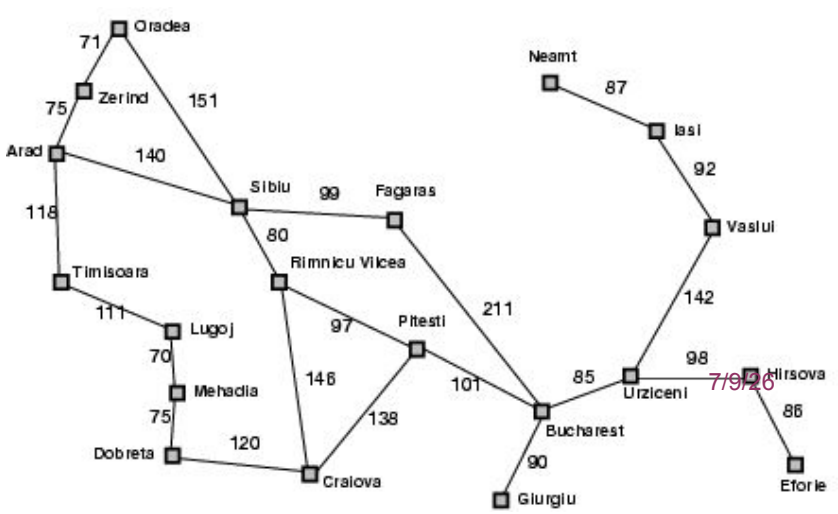




Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

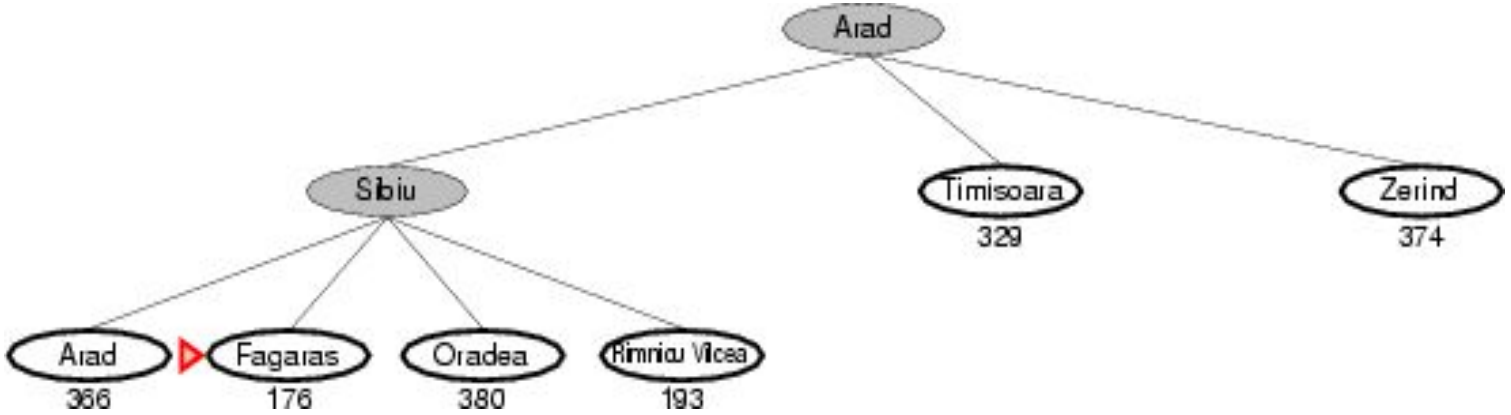
GREEDY BEST-FIRST SEARCH

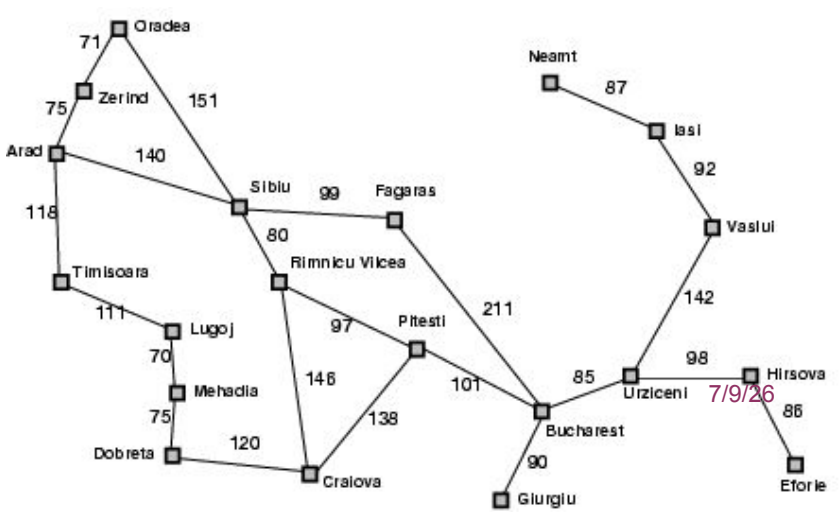




Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

GREEDY BEST-FIRST SEARCH

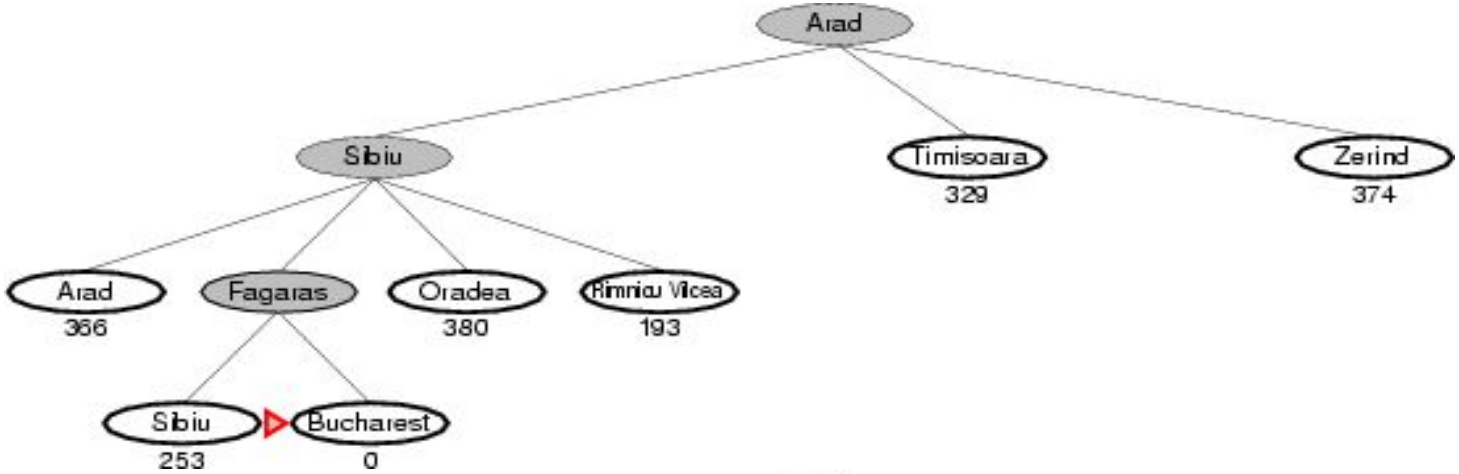


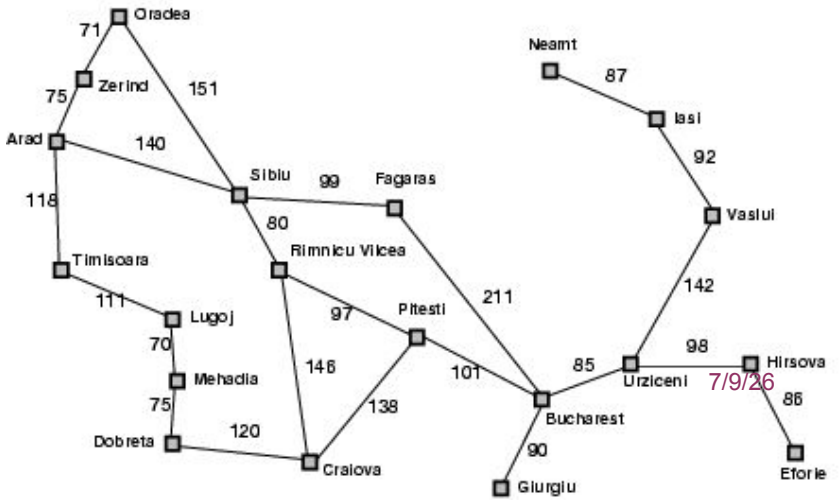


Straight–line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

GREEDY BEST-FIRST SEARCH





Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

GREEDY BEST-FIRST SEARCH

Advantages

- Faster than uninformed search algorithms.
- Uses heuristic information to guide the search.
- Easy to implement.

Disadvantages

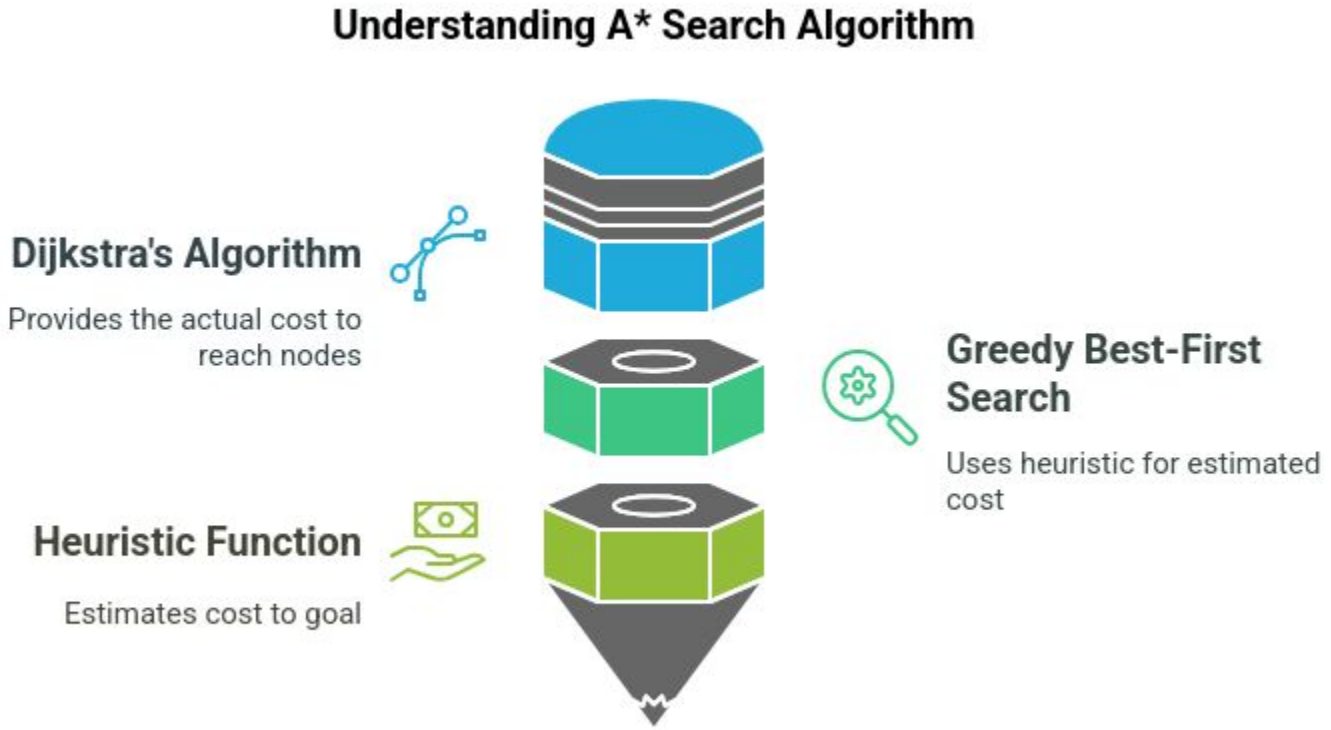
- Does **not guarantee the shortest path**.
- May get trapped in a local optimum.
- Performance depends on the quality of the heuristic.

GREEDY BEST-FIRST SEARCH

- Complete? No – can get stuck in loops.
- Time? $O(b^m)$, but a good heuristic can give dramatic improvement
- Space? $O(b^m)$ - keeps all nodes in memory
- Optimal? No

A* SEARCH

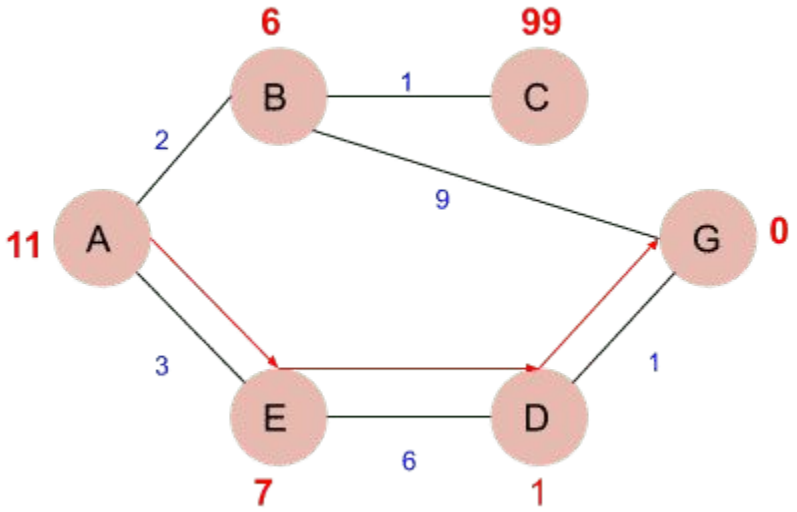
The A* Search Algorithm combines Dijkstra's and Greedy Search to find optimal paths in AI, robotics, and games.



What is A* Search Algorithm?

The A* Search Algorithm is an optimal and efficient method for solving path finding problems. When seeking the **shortest path** between a starting point and a goal, A* is often the best choice due to its versatility and robustness.

` Widely used in robotics, video games, and navigation systems, A* is one of the most important algorithms in graph traversal.



In a*,

1. **Space** is represented as a graph,
2. **Nodes** represent positions or states
3. **Edges** represent transitions between states. Edges are associated with costs or a given set of rules.

A* SEARCH

A* (A-Star) Search is an **informed search algorithm** that finds the **shortest and optimal path** from the initial state to the goal state. It combines:

1. **Actual path cost** from the start node (**g(n)**)
2. **Estimated cost** from the current node to the goal (**h(n)**)

- Idea: avoid expanding paths that are already expensive
- Evaluation function f(n) = g(n) + h(n)
- g(n) = cost so far to reach n
- h(n) = estimated cost from n to goal
- f(n) = estimated total cost of path through n to goal

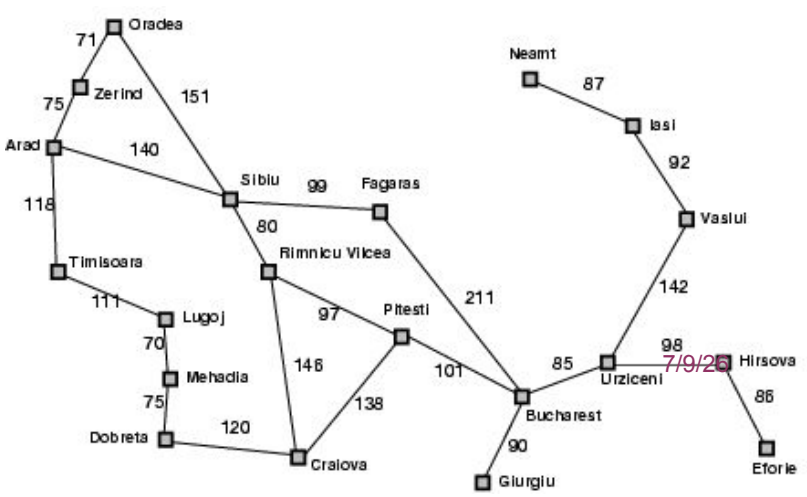
A* SEARCH

Algorithm

1. Place the start node in the **OPEN list**.
2. Calculate **f(n) = g(n) + h(n)**.
3. Select the node with the **smallest f(n)**.
4. Move it to the **CLOSED list**.
5. Generate all child nodes.
6. Compute **g(n), h(n), and f(n)** for each child.
7. Add unexplored nodes to the OPEN list.
8. Repeat until the goal is found.

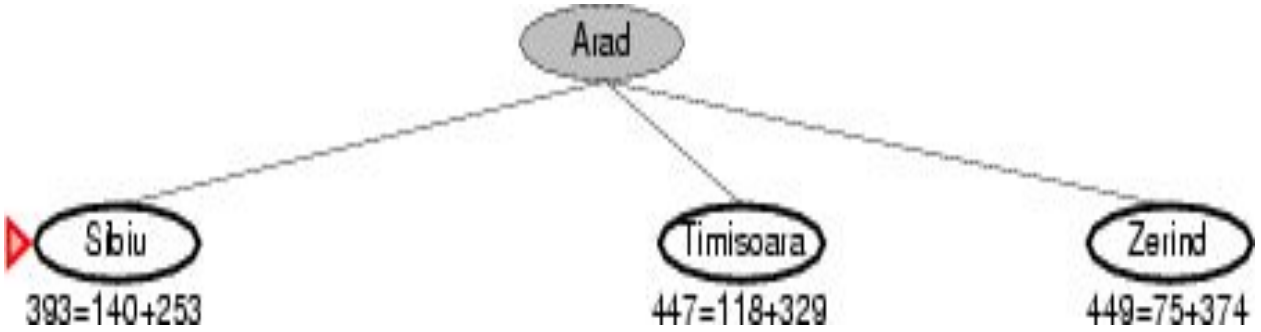
A* SEARCH EXAMPLE

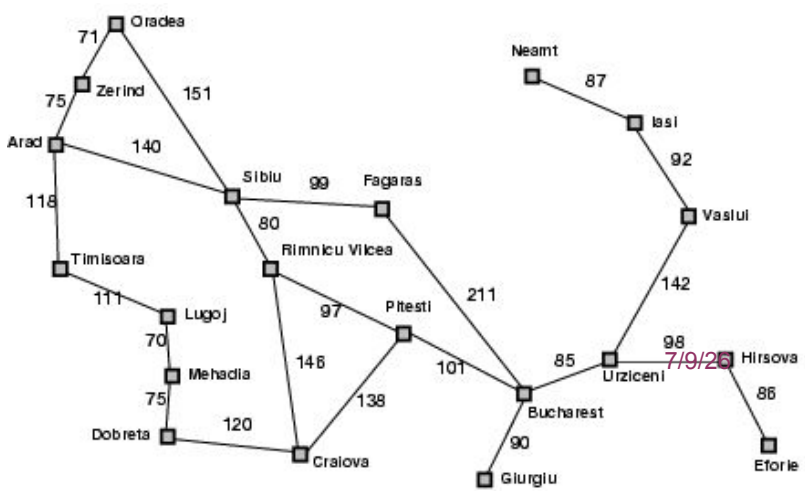




Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

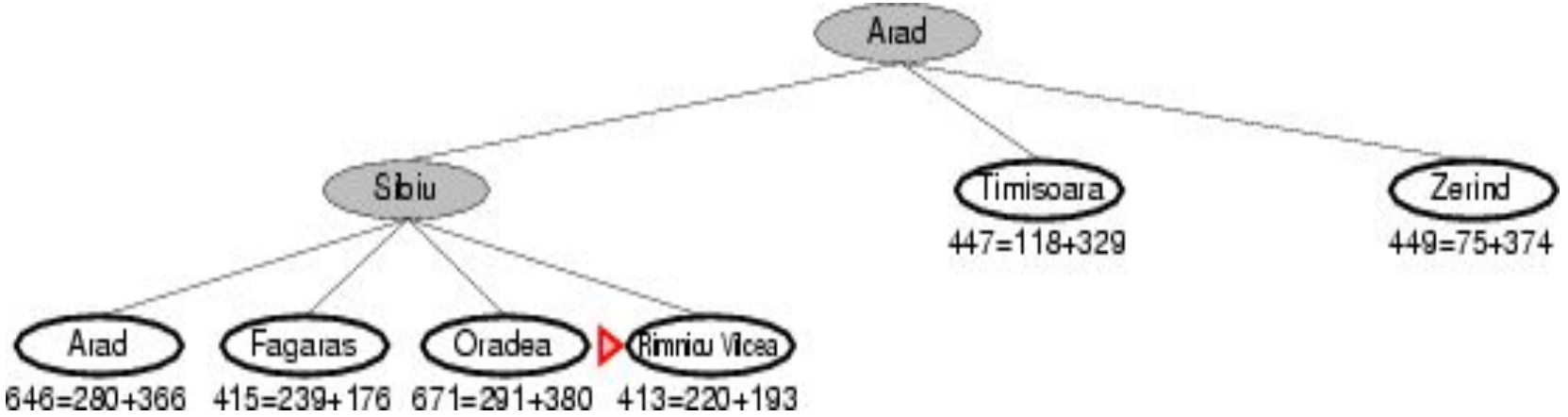
A* SEARCH EXAMPLE

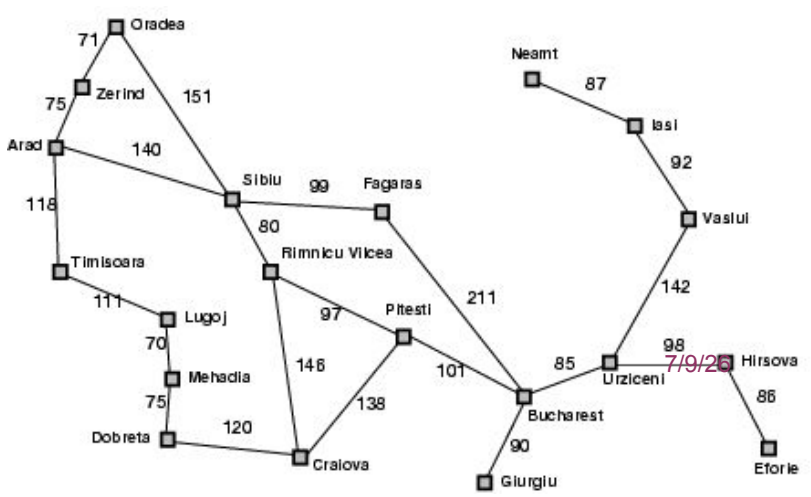




Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

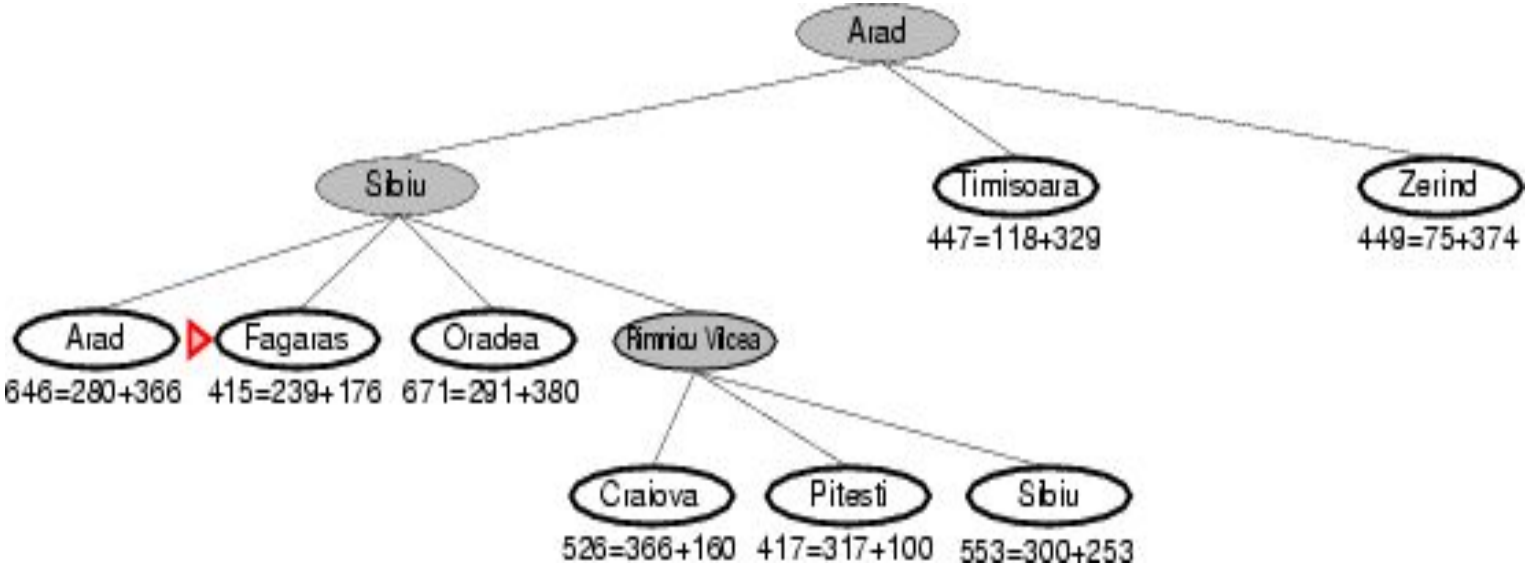
A* SEARCH EXAMPLE

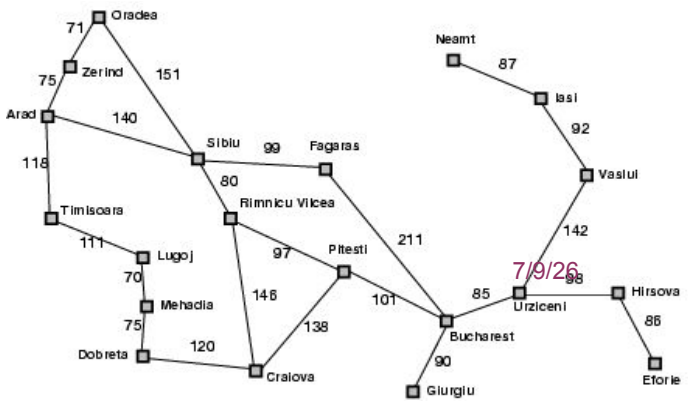




Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

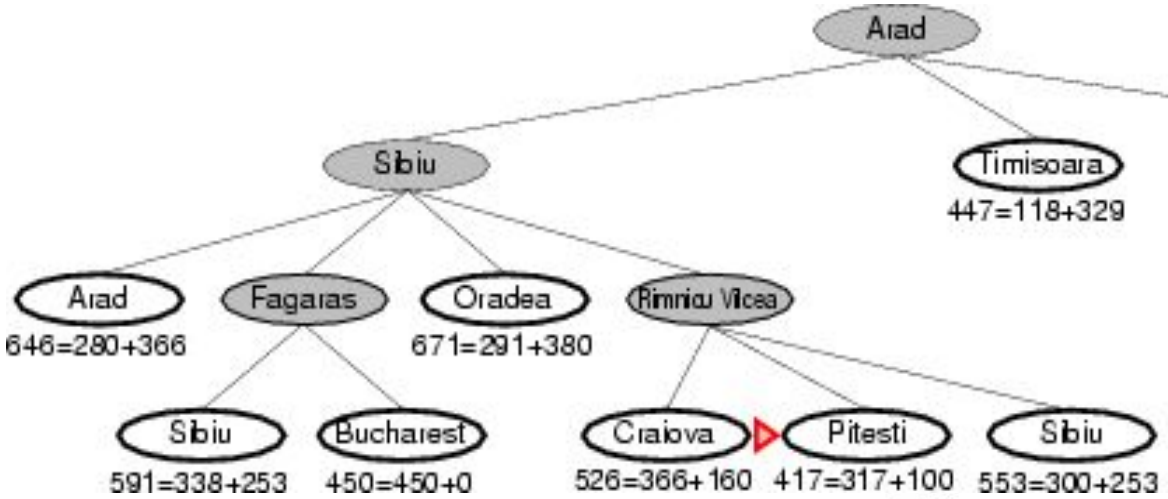
A* SEARCH EXAMPLE

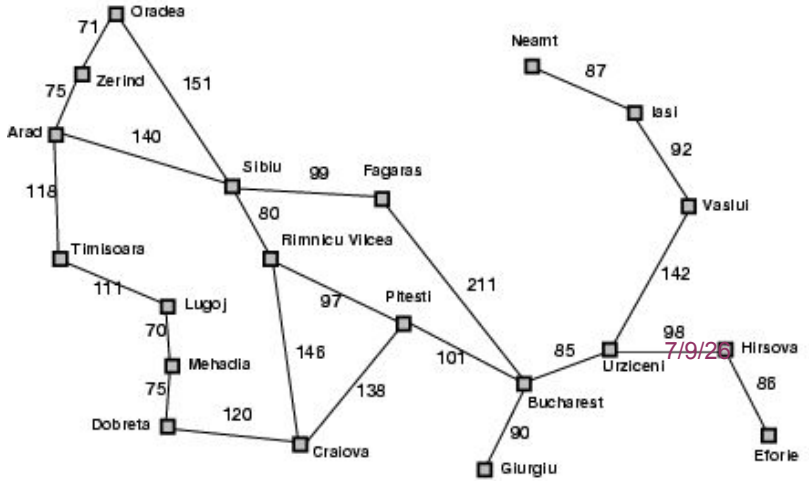




Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

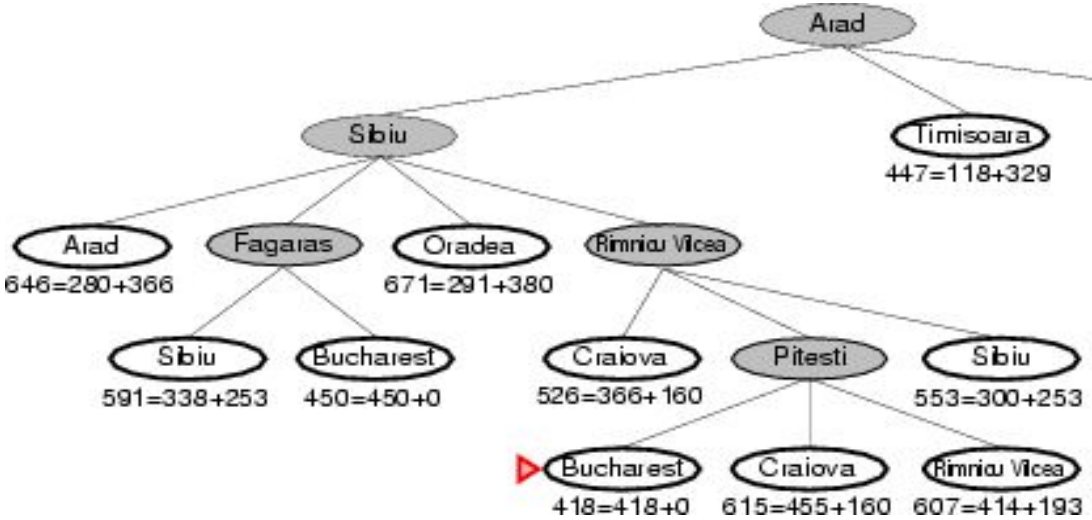
A* SEARCH EXAMPLE

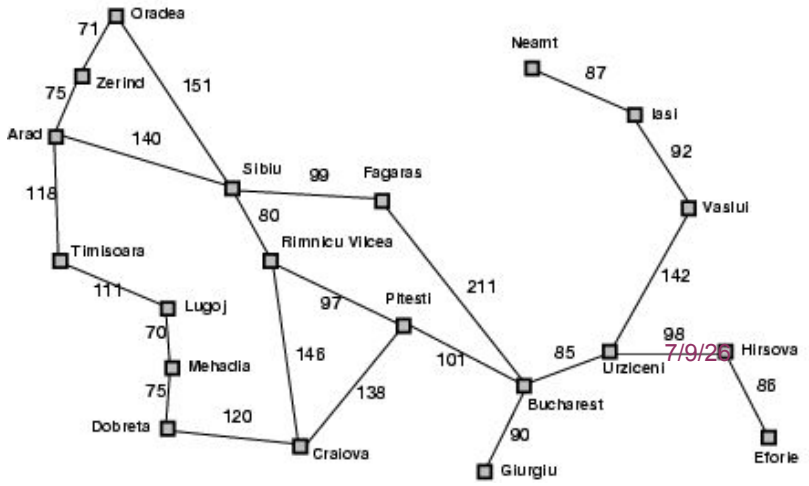




Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

A* SEARCH EXAMPLE





Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

A* SEARCH

Advantages

1. Finds the **optimal (shortest) path** if the heuristic is admissible.
2. More efficient than uninformed search algorithms.
3. Widely used in AI and robotics.

Disadvantages

1. Requires more memory to store OPEN and CLOSED lists.
2. Performance depends on the quality of the heuristic.
3. Can be slow for very large search spaces.

ADMISSIBLE HEURISTICS

An **admissible heuristic** is a heuristic function that **never overestimates** the actual minimum cost to reach the goal state.
In other words,

$$h(n) \leq h^*(n)$$

where:

h(n) = Estimated cost from node **n** to the goal.

h*(n) = Actual minimum cost from node **n** to the goal.

An admissible heuristic is **optimistic**, meaning it either gives the exact cost or an underestimate, but never an overestimate.

A* EXAMPLE

8-PUZZLE HEURISTICS

1) MISPLACED TILE HEURISTIC (h_1)

$h_1(n)$ = Number of tiles that are not in their goal position.
(Blank tile '0' is not counted)

Current State (S)

2	8	3
1	6	4
7	0	5

Goal State (G)

1	2	3
4	5	6
7	8	0

Misplaced tile
Correct tile
Blank (ignored)

2	8	3
1	6	4
7	0	5

Misplaced tiles = 2, 8, 1, 6, 4, 5 → Total = 6

$h_1(S) = 6$

2) MANHATTAN DISTANCE HEURISTIC (h_2)

$h_2(n)$ = Sum of the Manhattan distances of each tile from its goal position.
(Blank tile '0' is not counted)

Current State (S)

2	8	3
1	6	4
7	0	5

Goal State (G)

1	2	3
4	5	6
7	8	0

Tile	Current Position (row, col)	Goal Position (row, col)	Manhattan Distance (|Δrow| + |Δcol|)
1	(1, 0)	(0, 0)	[1-0] + [0-0] = 1
2	(0, 0)	(0, 1)	[0-0] + [0-1] = 1
3	(0, 2)	(0, 2)	[0-0] + [2-2] = 0
4	(1, 2)	(1, 0)	[1-1] + [2-0] = 2
5	(2, 2)	(1, 1)	[2-1] + [2-1] = 2
6	(1, 1)	(1, 2)	[1-1] + [1-2] = 1
7	(2, 0)	(2, 0)	[2-2] + [0-0] = 0
8	(0, 1)	(2, 1)	[0-2] + [1-1] = 2

$h_2(S) = 1 + 1 + 0 + 2 + 2 + 1 + 0 + 2 = 9$

Note: Lower heuristic value means the state is closer to the goal.

A* EXAMPLE

Heuristic	Value	Meaning
Misplaced Tiles (h_1)	**6**	6 tiles are in the wrong position
Manhattan Distance (h_2)	**9**	Total minimum tile movement is 9

A* EXAMPLE calculating g(n)

Initial State (Start)

```
2 8 3
1 6 4
7 _ 5
```

At the start:

- Moves made = **0**

Therefore,

$$\boxed{g(n) = 0}$$

Suppose we move the blank **RIGHT**.

A* EXAMPLE calculating g(n)

State 1

```
2 8 3
1 6 4
7 5 _
```

Moves made = **1**

Therefore,

$$\boxed{g(n) = 1}$$

Suppose from this state we move the blank **UP**.

A* EXAMPLE calculating g(n)

State 2

```
2 8 3
1 6 _
7 5 4
```

Moves made = **2**

Therefore,

$$\boxed{g(n) = 2}$$

A* EXAMPLE

Combining with the Heuristic

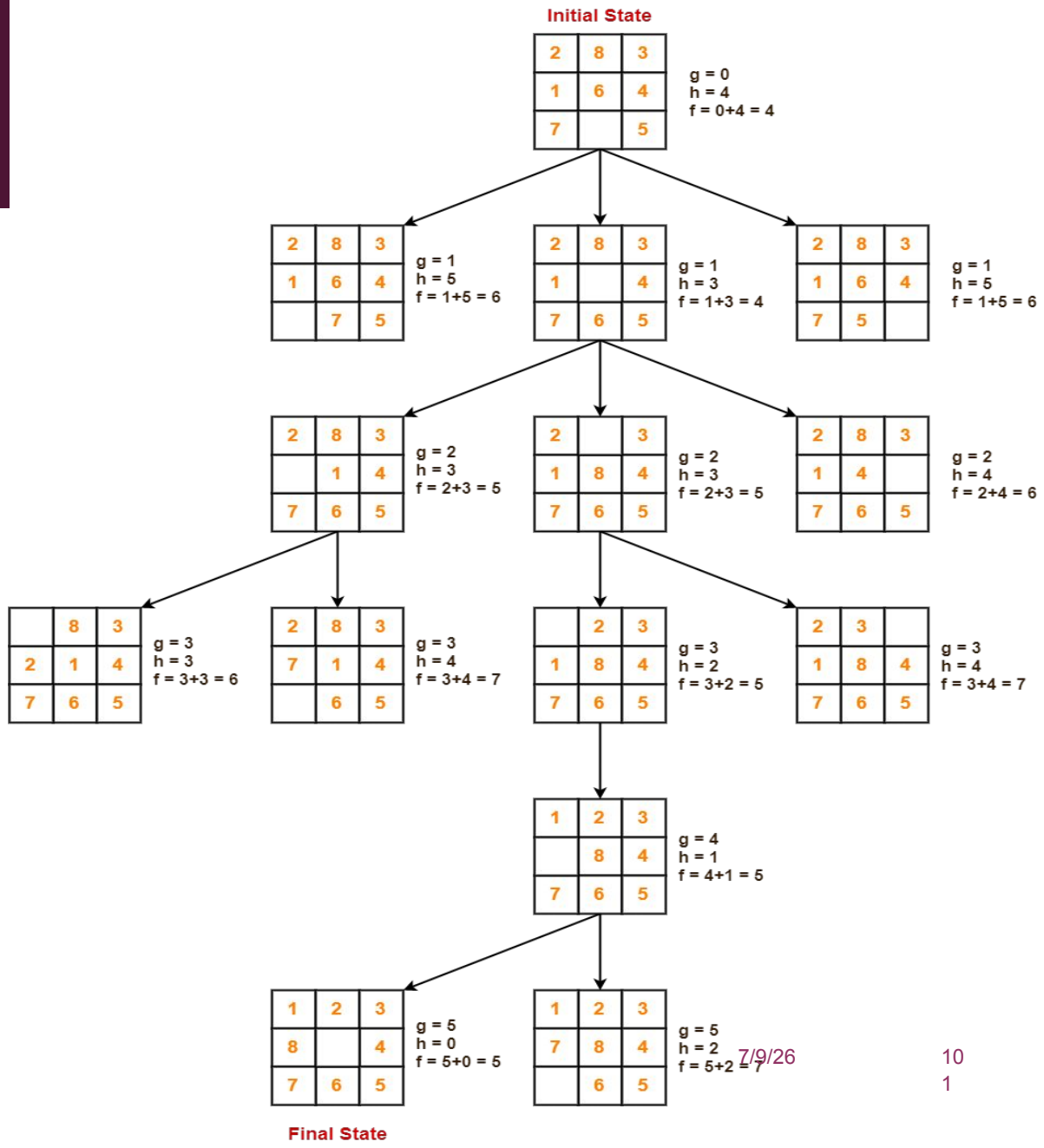
Assume the heuristic is **Misplaced Tiles**.

State	g(n)	h(n)	f(n)=g(n)+h(n)
Start	0	6	**6**
State 1	1	5	**6**
State 2	2	4	**6**

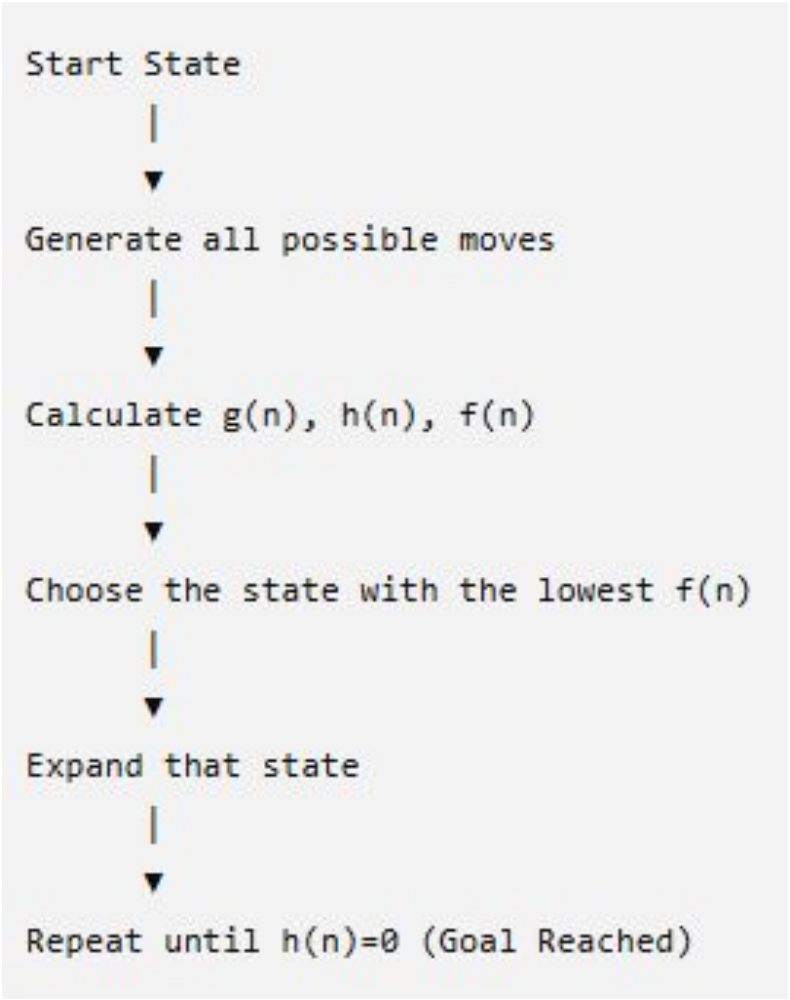
A* computes **f(n)** for every generated state and expands the state with the **lowest f(n).**

A* EXAMPLE

g(n) = Number of moves made from the **initial state**.
h(n) = Number of misplaced tiles (blank tile is ignored).
f(n) = Total estimated cost.



A* EXAMPLE


Start State
Generate all possible moves
Calculate g(n), h(n), f(n)
Choose the state with the lowest f(n)
Expand that state
Repeat until h(n)=0 (Goal Reached)

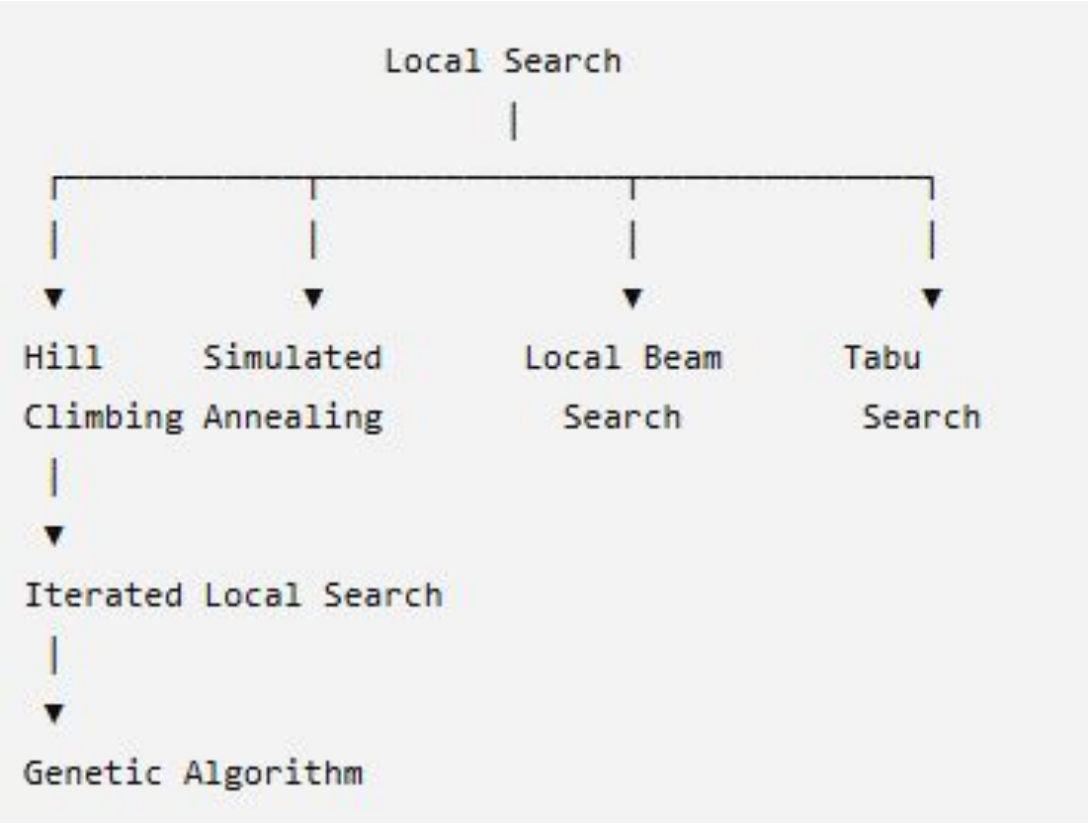
LOCAL SEARCH

Local Search is a search technique that starts with an initial solution and repeatedly moves to a **neighboring state** to improve the solution. Unlike global search algorithms, it **does not keep track of the path** taken; it focuses only on the **current state** and aims to find the best solution.

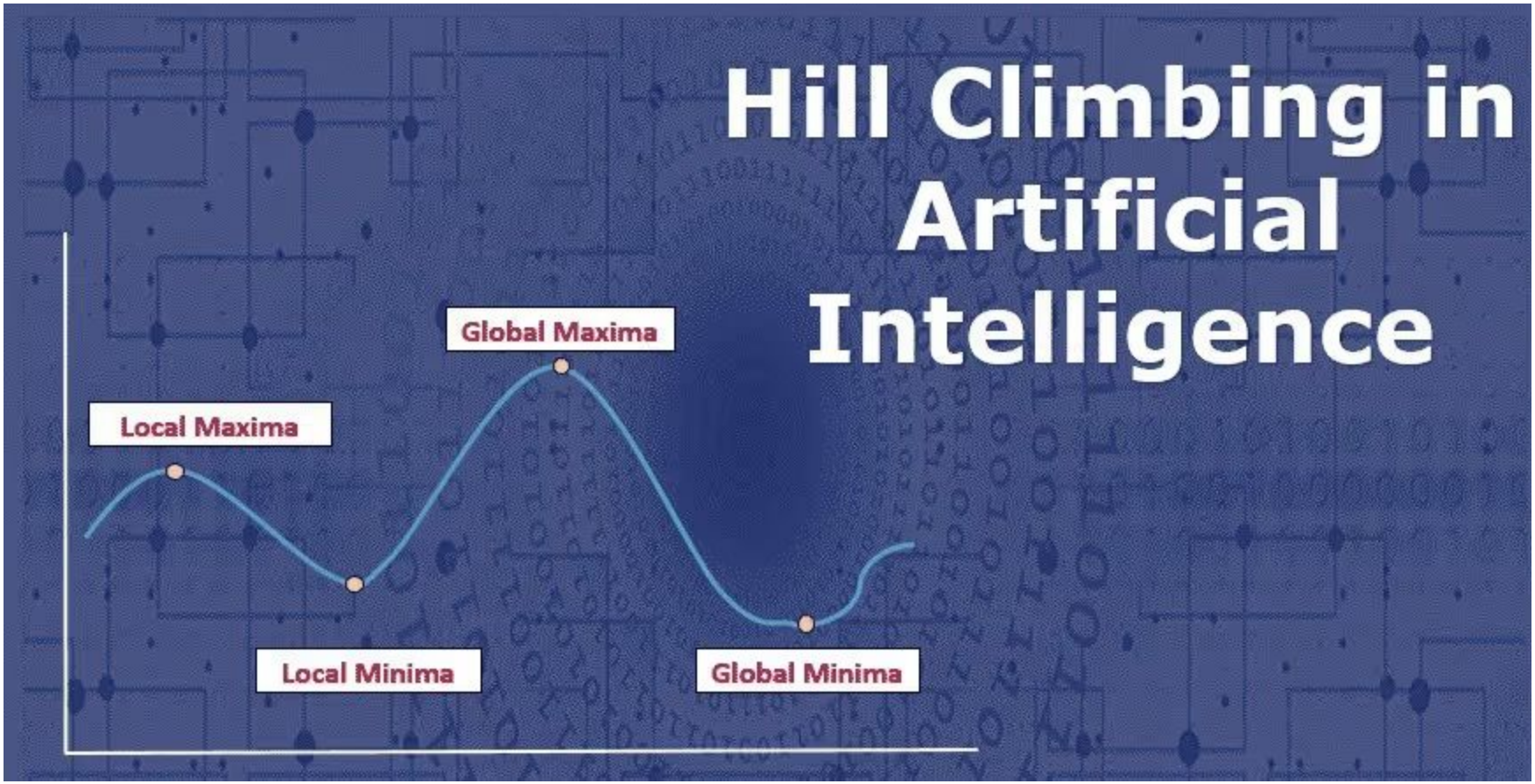
How It Works

1. Start with an initial state.
2. Generate neighboring states.
3. Evaluate each neighbor using an objective (evaluation) function.
4. Move to the best neighboring state.
5. Repeat until no better neighbor is found or the goal is reached.

LOCAL SEARCH



HILL CLIMBING



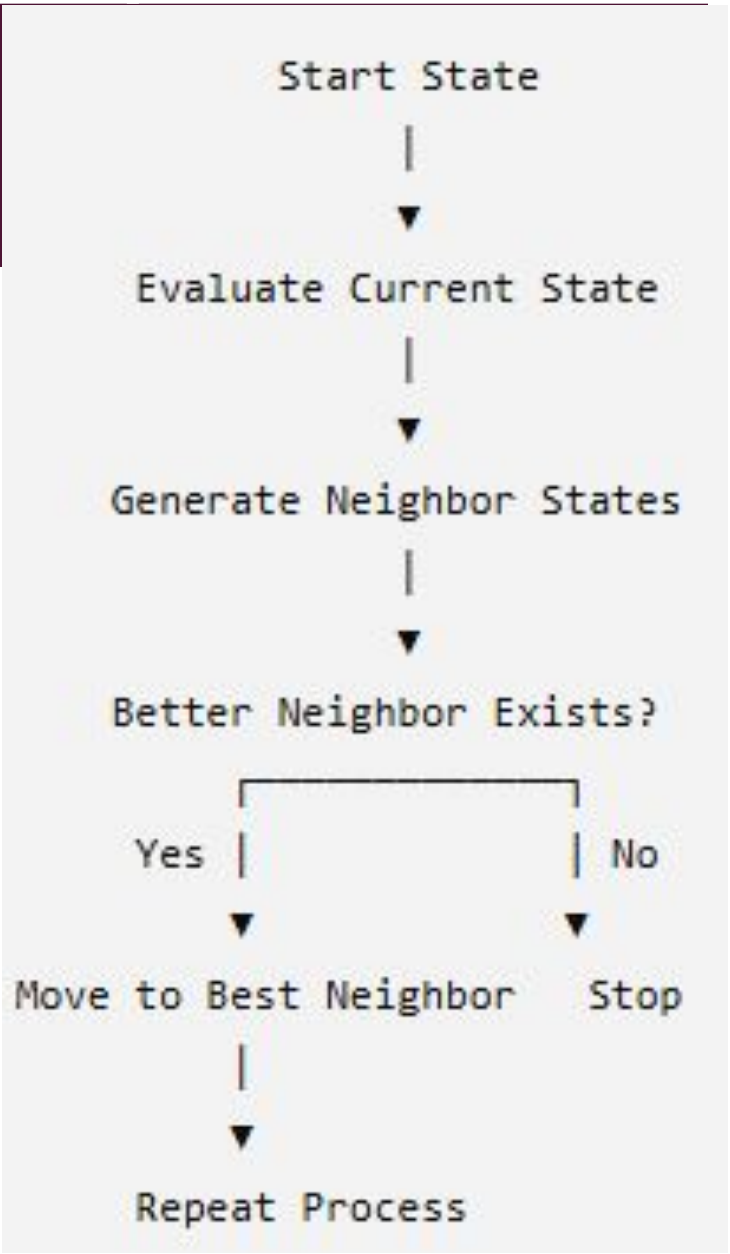
HILL CLIMBING

Generate-and-Test is a problem-solving strategy in which the algorithm:

1. **Generates** one or more possible neighboring solutions (candidate states).
2. **Tests (evaluates)** each solution using an evaluation or heuristic function.
3. **Moves** to the most promising solution according to predefined rules.

Repeats the process until a satisfactory or optimal solution is found.

Unlike BFS or DFS, local search does **not** remember the entire search tree. It focuses only on the **current state** and its neighboring states.



HILL CLIMBING

- Hill climbing is an optimization technique for solving computationally hard problems.
- Used in problems with “the property that the state description itself contains all the information”
- The algorithm is memory efficient since it does not maintain a search tree
- Hill climbing attempts to iteratively improvethe state current means of an evaluation function by
- Searching for a goal state = Climbing to the top of a hill

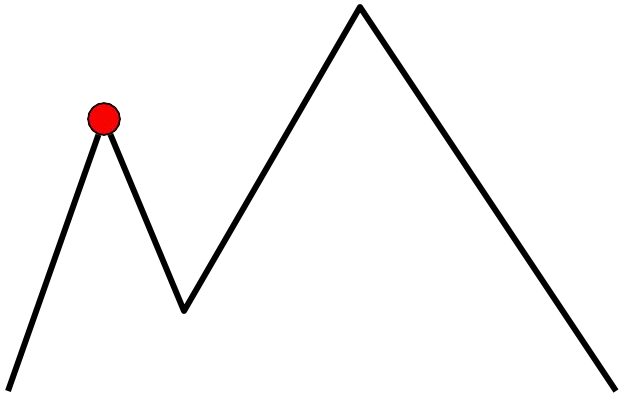
HILL CLIMBING- ALGORITHM

1. **Start** with an initial state.
2. **Evaluate** the current state using a heuristic function.
3. **Generate** all neighboring states.
4. **Compare** the heuristic values of all neighbors.
5. **Move** to the neighbor with the best heuristic value.
6. **Repeat** the process.
7. **Stop** when:

- Goal state is reached, or
- No neighboring state has a better heuristic value.

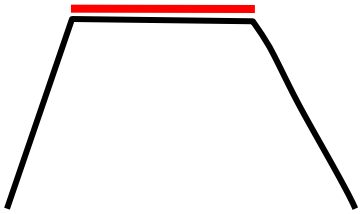
HILL CLIMBING:

- Local maximum: A state that is better than all of its neighbors, but not better than some other states far away.



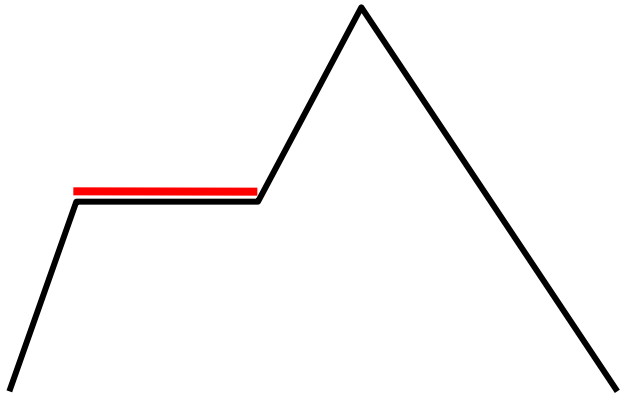
HILL CLIMBING:

- **Plateau:** A flat area of the search space in which all neighboring states have the same value.



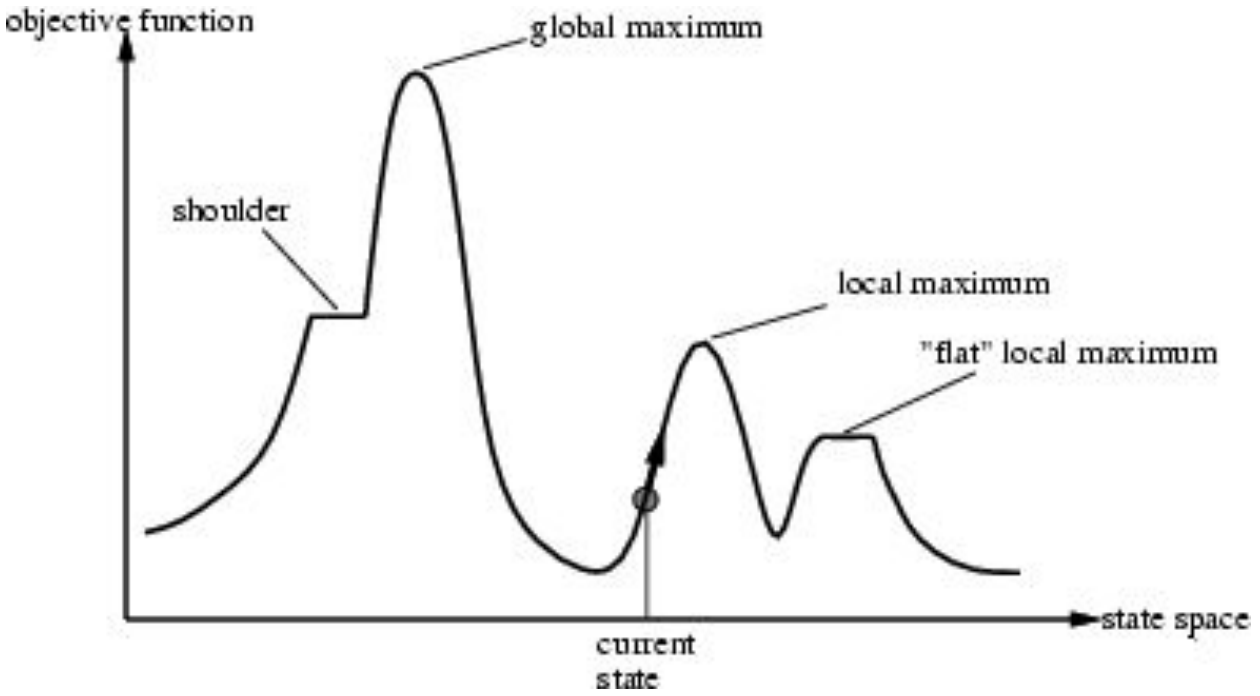
HILL CLIMBING:

- **Shoulder:** It is a plateau region which has an uphill edge.



HILL-CLIMBING SEARCH

- **Problem:** depending on initial state, can get stuck in local maxima



HILL CLIMBING Example : Traveling Salesman Problem (TSP)

Problem Statement

The **Traveling Salesman Problem (TSP)** is a well-known optimization problem in Artificial Intelligence and Operations Research.

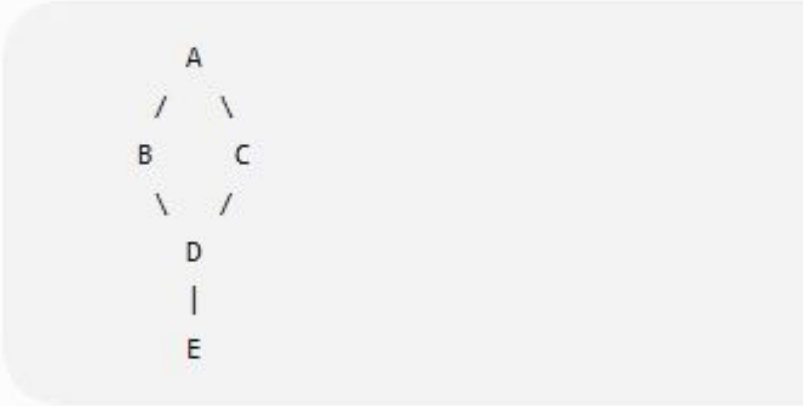
Objective

A salesman must visit **each city exactly once** and return to the starting city while minimizing the **total travel distance (or cost)**.

HILL CLIMBING Example : Traveling Salesman Problem (TSP)

Example

Suppose a salesman has to visit **5 cities.**



Trying "every" possible route is impractical for large numbers of cities. Hill Climbing provides a **fast approximate solution** by continuously improving the current route.

One possible tour is:
A → B → D → E → C → A
The objective is to find the **shortest possible route**.

For **n cities**, the number of possible routes is: (n−1)!

Number of Cities	Possible Routs
5	12
10	181,440
15	43 Billion+
20	60 Quintillion+

HILL CLIMBING: Advantages

1. **Simplicity and Ease of Implementation**: Hill Climbing is a simple and intuitive algorithm that is easy to understand and implement making it accessible for developers and researchers.
2. **Versatility**: The algorithm can be applied to a wide variety of optimization problems, including those with large search spaces and complex constraints.
3. **Efficiency in Finding Local Optima**: Hill Climbing is often highly efficient at finding local optima making it a suitable choice for problems where a good solution is required quickly.
4. **Customizability**: The algorithm can be easily modified or extended to incorporate additional heuristics or constraints allowing for more tailored optimization approaches.

HILL CLIMBING: APPLICATIONS

1. **Path finding:** It is used in AI systems that need to navigate or find the shortest path between points such as in robotics or game development.

2. **Optimization:** It can be used for solving optimization problems where the goal is to maximize or minimize a particular objective function such as scheduling or resource allocation problems.

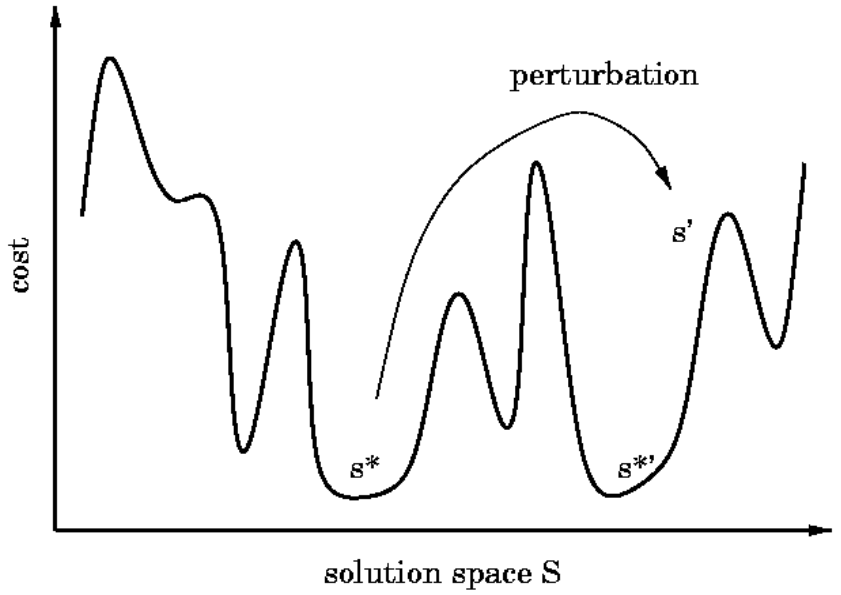
3. **Game AI:** In certain games, AI uses hill climbing to evaluate and improve its position relative to an opponent's.

4. **Machine Learning:** It is sometimes used for hyperparameter tuning where the algorithm iterates over different sets of hyperparameters to find the best configuration for a machine learning model.

Iterated Local Search

Iterated Local Search (**ILS**) is a term in applied mathematics and computer science defining a modification of local search or hill climbing methods for solving discrete optimization problems.

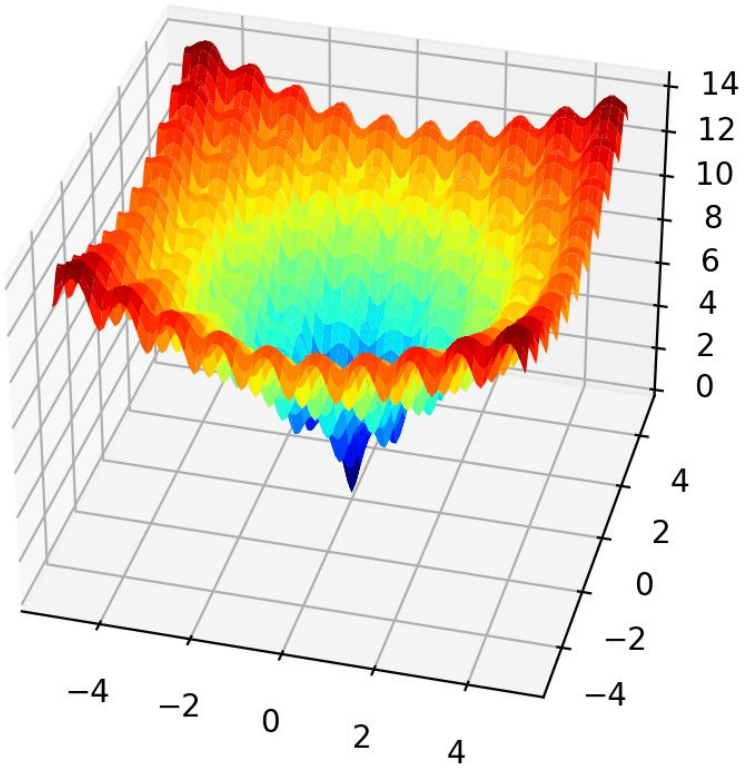
Local search methods can get stuck in a local minimum, where no improving neighbors are available.



Iterated Local Search

1. **Iterated Local Search** is a simple extension of the basic local search algorithm.
2. The local search algorithm is **executed multiple times**.
3. Each execution starts from a **different randomly generated initial solution**.
4. Every run is **independent** of the previous runs.
5. If one run gets stuck in a **local optimum**, the algorithm restarts with a new initial configuration.
6. The **best solution** among all runs is selected as the final solution.
7. It increases the chance of finding a better solution but **does not learn** from previous searches.

Iterated Local Search



Iterated Local Search : Algorithm

1. **Generate an initial solution** randomly or using a heuristic.
2. **Apply a Local Search algorithm** (e.g., Hill Climbing) to improve the solution until a local optimum is reached.
3. **Store** this solution as the current best solution.
4. **Perturb** (slightly modify) the best solution to escape the local optimum.
5. **Apply Local Search** again on the perturbed solution.
6. **Compare** the new solution with the current best solution.
7. **Update** the best solution if the new solution is better.
8. **Repeat** Steps 4–7 until the stopping condition is met.
9. **Return** the best solution obtained.

Iterated Local Search :Advantage

- Escapes Local Optima
- Better Solution Quality
- Simple Implementation
- Low Memory Usage
- Flexible
- Efficient
- Less Sensitive to Initial Solution
- Customizable
- Balanced Search

Iterated Local Search :Real-World Applications

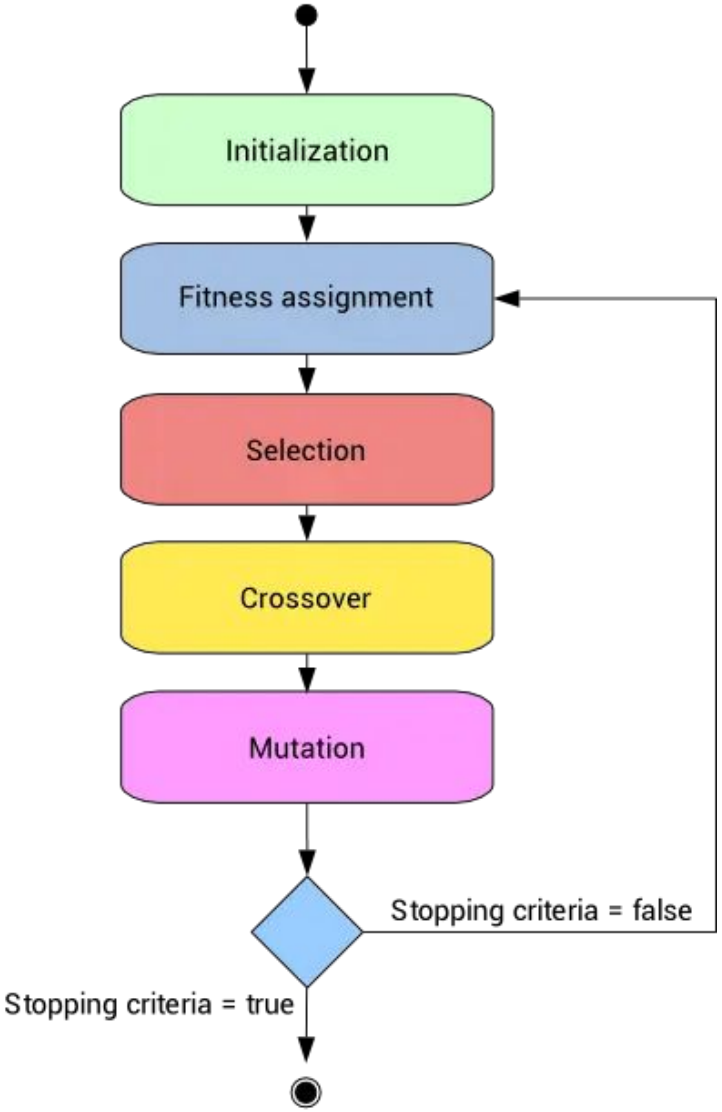
1. Traveling Salesman Problem (TSP)
2. Job Scheduling
3. Vehicle Routing
4. Timetable Generation
5. Resource Allocation
6. Network Optimization

Genetic Algorithm

A **Genetic Algorithm** (GA) is a population-based evolutionary optimization technique inspired by the principles of natural selection and genetics.

It works by **iteratively** evolving a population of candidate solutions using biologically motivated operators such as **selection**, **crossover** and **mutation** to find optimal or near-optimal solutions to complex problems where traditional optimization techniques are ineffective.

Genetic Algorithm



Genetic Algorithm

Term	Meaning
Population	Collection of possible solutions
Chromosome	One complete solution
Gene	One element of a chromosome
Fitness Function	Measures how good a solution is
Selection	Selects suitable parents
Crossover	Combines two parents
Mutation	Randomly changes genes
Generation	One complete cycle of the algorithm

Genetic Algorithm

Genetic_Algorithm()

1. Generate an initial population randomly.
2. Evaluate the fitness of every chromosome.
3. Repeat:
    a. Select parents based on their fitness.
    b. Perform crossover on selected parents.
    c. Apply mutation to offspring.
    d. Evaluate the fitness of offspring.
    e. Create the next generation.
4. Until stopping condition is satisfied.
5. Return the best chromosome.

Genetic Algorithm
1. **Initialization**

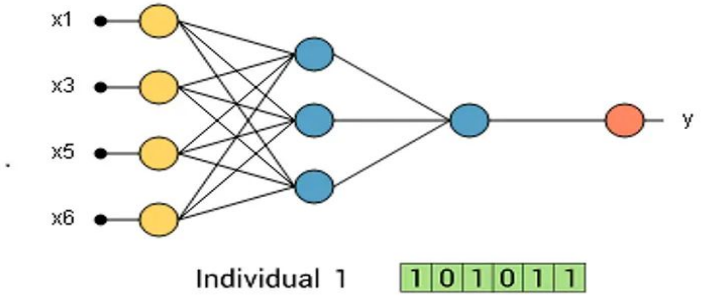
The first step is to create and initialize the individuals in the population. Since the genetic algorithm is a stochastic optimization method, we randomly initialize the genes of all individuals.

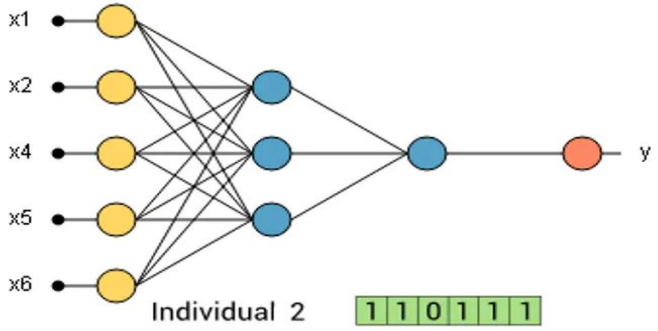
To illustrate this operator, consider a predictive model represented by a neural network with 6 possible features. If we generate four individuals, we have four different neural networks with random features.

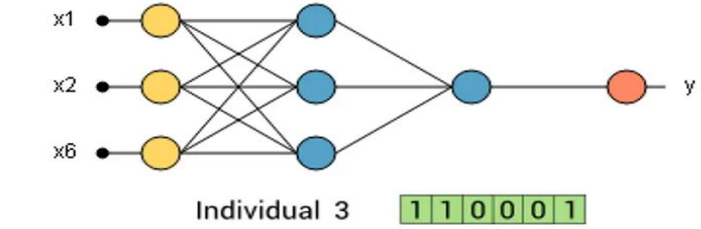
Genetic Algorithm

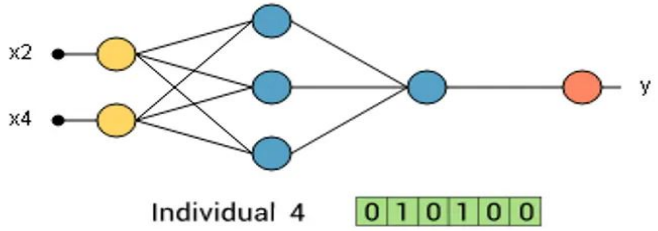
The figure illustrates this population. As we can see, each individual is represented by 6 binary genes.

Each positive gen means that the corresponding feature is included in the neural network.









Genetic Algorithm : 2. **Fitness Assignment**

Fitness assignment operator:

1. After the initialization, we need to assign a fitness value to each population member.
2. We train each neural network with the training instances and then evaluate their error with the selection instances.
3. A significant selection error means low fitness. Individuals with higher fitness or lesser selection error are more likely to be selected for recombination.
4. The most used method for fitness assignment is known as a *rank-based fitness assignment*.

With this method, the selection errors of all the individuals are sorted.

The table depicts the selection error, the rank, and the corresponding probability of being selected (fitness) of each individual in our example.

	Selection error	Rank	Probability
Individual 1	0.9	1	0.1
Individual 2	0.6	3	0.3
Individual 3	0.7	2	0.2
Individual 4	0.5	4	0.4

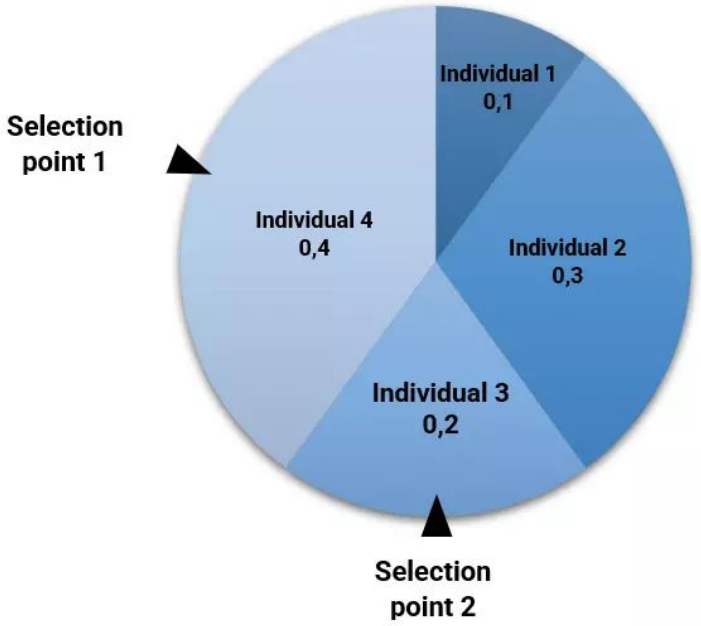
$$P_i = \frac{R(i)}{\sum_{j=1}^{N} j}$$

$$\Phi(i) = R(i) \qquad where \quad i = 1, \ldots, N.$$

Genetic Algorithm : 3. **Selection**

- After a fitness assignment has been performed, the selection operator chooses the individuals that recombine for the next generation.
- The individuals most likely to survive are those more fitted to the environment. Therefore, the selection operator selects the individuals according to their fitness level. The number of selected individuals is N/2, being N the population size.

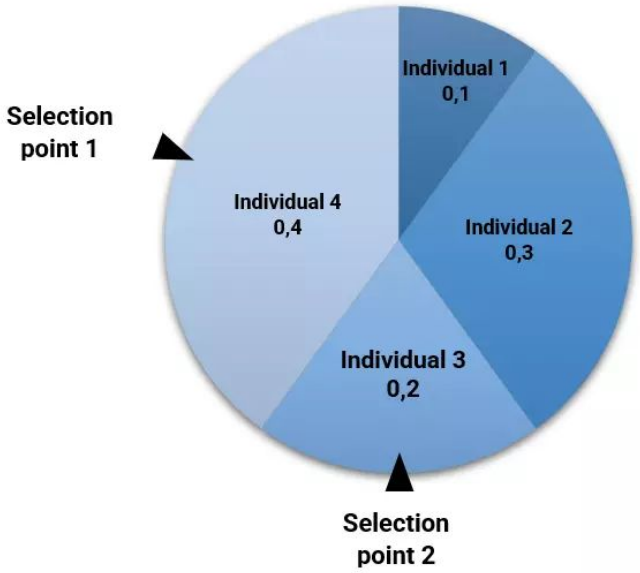
As we can see, the fittest individual (#4) is the one who has the most significant area (40%). Conversely, the least fitted individual (#1) has the smallest area (10%).



Genetic Algorithm3. **Selection**

In this case, neural network #4 has been selected by elitism, and #3 has been selected by the roulette wheel. Note that, although individual #2 has more fitness than #3, it has not been selected due to the stochastic (Ranmdpmness or probability based) nature of the genetic algorithm.

Here, the number of selected individuals is half of the population size.



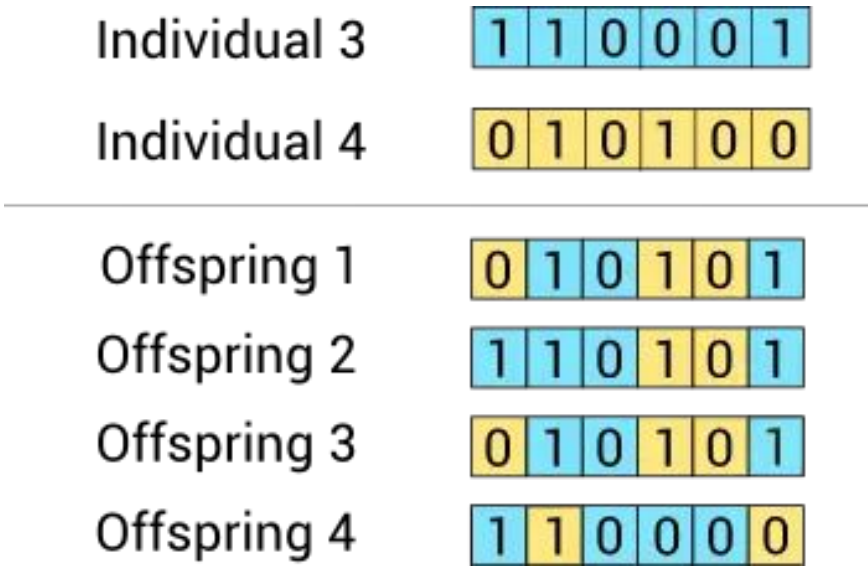
Genetic Algorithm : 4. Crossover operator

Once the selection operator has chosen half of the population, the crossover operator recombines the selected individuals to generate a new population.

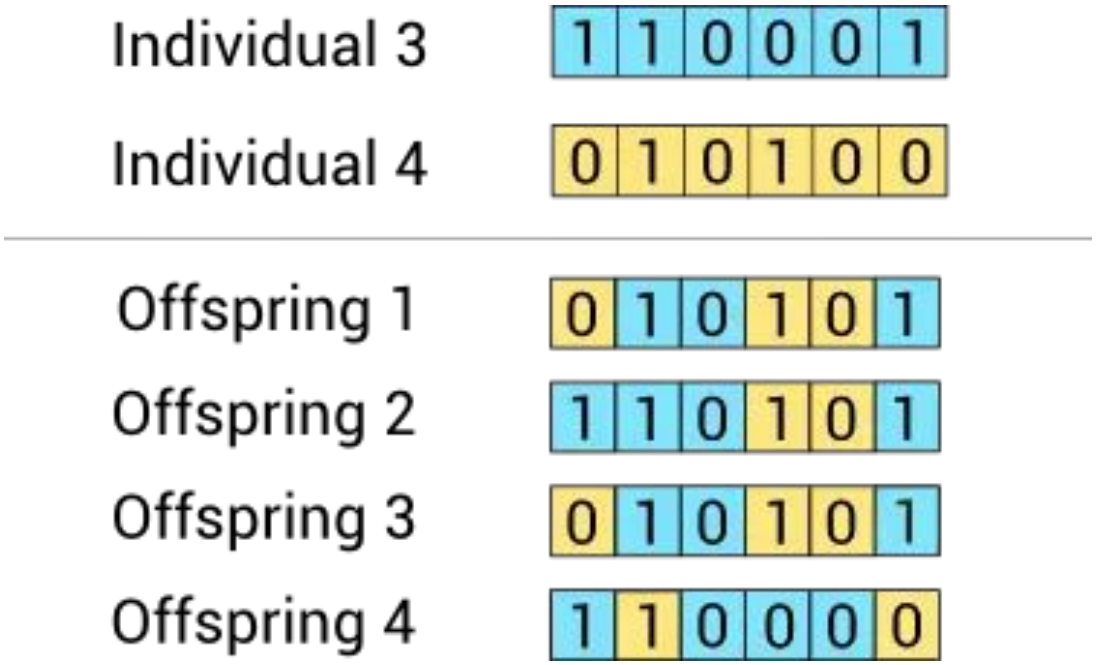
This operator picks two individuals randomly from the previously selected and combines their features to get four offspring for the new population until the new population is the same size as the old one.

The *uniform crossover* method decides whether each offspring's features come from one parent or another.

The following figure illustrates the uniform crossover method for our example.



Genetic Algorithm :4.Crossover operator



Here we have generated four offspring from two parents. Some features of each neural network correspond to one ancestor and others to the other.

In this case, the parents are directly replaced by the offspring. In this way, the population size remains constant.

Genetic Algorithm :5. Mutation operator

The crossover operator can generate offspring that are very similar to the parents. This might cause a new generation with low diversity.

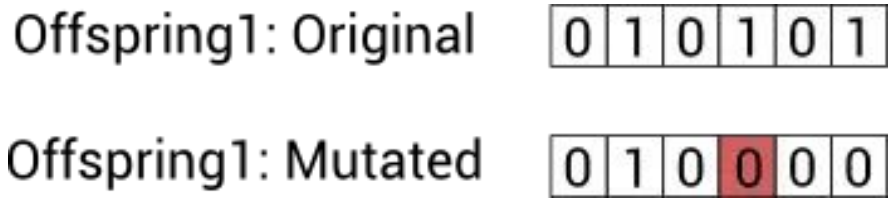
The mutation operator solves this problem by randomly changing the value of some features in the offspring.

We generate a random number between 0 and 1 to decide if a feature is mutated. If the random number is lower than the mutation rate value, the operator flips that variable.

The following image shows the mutation of one offspring of the new generation.

Genetic algorithm mutation

As we can see, the fourth input of the neural network has been mutated.



Genetic Algorithm:Process and results

The whole fitness assignment, selection, recombination, and mutation process is repeated until a stopping criterion is satisfied.

Each generation will likely adapt more to the environment than the old one.

Genetic Algorithm

TSP Using Genetic Algorithm

Objective: Find the shortest route that visits every city exactly once and returns to the starting city.

Steps:

1. **Initial Population:** Generate different random routes.
2. **Fitness Function:** Calculate the total distance of each route.

   Fitness=Total Distance
3. **Selection:** Select shorter routes as parents.
4. **Crossover:** Combine two parent routes to create new routes.
5. **Mutation:** Randomly swap two cities to create variation.
6. **Repeat:** Continue for several generations.
7. **Final Solution:** Select the route with the **minimum total distance**.

Genetic Algorithm

Applications

1. **Traveling Salesman Problem**
2. Job Scheduling
3. Vehicle Routing
4. Feature Selection
5. Network Optimization
6. Robotics
7. Resource Allocation
8. Engineering Design
9. Machine Learning
10. Image Processing

Genetic Algorithm

Advantages of Genetic Algorithm

1. Works with a **population of solutions**.
2. Can explore multiple regions of the search space simultaneously.
3. Less likely to get trapped in a single local optimum.
4. Suitable for **large and complex optimization problems**.
5. Does not require gradient information.
6. Can solve both **discrete and continuous optimization problems**.
7. Flexible and applicable to many real-world problems.

Genetic Algorithm

Disadvantages

1. Can be computationally expensive.
2. Requires proper selection of parameters such as:

   Population size

   Crossover probability

   Mutation probability

   Number of generations

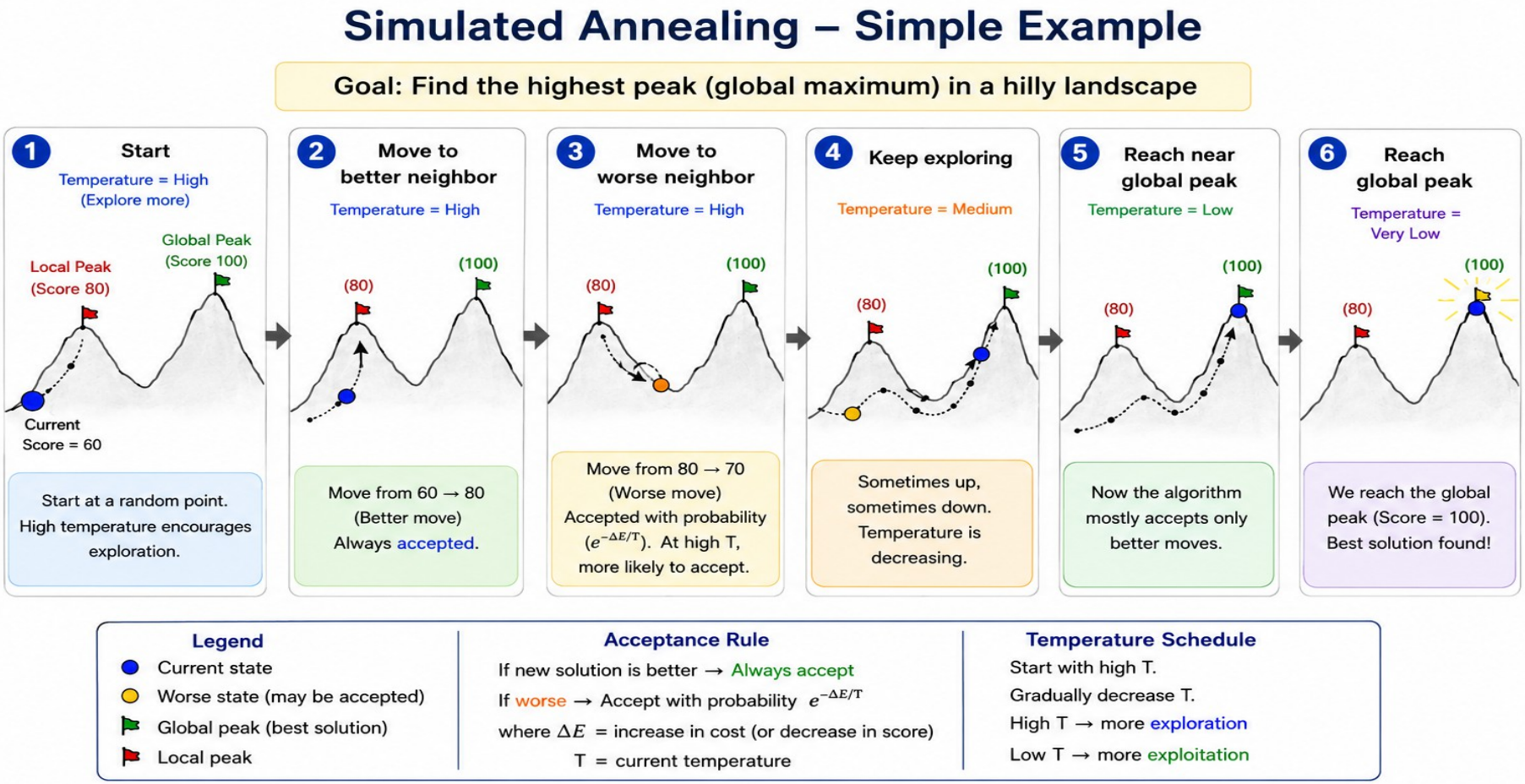
2. Does not always guarantee the global optimum.
3. May converge prematurely.
4. Fitness calculation can be expensive for complex problems.

SIMULATED ANNEALING

Simulated Annealing (SA):-

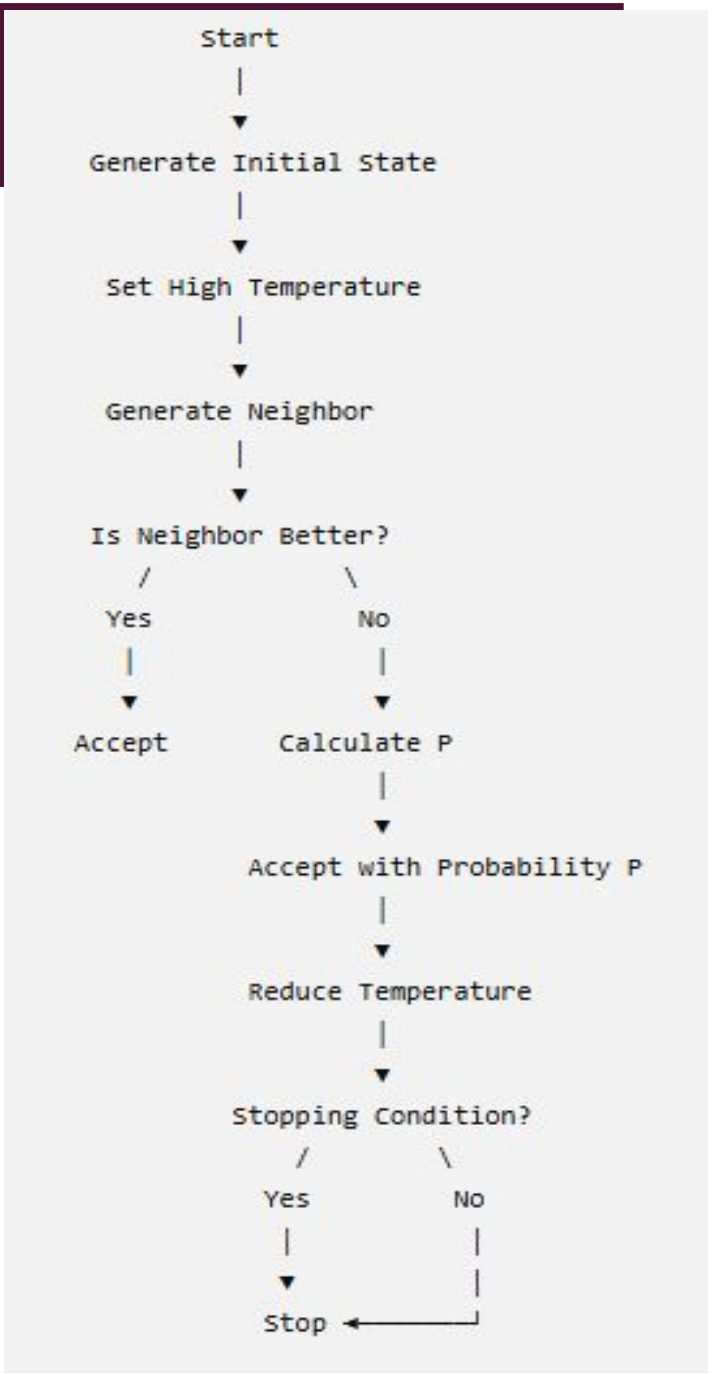
Simulated Annealing is a local search optimization algorithm inspired by the process of annealing in metallurgy, where a material is heated and then slowly cooled to obtain a stable structure. Unlike Hill Climbing, Simulated Annealing can sometimes accept a worse solution to escape from a local optimum.

SIMULATED ANNEALING



SIMULATED ANNEALING

1. Start with an initial solution.
2. Set initial temperature T.
3. Repeat:
    a. Generate a neighboring solution.
    b. Calculate the change in cost ΔE.
    c. If the new solution is better:
        Accept it.
    d. Else:
        Accept it with probability
        P = e^(-ΔE/T)
    e. Reduce the temperature.
4. Stop when temperature becomes very low or another stopping condition is reached.
5. Return the best solution found.



Example: TSP

Suppose the current route is:

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$$

Total distance:

$$100 \text{ km}$$

A neighboring route is generated by swapping two cities:

$$A \rightarrow C \rightarrow B \rightarrow D \rightarrow A$$

New distance:

$$105 \text{ km}$$

The new route is worse.

Hill Climbing

❌ Reject it immediately.

Simulated Annealing

It may **accept the worse route with some probability**.

Why?

Because this move might allow the algorithm to reach a much better route later.

$$100 \text{ km}$$

$$\downarrow$$

$$105 \text{ km} \leftarrow \text{temporarily worse}$$

$$\downarrow$$

$$90 \text{ km} \leftarrow \text{much better}$$

Thus, SA can escape local optima.

SIMULATED ANNEALING

Advantages

1. Can escape **local optima**.
2. Better exploration than Hill Climbing.
3. Useful for large and complex optimization problems.
4. Simple concept and implementation.
5. Requires relatively little memory.

SIMULATED ANNEALING

Disadvantages

1. Does not guarantee the global optimum.
2. Performance depends on the **initial temperature** and **cooling schedule**.
3. Can be slow if temperature decreases very gradually.
4. Can converge to a poor solution if temperature decreases too quickly.

SIMULATED ANNEALING

Applications

1. **Traveling Salesman Problem (TSP)**
2. Scheduling
3. Routing
4. Resource Allocation
5. Circuit Design
6. Image Processing
7. Machine Learning Optimization
8. Network Optimization

BEAM SEARCH

Beam Search is a heuristic search algorithm used in Artificial Intelligence to efficiently explore large search spaces by selecting only the most promising nodes at each level.

Instead of expanding every possible path like Breadth-First Search, it keeps a limited number of best nodes based on heuristic values, making the search faster and more memory-efficient.

- Uses a **fixed beam width** to limit node expansion
- Balances efficiency and solution quality
- Widely used in NLP, speech recognition, and path finding tasks

BEAM SEARCH: **Characteristics**

Width of the Beam (W): This parameter defines the number of nodes considered at each level. The beam width W directly influences the number of nodes evaluated and hence the breadth of the search.

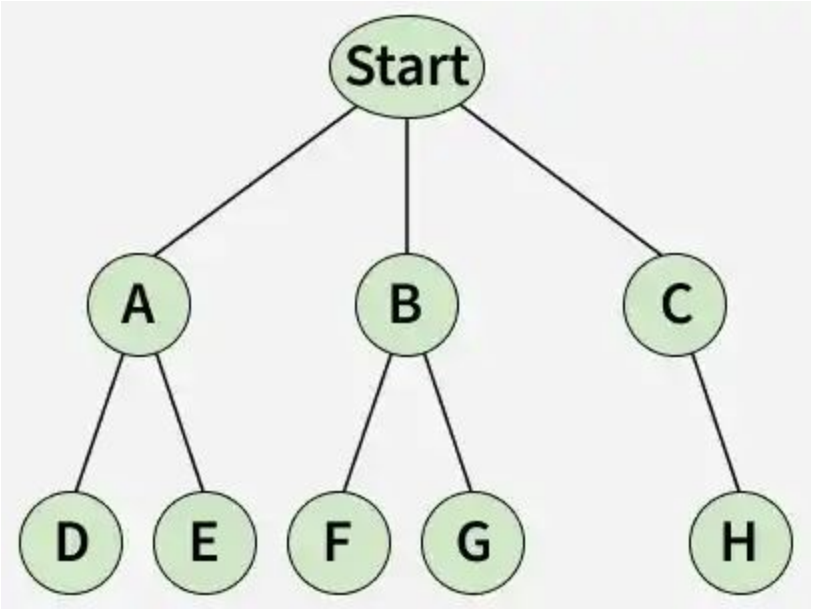
Branching Factor (B): If B is the branching factor, the algorithm evaluates W×B nodes at every depth but selects only W for further expansion.

Completeness and Optimality: The restrictive nature of beam search, due to a limited beam width, can compromise its ability to find the best solution as it may prune potentially optimal paths.

Memory Efficiency: The beam width bounds the memory required for the search, making beam search suitable for resource-constrained environments.

BEAM SEARCH

Beam Search works by selecting only a limited number of the best nodes at each level based on heuristic values. Suppose the beam **width W=2**, which means only the 2 most promising nodes are selected for further expansion at every level.



BEAM SEARCH

Step 1: Start from the initial node Start.

Step 2: Generate all successor nodes: A, B, and C.

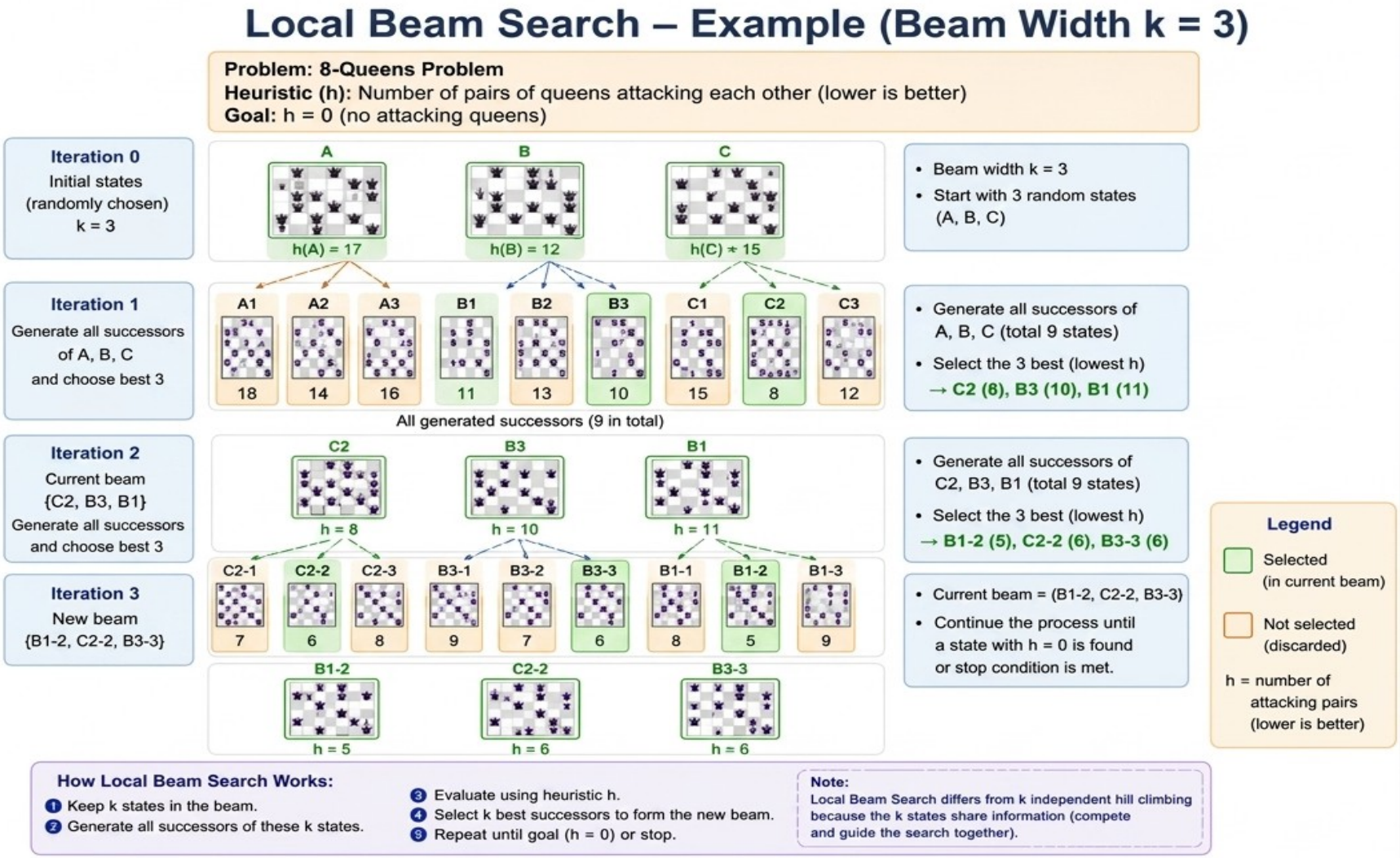
Step 3: Evaluate the nodes using heuristic values and select the best W=2 nodes. Suppose A and B are selected, while C is discarded.

Step 4: Expand the selected nodes:
A → D, E
B → F, G

Step 5: Again evaluate the generated nodes (D, E, F, G) and keep only the best 2 nodes for further exploration.

Step 6: Repeat the process until the goal node is reached.

Local Search: Local Beam Search



BEAM SEARCH

Advantages

1. Efficient for large search spaces because it expands only limited nodes
2. Flexible since beam width and heuristic functions can be adjusted
3. Scalable for complex problems with many possible solution paths
4. Requires less memory than exhaustive search method

Limitations

1. May miss the optimal solution because some paths are discarded early
2. Performance depends heavily on the quality of the heuristic function
3. Small beam width can remove potentially useful solutions
4. Large beam width increases computation time and memory usage

BEAM SEARCH

Applications

1. Used in machine translation, text generation, and speech recognition to predict the most likely sequence of words.
2. Helps robots find efficient paths and navigate through environments.
3. Used in strategic games where exploring every possible move is computationally expensive.

ONLINE SEARCH & UNKNOWN ENVIRONMENTS

- **Definition :-Online Search** is a search technique in AI where an agent explores and learns about an **unknown environment** while searching, making decisions based on current observations instead of complete prior knowledge.
- **Online Search** is a search strategy in which the AI agent **does not know the complete environment in advance**. The agent discovers information while interacting with the environment and decides its next action based on what it has learned so far.

ONLINE SEARCH & UNKNOWN ENVIRONMENTS

Unknown Environment

An **unknown environment** is one where:

1. The agent has no complete map or model of the environment.
2. The results of actions are not fully known beforehand.
3. The agent learns by exploring and observing.
4. Decisions are made step by step.

ONLINE SEARCH & UNKNOWN ENVIRONMENTS

How Online Search Works

1. Start from the initial state.
2. Observe the current environment.
3. Choose the best available action.
4. Execute the action.
5. Observe the new state and update knowledge.
6. Repeat until the goal is reached.

ONLINE SEARCH & UNKNOWN ENVIRONMENTS

Example

A robot is placed in an unfamiliar building and must find the exit.

- It does not know the layout initially.
- It explores corridors and rooms.
- It remembers visited locations.
- Eventually, it finds the exit through exploration.

ONLINE SEARCH & UNKNOWN ENVIRONMENTS

Applications

- Robot navigation
- Autonomous vehicles
- Video game AI
- Web crawling
- Exploration and rescue missions

1. Case Study: Warehouse Robots (Amazon Kiva)

Overview

Amazon Kiva (now called Amazon Robotics) uses autonomous mobile robots to move shelves of products to human workers, instead of workers walking through the warehouse.

How AI is Used

- Robot navigation using AI algorithms.
- Path planning and obstacle avoidance.
- Task scheduling and coordination.
- Real-time inventory management.

1. Case Study: Warehouse Robots (Amazon Kiva)

1. How many robots does Amazon have

:-over 1 million robots across its global operations and fulfillment centers.

Fleet and Usage Details

Scale: The company reached the one million milestone across more than 300 facilities worldwide.

Operations: Robotics assist with approximately 75% of all customer orders globally.

Key Models: Specialized systems include mobile drive units like *Hercules* and *Titan*, inventory management systems like *Sequoia*, and AI-powered robotic arms like *Sparrow*, *Robin*, and *Cardinal*.

Inside Amazon's Smart Warehouse:
https://www.youtube.com/watch?v=IMPbKVb8y8s

1. Case Study: Warehouse Robots (Amazon Kiva)

Key Amazon Robot Models

Amazon utilizes specialized robots across different stages of the fulfillment process:

1. **Sequoia:** A containerized inventory system that stacks totes, identifies contents, and delivers goods directly to employee workstations at ergonomic heights.
2. **Sparrow:** An AI-powered robotic arm capable of detecting, selecting, and handling millions of individual, unstructured products before they are packaged.
3. **Robin and Cardinal:** Robotic arms designed to pick, sort, and organize customer packages onto delivery carts based on their final shipping destinations.
4. **Proteus:** Amazon's first fully autonomous mobile robot that can move freely through warehouses without designated restricted zones, working safely alongside humans.

1. Case Study: Warehouse Robots (Amazon Kiva)

AI Techniques

1. Path finding algorithms (A*)
2. Multi-agent systems
3. Machine learning
4. Computer vision

Benefits

1. Faster order processing.
2. Reduced human effort.
3. Higher warehouse efficiency.
4. Fewer picking errors.
5. 24×7 operation.

Case Study

- Self-driving cars

  https://youtube.com/shorts/H52u9CBZIL0?si=lwAKlrzNRzz98Yl8

- Logistics and Routing: Traveling Salesman Problem

  https://www.youtube.com/watch?v=1pmBjIZ20pE

- Google DeepMind

  https://youtube.com/shorts/iDL0-UKMxjo?si=BIINdpfGwlRIlhtk

- AI for Energy Efficiency in Data Centers

  https://youtube.com/shorts/S1gdNOlF23Y?si=ElEueb-a5zEXsOms

THANK YOU